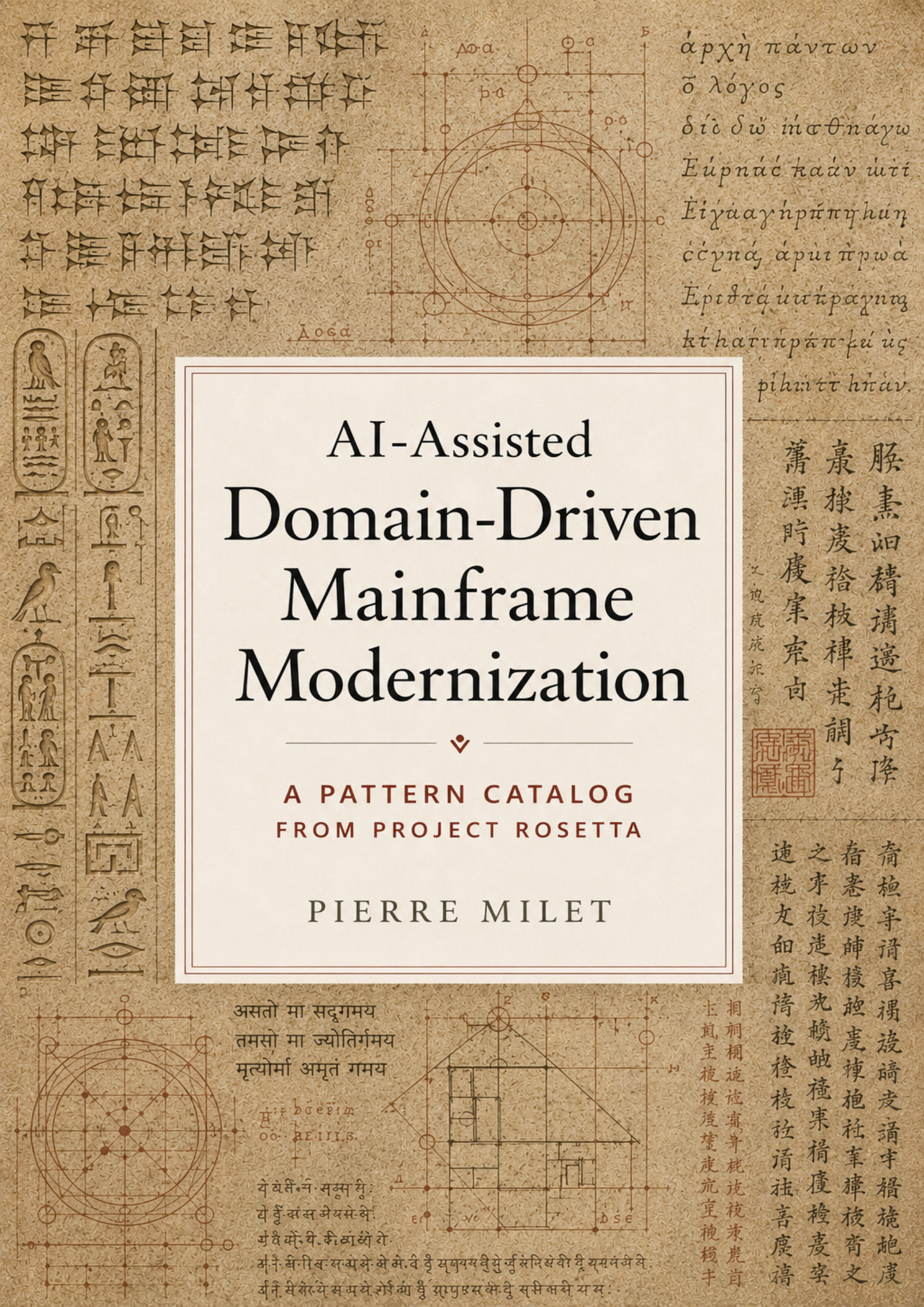

AI-Assisted Domain-Driven Mainframe Modernization

A PATTERN CATALOG FROM PROJECT ROSETTA

PIERRE MILET

Contents

AI-Assisted Domain-Driven Mainframe Modernization

A Pattern Catalog from Project Rosetta

Twenty-three patterns where Domain-Driven Design meets AI-assisted blackfield mainframe modernization.

How to read this catalog

This catalog documents twenty-three patterns I've discovered building Project Rosetta, a research prototype for AI-assisted modernization of COBOL/CICS mainframe systems. It follows the structure popularised in software engineering by the Gang of Four's *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994) — each pattern named, situated in its context, articulated as a solution to a recurring problem with its forces, consequences, and relationships to other patterns. The form is established; what's specific here is the territory.

The name *Project Rosetta* is deliberate. The Rosetta Stone made ancient Egyptian readable by presenting the same decree in three scripts — hieroglyphic, demotic, Greek — and allowing meaning to be recovered through triangulation between representations. Project Rosetta does the same with legacy mainframe systems: the COBOL source, the structural graph, the domain ontology, and the intermediate representation are different representations of the same system, and meaning is recovered through triangulation between them. The legacy is not abandoned and rewritten; it is read, understood, and translated — with the running system as oracle and the substrates as the parallel inscriptions that let understanding survive the translation.

The catalog has a thesis: **AI-assisted mainframe modernization is, at its core, a Domain-Driven Design activity at scale**. The scale is what makes the work hard — and what makes AI assistance necessary. Strategic design (recovering the domain, identifying bounded contexts, establishing ubiquitous language) and tactical design (modelling aggregates, domain events, handlers) are hard, and are the work the modernization actually performs. AI agents accelerate the mechanical parts; humans direct the strategic ones; the boundary between agentic and human work is itself an architectural commitment that needs explicit design.

These are the patterns I've discovered while building Project Rosetta — the ones I think make sense for treating true blackfield legacy modernization through a DDD lens and with AI assistance. Some

are validated; some are in active construction; some are projected from validated principles. None is definitive. Other practitioners will find other patterns. These are my own. And one operational note: fewer patterns, denser articulation. More is not better. Less is better.

DDD has been applied extensively to greenfield development and to incremental refactoring of in-life systems. It has been applied less systematically to the territory where it is most needed: legacy mainframe systems decades old, written in languages whose original specifications were never written down, where the domain has drifted from whatever it once was and accumulated three definitions of every important concept. *Brownfield* is the established term for software work on existing systems; this catalog uses **blackfield** for the harder case — systems where the original engineers have moved on, the documentation is lost, and the domain knowledge has decayed into operational dialect, fragmented across modules. Brownfield is "existing systems you understand." Blackfield is "existing systems where understanding itself has to be recovered." Mainframe modernization is blackfield at its hardest: COBOL written in the eighties and nineties, CICS transactions evolved across decades, JCL and copybooks woven through millions of lines of code that the original team has long since left. AI assistance changes the economics of this work — what was previously infeasible at scale becomes tractable. This catalog is a contribution to that practice.

The patterns are calibrated to mainframe COBOL/CICS modernization, where Rosetta has been validated. Many patterns generalise to other legacy stacks — the compiler principle, the IR contract, twin verification, the harness state machine, the control plane — apply to any AI-assisted modernization where deterministic and probabilistic work need clear separation. Other patterns are mainframe-specific in their implementation — the linguistic cues for slice discovery (`XCTL`, `START TRANSID`, `EXEC CICS LINK`), the Raincode-compiled COBOL container as oracle, the CICS pseudo-conversational boundaries — and would need adaptation for other legacy stacks (PL/I, RPG, Adabas Natural, Java legacy). Each pattern body indicates where the underlying principle is universal versus where the implementation is mainframe-specific.

The catalog is a mix of three kinds of patterns. **Original patterns** describe practices that emerged from building Project Rosetta and that the field has not yet articulated as patterns in their own right — the compiler principle for agentic workflows, the intermediate representation as contract between analysis and generation, the heuristic catalog as queryable infrastructure, architecture documentation as pluggable emitter. **Adapted patterns** apply established practices to mainframe modernization with explicit recalibration — Martin Fowler's strangler fig at bounded context granularity, Eric Evans' anti-corruption layer as migration infrastructure, the modular monolith as transitional architecture. **DDD re-articulations** recast canonical DDD patterns through the lens of legacy mainframe modernization — the domain ontology as substrate, commands and events as logical boundaries, aggregate boundaries recovered from structural and data-layer evidence. Each pattern indicates its lineage explicitly: where the principle originates, what this catalog adds, what's calibrated to mainframe specifically. The catalog stands on shoulders — Eric Evans, Vaughn Vernon, Martin Fowler, Sam Newman, Alberto Brandolini, Cyrille Martraire, Birgitta Böckeler, Charity Majors, Nick Tune, Uberto Barbini, Kent Beck, Matthew Skelton, Manuel Pais — and names them as it builds.

I write for two audiences. **DDD practitioners** will recognise vocabulary they already use — bounded contexts, ubiquitous language, subdomain types, aggregates, domain events, anti-corruption layers — and find them deployed in a domain DDD has rarely entered: mainframe systems that have accumulated decades of legacy code and behaviour, undergoing AI-assisted modernization at scale. **Mainframe**

modernization practitioners less familiar with DDD will encounter the vocabulary deliberately. Where a DDD concept appears for the first time, I provide a brief inline gloss; the glossary at the end gives more complete definitions and pointers to canonical sources. The catalog rewards readers from either community — though differently.

Of the twenty-three patterns, fifteen are validated inside the Rosetta prototype (status: *prototype-validated*). Six are in active construction (status: *in construction*). Two are designed from validated principles but not yet built (status: *designed*). The two *designed* patterns are notable in different ways: Pattern 4 (*Domain Ontology as Independent Substrate*) is the most distinctive contribution of the catalog, and its current status reflects the order in which the prototype's substrates have been built rather than uncertainty about the underlying principle; Pattern 23 (*Completion Criteria as Designed Property of Each Bounded Context*) addresses a load-bearing gap the catalog would otherwise leave unaddressed — the question of when a bounded context is *done* — and stands on architectural reasoning until engagement experience can refine it. None has yet been validated against real customer engagements — that's the next phase, not yet started. I include all three categories deliberately. The reader who wants only validated patterns can filter by status; the reader interested in the architectural reasoning can engage with all of them.

A note on what "prototype-validated" means and doesn't mean. The pattern works inside Rosetta. The pattern has been exercised against representative legacy code. The pattern has not yet been put under load by a real customer engagement at production scale. When the engagements come, some prototype-validated patterns will turn out to need revision; that revision is part of the work the catalog signs up for. The catalog improves through use.

Kent Beck's 3X framework names three distinct modes of software work — Explore, Expand, Extract. Each has its own economics. Explore tolerates high failure rates because each failure teaches; Expand demands rigour because breakages multiply at scale; Extract optimises for stability and efficiency once growth has plateaued. This catalog is in Explore. The principles have held inside the Rosetta prototype; whether they hold against real engagements is the next phase, not yet started. When the engagements come, some patterns will sharpen and earn their way into Expand. Some will turn out to be wrong, or specific to environments I haven't yet encountered, and will be replaced. The next version of this catalog will be honest about what changed, why, and which patterns have moved between modes.

Each pattern follows the same shape. **Status** says where the pattern stands in the prototype today. **Context** describes the situation in which the pattern applies. **Problem** describes the tension or difficulty the pattern addresses. **Forces** describes the factors in conflict. **Pattern** is the solution, articulated as principle. **Consequences** describes what applying the pattern produces — both gains and costs. **Related patterns** points to other entries that depend on, support, or are supported by this one.

The patterns are grouped into five parts that follow the structure of DDD as discipline, extended into the operational territory where modernization meets cutover and coexistence:

- **Part I — Strategic Recovery.** What is the domain, what bounded contexts exist, what is the ubiquitous language. The patterns that recover what the legacy *is* and what it *should be about*.
- **Part II — Tactical Generation.** How each bounded context materialises in modern code. Aggregates, domain events, handlers, scaffolds matched to subdomain type.
- **Part III — Verification.** Validating that strategic and tactical designs preserve behavioural equivalence to the legacy.

- **Part IV — Governance.** Governing the agentic system that performs the recovery, generation, and verification work, and the team structures that operate around it.
- **Part V — Safe Transition and Coexistence.** The disciplined movement of bounded contexts from legacy authority to modernized authority, and the dual-run period during which both sides operate.

The five-part structure mirrors how Rosetta itself organises its stages: Strategic Recovery, Tactical Generation, Verification, Governance, and Safe Transition with Coexistence. The platform and the book share the same conceptual spine.

The grouping is for navigation — patterns interact across groups, and the *Related patterns* sections trace those interactions.

Between Part I and Part II, a short architectural interlude makes explicit two pieces of foundation that operate across the entire catalog: the three-layer recovery architecture, and the source provenance discipline that holds the layers honest. Readers who want the architectural framing before the patterns can read it first; readers who prefer to encounter the patterns first will find the interlude in sequence.

The three-layer architecture (L1 syntactic, L2 semantic intent, L3 architectural target) is a perpendicular lens to the DDD arc. The DDD arc organises the catalog by *what the modernization is doing* — recovering strategy, generating tactics, verifying behaviour, governing the agents. The three-layer architecture organises the substrates the work operates on — what the legacy literally says, what it actually does, how it should be expressed. The same pattern can be located in both schemes: Pattern 3 (The Graph as Projection) lives in Part I's strategic recovery and produces an L1 substrate; Pattern 4 (Domain Ontology) lives in Part I and produces an L2 substrate; Pattern 6 (The Compiler Principle) lives in Part II and operates the L2 $\rightarrow$ L3 transition. The DDD arc is the primary spine of the catalog; the three-layer architecture is the cross-cutting lens that makes the substrate work visible.

A short section names nine antipatterns this catalog is built against. They're not bad practices in the abstract; they're failure modes the field has encountered, and naming them helps clarify what the patterns are correcting.

After the antipatterns, a glossary defines the DDD and modernization terms used throughout the catalog. After that, a final section maps each pattern to the concrete technology that realises it in Rosetta today. The pattern bodies stay abstract because principles outlive implementations. The reference section is a snapshot of what's currently doing the work — it will date faster than the rest of the book.

I came to mainframes as an outsider and have spent 15 years on mainframe modernization projects, with a foundation in DDD practice for .NET work. I learned the COBOL/CICS world the slow way — through migrations done before AI assistance was practical, when modernization was a matter of careful manual work. Project Rosetta is the experiment in what becomes possible when AI assistance changes that economics. The patterns here are reports from inside the experiment, not claims about a finished system.

This is the long version of what I've been writing about on LinkedIn under the LegacyLabs name. The shorter posts and newsletter there were the introduction.

I wrote this catalog partly so I would not have to keep explaining these concepts from scratch. Years of mainframe modernization work teach you that the same simplistic approaches keep being proposed by people who do not have time to hear why they fail. *"The essence of tyranny is the denial of complexity."* Camus had something larger in mind, but the principle applies here too. The patterns in this catalog are my own, accumulated from work I have done; they are also my way of pushing back, in writing rather

than in conversation, against the recurrent assumption that mainframe modernization is straightforward if you just have the right tool.

The catalog will not convince anyone who is already determined not to be convinced. But for the reader who suspects that this work is harder than it is being made out to be, and who wants the vocabulary to articulate why, this is what I have built.

Part I: Strategic Recovery

The patterns in this group address the strategic question that opens any DDD engagement: *what is the domain, and what bounded contexts compose it?* In a greenfield context, strategic design starts from a clean slate; in legacy modernization, it starts from a system that has been running for decades and has its own answers — partial, contradictory, and frequently undocumented. The patterns here recover those answers from the legacy as evidence, and ground them against canonical domain understanding that the legacy alone cannot provide.

Without strategic recovery, generation has nothing to work from and verification has nothing to compare against. Without honest distinction between behavioural recovery (what the legacy does) and ontological grounding (what the domain really is), strategic recovery silently inherits the legacy's accumulated drift.

Pattern 1: Business-Aligned Capability Strategy

Status: prototype-validated.

Context

A mainframe modernization engagement that touches a system spanning decades of accumulated business logic — millions of lines of COBOL, hundreds of CICS transactions, dozens of bounded contexts implementing capabilities the business depends on. Some of these capabilities are strategic differentiators; some are commodity work; some are legacy debt that exists only because no one has authorised retiring it. The modernization team must decide what to modernize, how deeply, and what to leave alone — and these decisions must precede any technical work.

The default frame for modernization is technical: convert COBOL to C#, move from mainframe to cloud, declare victory. This frame skips the question that determines whether the modernization delivers value: which business capabilities deserve which treatment, and why.

Problem

The dominant antipattern in the field is technical-first modernization. Teams treat the legacy code as if it were the specification — every paragraph must be migrated, every transaction must be preserved, every batch job must be replatformed. The result is a brilliant technical transformation that costs millions and delivers a modernized system functionally identical to the legacy. The business strategy isn't served;

opportunities are missed; capabilities that should have been retired survive into the modern system, carrying their maintenance burden forward.

The deeper failure is treating all capabilities as if they were equivalent. A small percentage of transactions typically represents 80% of the value, the traffic, the risk, the maintenance cost — the Pareto distribution applies almost everywhere in mainframe systems. Treating all capabilities with the same modernization discipline produces over-investment in commodity work and under-investment in core. Sometimes the highest-ROI decision is *not* to modernize at all — to retire a capability, replace it with SaaS, or simply turn it off — and a technical-first modernization never surfaces these options because it never asks the question.

Forces

The business has a strategy: differentiate where it matters, achieve parity where it doesn't, retire what no longer earns its place. The legacy mainframe encodes a snapshot of past business decisions, frozen in COBOL since the eighties or nineties. The mismatch between current strategy and encoded snapshot is the territory modernization must navigate. Done well, the modernization closes the gap. Done poorly, it preserves the snapshot in modern syntax.

Investment must be distributed unevenly. Treating all bounded contexts as equal squanders resources in commodity work and starves strategic core. But the distribution cannot be improvised — it must be grounded in evidence about each capability's role, value, volatility, and cost.

Pattern

Before any technical work begins, map the business capabilities the legacy implements. By "technical work" I mean identifying feature slices to extract (Pattern 5), deciding which contexts deserve which architecture (Pattern 8), and generating scaffolds for them (Patterns 6, 7, 9). Each of these is downstream of strategic understanding. For each capability, consider four dimensions:

- **Strategic value** — is this capability a differentiator (something the business does better than competitors, where investment compounds) or commodity (something everyone in the industry does, where parity is sufficient)?
- **Functional volatility** — does the business logic for this capability change frequently (regulatory updates, market shifts, product evolution) or has it been stable for years?
- **Operational criticality** — what is the cost of failure? A real-time payment authoriser failing is catastrophic; an internal reporting batch failing is recoverable.
- **Current maintenance cost** — how much does this capability cost to maintain today, in developer hours, infrastructure, and operational incidents?

These dimensions are the vocabulary for a conversation between business and architecture. They are not the inputs to a decision tree. There is no fixed mapping from the four dimensions to a single recommended strategy; the dimensions inform judgment, not algorithm.

Six strategies recur across engagements as anchoring options. An architect classifying a specific capability considers all four dimensions and selects the strategy that fits — sometimes cleanly, often with trade-offs that have to be made deliberately:

- **Core differentiator + volatile** → cloud-native rewrite, deep DDD investment, full hexagonal architecture, dedicated team. This is where the modernization must excel.
- **Core differentiator + stable** → AI-assisted migration with semantic preservation. The behaviour matters; the architecture can stay conservative. Twin Verification (Pattern 13) is critical here.
- **Supporting subdomain** (necessary for the core to function but not differentiating in itself — order processing, inventory management, audit logging) → AI-assisted migration with hybrid scaffolding (Pattern 8). Investment is proportionate: more than commodity, less than core. The hybrid scaffold (Wolverine with lightweight domain models) usually fits.
- **Commodity + stable** → lift & shift, or replace with SaaS / managed service. Investment beyond parity is waste.
- **Commodity + high maintenance** → retire, consolidate with another capability, or externalize. The maintenance cost itself signals that this capability is overdue for elimination.
- **Obsolete / no real usage** → turn off. This is more frequent than expected. Systems running for decades accumulate code paths that no one exercises but no one has authorised retiring. Identifying and shutting these down is often the highest-ROI work of the modernization.

Some capabilities will fit cleanly into one strategy. Many will sit between strategies, and the architect must decide which trade-offs to prioritize for that capability. A capability might be commodity in business terms but have maintenance cost high enough to justify retiring it rather than replacing it with SaaS. A capability might be a core differentiator with stable volatility but operational criticality high enough to justify the deeper DDD investment of the volatile cell. The criticality dimension in particular rarely selects a strategy on its own; it determines the rigor of the chosen strategy's execution. A critical capability in the lift & shift cell gets a more careful lift & shift than a non-critical one.

The classification is not done by the technical team alone. It requires business stakeholders — product owners, capability leads, finance partners — collaborating with architects to map the capabilities, consider the four-dimensional profile, and agree on the strategy per capability. Workshop techniques apply: Event Storming (Brandolini) for capability discovery, Wardley Mapping for value/commodity classification, Domain Storytelling (Hofer & Schwentner) for behavioural understanding.

The deeper purpose of these workshop techniques is not just to extract information from stakeholders — it is to build a shared understanding across roles that rarely think together. Product owners articulate what the business is trying to do; engineers articulate what the system actually does; operators articulate what fails in production; finance partners articulate what each capability costs. Each role holds part of the truth. The capability map is the artifact that emerges when those partial truths are reconciled, but the reconciliation is the real work. Facilitating the conversation — neutrally, with structure, with patience — is a discipline of its own, and one the modernization team has to take seriously. A capability map produced by architects alone, however technically sophisticated, will be commercially wrong. The map's value comes from the shared understanding it represents, not from the document itself.

The output is a capability map: every business capability in scope, with its four-dimensional profile, its chosen modernization strategy, and the rationale that connects business strategy to technical decision. This map is the primary input to every subsequent pattern. Slice discovery (Pattern 5) derives slices within capabilities, prioritised by strategic value. Tier-aware scaffolding (Pattern 8) operationalises the chosen strategy per bounded context. Rollout and cutover (Pattern 21) sequences the migration to deliver business value early.

The capability map and the team topology that owns it are inseparable. Which team owns which capability

shapes how the capability is recovered, modernized, and operated. Conway's Law — Melvin Conway's 1968 observation that organisations design systems whose structure mirrors their communication structure — applies whether the modernization acknowledges it or not. Pattern 20 (*Team Topology and Bounded Context Alignment*) addresses this directly, and the capability mapping work described here should be done with team-ownership questions visible alongside the four dimensions.

Consequences

Modernization investment becomes proportionate to business value. The team spends deep effort where it matters and light effort where it doesn't. Capabilities that should be retired are identified and shut down rather than migrated by default. The modernization delivers business value, not just technical transformation.

The capability map becomes a strategic artifact in its own right. Updated as business strategy evolves, it provides a continuous reference for "what are we doing and why." Future modernizations of adjacent systems have a precedent to follow. Business stakeholders have a shared vocabulary with the technical team for discussing what's being built and what isn't.

The cost is the discipline of doing the strategic work before the technical work. Many teams skip this because it feels slower than starting to code. The teams that skip it pay later — in over-engineered commodity work, in under-engineered core domain, in migrated capabilities that should have been retired. The work pays back; it must be done.

The capability map also exposes uncomfortable truths. Some capabilities that the business has been treating as strategic differentiators turn out to be commodity. Some that have been ignored as commodity turn out to be where genuine differentiation lives. The four-dimensional classification surfaces these mismatches and forces conversations the organization may have been avoiding.

The framing here is deliberately judgment-based rather than algorithmic. A more rigid mapping — sixteen cells from four binary dimensions, one strategy per cell — would look more rigorous but would mislead. Real capabilities don't fall into binary buckets. The strategic value of a capability is rarely a clean yes-or-no; volatility comes in degrees; criticality has thresholds the architect has to define for the engagement. A decision tree would force false precision; the dimensional vocabulary lets the conversation stay honest about where the judgment actually lives. The architect is equipped, not constrained.

The notion that *modernization is not code translation — it is reconstructing the business, segmenting capabilities by value and volatility, and designing the transition path* is what this pattern operationalises. Wardley Mapping (Wardley, 2018) provides the conceptual foundation for the strategic-value dimension. Eric Evans' subdomain classification (Evans, 2003) — core, supporting, generic — is the canonical DDD anchor; this pattern extends Evans by adding the volatility, criticality, and maintenance-cost dimensions, and by making *retirement* an explicit option, which standard DDD treatments do not.

Related patterns

Pattern 5 (*Vertical Slice Discovery*) operates within the capabilities this pattern maps — slices are derived only after capabilities are classified. Pattern 8 (*Tier-Aware Scaffolding*) operationalises the strategy per bounded context: core differentiator → full hexagonal; commodity → vertical slice. Pattern 4 (*Domain Ontology as Independent Substrate*) draws its vocabulary primarily from the capabilities this pattern identifies. Pattern 20 (*Team Topology and Bounded Context Alignment*) is the organisational counterpart:

capabilities map to bounded contexts; bounded contexts map (or fail to map) to teams. Pattern 21 (*Rollout and Cutover at Bounded Context Granularity*) sequences the migration in capability-priority order. The *Frozen Architecture* antipattern names what happens when modernization proceeds without this strategic framing — the legacy's accidental architectural decisions are preserved into the modern system because no one questioned whether they should be.

Pattern 2: The Legacy as Oracle

Status: prototype-validated.

Context

A legacy system that has been running in production for years or decades, processing real business workloads. The original specifications are typically lost, outdated, or never existed in written form. Tribal knowledge has moved with the people who built the system. The modernization effort needs ground truth about what the system does — the behavioural foundation on which any DDD strategic recovery has to stand.

Problem

Modernization platforms treat the legacy system as a starting point: something to be read, understood, eventually replaced. Once enough understanding has been extracted, the legacy fades to background. The modernization effort moves to the new system. Verification then becomes a question of "does the new system pass the tests we wrote." But the tests had to come from somewhere — and where they came from is the original system the team is trying to replace, mediated through human interpretation.

The result is that the modernization grades its own homework. In law, the principle of *separation of powers* names this failure: no party can be both judge and litigant in its own case. Modernization without an independent oracle violates exactly this principle — the team that builds the modernized system is the same team that decides whether it is right. With manual migrations, the team's misunderstandings shape both the new code and the tests for it. With AI-assisted migrations, the agents internalise the same misunderstandings whether they're generating code or generating tests. Either way, the verification is contaminated by interpretation.

Forces

The legacy must eventually be decommissioned, which makes the instinct to plan around its absence reasonable. But during modernization the legacy is the most reliable source of truth available — it has been producing correct outputs every day for thirty years. Building a parallel ground truth (specifications, behavioural models, test suites) is expensive and lossy. Yet running the legacy continuously throughout the modernization is operationally complex.

Pattern

Treat the running legacy as the live oracle for behavioural verification. Instrument it so it can be queried, observed, and compared against. Package it for local execution where possible — the Raincode-compiled

COBOL container (Raincode compiles COBOL to .NET IL, packageable as Docker) is one realisation of this. Make it available to the inner loop of the agentic workflow, not just to humans during review.

The legacy is more useful running than off. Decommissioning is the last step, not the first frame.

A precision worth stating explicitly: *the legacy is a behavioural oracle, not an ontological one*. The legacy reliably tells you what the system does. It does not reliably tell you what the domain really is. A system that has been running for thirty years has often accumulated ontological drift — three definitions of "active customer" across billing, support, and product modules; two incompatible interpretations of "balance" depending on which paragraph computes it; vocabulary that meant one thing in 1992 and another by 2008. Behavioural equivalence to such a legacy preserves the drift. The legacy answers *what happens*; the modernization team still has to answer *what should be true* about the domain.

This is what Brian Foote and Joseph Yoder named the *Big Ball of Mud* — systems that have grown by accretion over decades, where structure exists locally but no coherent global architecture survives. Foote and Yoder framed it as a structural antipattern; from the modernization perspective, the ontological drift is its semantic counterpart. The code may be a Big Ball of Mud; the domain vocabulary, by the time it has run for thirty years, is the same ball of mud expressed in concepts.

> Pattern 2 answers *what does the legacy do*. The complementary question — *what should the domain itself be about, independent of what the legacy happens to do* — belongs to Pattern 4: Domain Ontology as Independent Substrate. The two patterns are siblings: behavioural recovery (Pattern 2) and ontological recovery (Pattern 4). Neither is sufficient alone.

Consequences

The agents iterate against evidence rather than against assumptions. Verification becomes part of the inner loop instead of a downstream activity. The cost of being wrong drops sharply, which lets the agents explore more aggressively. The agents converge faster and more honestly because they're matching behaviour the system actually exhibits, not behaviour someone wrote down and trusted.

The cost is structural. The legacy must be packageable for local execution, which requires tooling that compiles the legacy runtime to a portable form (in the COBOL/CICS case, Raincode does this; for other legacy stacks the equivalent tooling may or may not exist). The legacy must remain observable throughout the modernization, which means the modernization platform has to integrate with it operationally, not just textually.

There is also a cost that the pattern alone does not pay: behavioural fidelity is necessary but not sufficient. The complementary discipline lives in Pattern 4 and the *Behavioural Equivalence Without Ontology* antipattern; without them, this pattern protects against the wrong thing.

I have only tested this pattern for COBOL/CICS. For that case, it has earned its place in Rosetta. The principle that legacy systems are more useful running than off is what generalises; the specific implementation is calibrated to CICS.

The notion of an "oracle" in software verification has a long history — William Howden formalised it in testing theory (Howden, 1978) as the source of authoritative answers against which test outputs are compared. This catalog applies the same notion at modernization scale: the legacy system itself, running, is the oracle. What is original here is not the concept of an oracle, but treating the running legacy

as the oracle in the *agentic inner loop* — comparing candidate translations against legacy execution at millisecond latency, rather than against test suites the modernization team itself wrote.

Related patterns

Pattern 4 (*Domain Ontology as Independent Substrate*) is the complementary recovery: the legacy is behavioural oracle, but ontology requires independent grounding. Pattern 13 (*Twin Verification*) is the operationalisation of this principle in the inner loop. Pattern 14 (*Hypothesis-Driven Verification*) extends it from dev mode to production mode. Without Pattern 2, neither of those is implementable.

Pattern 3: The Graph as Projection

Status: prototype-validated.

Context

A legacy codebase consisting of thousands of source files in a language that LLMs read poorly. The architectural concepts that matter — bounded contexts (DDD's term for explicit boundaries within which a particular domain model applies), candidate slices for modernization, the side-effect surfaces of programs — are not stated explicitly anywhere. They must be derived from structural relationships within the code.

Agents reasoning over the legacy need two distinct kinds of access. They need to traverse explicit relationships (this program calls that one, this paragraph accesses that resource) with certainty. They need to find similarity (these paragraphs are doing similar things even though no edge connects them) with proximity. The two kinds of access answer different questions and have different epistemologies — different theories of what counts as evidence and how confident the agent should be in what it sees. The graph traversal asks *what is structurally true*; the proximity search asks *what is similar enough to be informative*. One delivers certainty; the other delivers ranked plausibility. Both are legitimate; they have to be kept apart so neither contaminates the other.

A third kind of access — observation of runtime dynamic behaviour, what the legacy actually does when exercised — is the territory of Pattern 13 (Twin Verification) and Pattern 14 (Hypothesis-Driven Verification), not Pattern 3. Static substrates and runtime observation complement each other; together they recover what the legacy is on paper and what it does in execution.

Problem

Asking an LLM to reason about COBOL directly produces poor results. COBOL is verbose, full of historical idioms, and structurally unlike anything in the modern training corpus. The LLM pattern-matches to surface features and misses architectural intent. Multiple passes don't help because the underlying obstacle is representational: the LLM is trying to extract architecture from a representation that obscures it.

The instinct to give the agents a single unified representation also fails. A representation that is both discrete-and-exact (so it can answer "which programs call this one") and continuous-and-approximate (so it can answer "which paragraphs are semantically similar to this one") flattens one shape into the

other. Forcing similarity into typed edges loses the gradient; forcing typed relationships into vector space loses the precision. Both shapes are necessary, and they have to coexist.

Forces

Architectural recovery requires understanding relationships across the codebase: which programs call which, which data structures flow through which paragraphs, which CICS commands access which resources. This information exists in the source but is distributed and implicit. Capturing it as a queryable representation is expensive but unlocks downstream analysis. Doing so naively (as raw abstract syntax tree, capturing every token) produces a representation too large to query usefully and too detailed to be informative.

Semantic similarity is a different shape of query. Two paragraphs with no structural connection can be near-duplicates in what they do; a single `CALL` edge between programs is structurally explicit but semantically obscure (calling another program looks similar to many other operations in vector space). The two kinds of query want different infrastructures.

Operating two representations is more complex than operating one. Synchronisation across them adds engineering cost. But forcing them into a single representation loses the value of having either shape natively.

Pattern

Parse the legacy source into a property graph. Capture programs, paragraphs, data structures, control flow, side effects, predicates, and entry points as nodes. Connect them with typed edges. Make the graph the queryable architectural projection of the source.

Maintain a semantic index over the same source as a complementary substrate. The index captures what is semantically similar and queryable through proximity: vocabulary alignment, code-shape similarity, intent matching, naming convergence. Where the graph holds discrete-and-exact relationships, the index holds continuous-and-approximate proximity.

Define explicitly what information lives in each substrate. The graph holds what is structurally explicit and queryable through traversal: containment, calls, accesses, predicates, entry points. The index holds what is semantically similar and queryable through proximity. The boundary between them is part of the architecture: when in doubt, ask which kind of question would the operator pose, and put the answer in the substrate that answers that kind of question naturally.

Keep ingestion deterministic and idempotent where it can be. Two runs over the same source must produce the same graph — node identifiers, edges, properties — to the bit. The semantic index cannot meet the same standard: embeddings drift across model versions, batch effects, and floating-point variance. What the index must guarantee instead is *stable retrieval semantics*: the same query against the same corpus must surface the same top-N results in the same order, even when the underlying vectors are not bit-identical. Synchronise the two substrates through a shared ingestion pipeline. When source changes, both substrates update from the same input. Put semantic interpretations (bounded contexts, slice candidates, discriminator fields) in a separate derived layer computed by analysis passes. The agents reason about the graph and the index together, not about the source.

Agent queries combine the two substrates by composition. A typical hybrid query starts in the graph

(find the entry point for a specific transaction), expands structurally (follow PERFORMs and CALLs to depth N), then expands semantically (find paragraphs similar to those reached, even if not structurally connected). The reverse also works — start with semantic similarity, anchor the matches structurally — and complex queries iterate between the two. The result, regardless of direction, is a richer working set than either substrate alone would produce.

Source provenance (introduced in the architectural interlude) is what makes the dual substrate auditable. Every graph node has `source_file`, `start_line`, `end_line`; every index entry carries the same. When a hybrid query finds "paragraphs structurally connected to entry point X and semantically similar to paragraph Y," the result set is traceable on both axes. The agent doesn't just produce a working set — it produces a working set whose every member can be defended back to source.

Consequences

Each substrate stays simple in what it does. The graph remains deterministic, reproducible, and exactly queryable. The index remains rich, semantic, and approximately queryable. Neither has to compromise to accommodate the other.

The agents reason about something LLMs handle well — graph relationships, named concepts, queryable structure, semantic proximity — while the underlying COBOL stays in its place as the source of truth. Architectural concepts emerge through analysis rather than through interpretation. By *analysis* I mean structured queries over the graph and index that surface candidates with cited evidence — community detection finds clusters; reachability finds slice boundaries; predicate similarity surfaces discriminator fields. By *interpretation* I mean an LLM reading raw COBOL and being asked to assert what the architecture is. The first is reproducible and reviewable; the second is not. Analysis produces candidates the architect validates; interpretation produces claims the architect has no leverage to verify.

Community detection on the graph surfaces bounded contexts. The Louvain algorithm (Blondel et al., 2008) is the most widely used and the default in most graph databases; Leiden (Traag et al., 2019) improves on Louvain's resolution limit and produces more stable communities; modularity-based methods, label propagation, and infomap give different cuts with different trade-offs. The choice of algorithm is part of the architecture: different algorithms surface different boundaries on the same graph, and engagement-specific experience teaches which algorithm fits which legacy shape.

Entry-point reachability and pseudo-conversational boundaries surface candidate slices for modernization. Predicate analysis at low depth surfaces discriminator fields. The IF statements (and EVALUATE statements) near a transaction's entry point typically test a small set of fields to decide what the transaction does — what type of claim is being submitted, what kind of customer is calling, which channel the request came through. Those fields are the *discriminators*: the data that branches the system into qualitatively different behaviours. Recovering them surfaces the implicit business taxonomies the legacy encodes — which is exactly the input strategic recovery needs. Semantic clustering surfaces concept candidates the structural graph alone wouldn't see. Each technique becomes a reusable lens.

Hybrid queries become a first-class capability of the modernization pipeline. The agents have access to both kinds of relationships, and the platform doesn't prejudge which kind matters for a given query. The graph and index together render something legacy modernization rarely produces directly: a context map. A context map, in Eric Evans's DDD vocabulary, is the diagram of how bounded contexts relate to one another — which contexts integrate, which translate vocabulary at the boundary, which conflict over

shared concepts, which share a kernel of common types. In greenfield DDD, the context map is drawn by the team designing the system. Here, the context map *emerges from the legacy itself* — derived from structural relationships in the graph and semantic similarities in the index, rather than from someone's prior description of how the system was supposed to be organised. The difference matters: the emergent map captures the legacy as it actually is, which is rarely how anyone remembered it.

The cost is the ingestion pipeline and the dual-substrate discipline. Building a parser that captures the right level of structure requires careful schema design. The temptation is to capture everything — full abstract syntax tree, every token, every node — which produces a graph too large to query usefully. The right level is summarised: programs, paragraphs, control flow, data flow, side effects, predicates, entry points, and the typed relationships between them. Token-level detail belongs in the source file, not in the graph. Choosing what to elide is as important as choosing what to capture. Schema versioning becomes a discipline because the substrates are the long-lived artifacts; ingestion can be re-run, but the schemas need to evolve coherently across both. Two substrates require two query interfaces and two consistency disciplines. When one substrate updates, the other must follow. The dual maintenance is real.

The benefit pays back over time. Modernization queries get richer, more nuanced, more capable of handling the messy reality of legacy code. The two substrates together approximate observations a domain expert with deep knowledge of the codebase would surface from intuition: "these unconnected paragraphs are doing the same thing," "this CALL is incidental, not semantic," "these similar-looking blocks implement different intents." That's not the totality of what an expert sees, but it is the part that scales with the codebase.

Anthony Alcaraz has written about agentic GraphRAG as a general pattern, and combining knowledge graphs with vector indices is an emerging practice in retrieval-augmented generation systems generally — Microsoft's GraphRAG work and the broader hybrid-retrieval literature have articulated the value. The intellectual lineage extends to program comprehension research (Storey 2005; Müller and Klashinsky's earlier work on software architecture recovery) which has long argued that source code alone is insufficient for understanding large legacy systems — derived representations are necessary.

What this catalog contributes is two things. First, applying the graph-plus-index pattern specifically to mainframe specification recovery: COBOL's representational distance from modern code is wide enough that no single representation reads well, which makes the dual projection essential rather than optional. Second, framing the two substrates as *complementary representations with different epistemologies* — the graph is discrete and exact (it answers structural questions with certainty); the index is continuous and approximate (it answers similarity questions with ranked plausibility) — and treating the boundary between them as a deliberate architectural commitment. Many systems collapse one shape into the other to simplify the engineering. The discipline this pattern argues for is the opposite: keep both shapes, and design the boundary between them explicitly. Where each kind of question goes is part of the architecture, not an implementation detail.

Related patterns

Pattern 4 (*Domain Ontology as Independent Substrate*) is what the structural graph and semantic index alone cannot provide — recovering what entities exist in the domain and how they relate, beyond what the legacy artifacts encode. The ontology consumes both substrates as inputs (semantic clustering surfaces concept candidates; structural graph reveals containment and relationship) but lives independently of either. Pattern 5 (*Vertical Slice Discovery*) consumes the dual substrate to identify slice candidates through

hybrid queries. Pattern 7 (*The Intermediate Representation*) is how the graph and the index cross over into deterministic generation. The source provenance discipline (architectural interlude) extends through both substrates — every node and every index entry carries source coordinates.

Pattern 4: Domain Ontology as Independent Substrate

Status: designed.

Context

A modernization with structural graph and semantic index (Pattern 3) in place. The legacy is being treated as behavioural oracle (Pattern 2). The agents are reasoning over real artifacts of the legacy system. But the modernization team eventually has to answer a question that the legacy alone cannot answer: *what is the domain this system is supposed to be about*. In DDD vocabulary, this is the question of establishing the **ubiquitous language** — the shared vocabulary of the domain that the team and the system both speak — and articulating the **strategic design** that organises bounded contexts within that language.

Problem

A system that has been running for years or decades has accumulated ontological drift — the kind already named in Pattern 2: three definitions of "active customer," two interpretations of "balance," vocabulary that meant one thing in 1992 and another by 2008. The legacy faithfully implements all of these, often in tension with each other, often without anyone noticing. In DDD terms, the ubiquitous language has been lost or never established; what survived is operational dialect, fragmented across modules.

Modernizing such a legacy through structural and behavioural fidelity alone preserves the drift. The agents reason over the structure of the legacy — its tables, its programs, its data flows, its call graphs — not over what those structures are *about*. They can faithfully translate a paragraph that computes "balance" without ever asking which of the three definitions of "balance" the paragraph implements. The pattern can be flawless and the output will still be confidently wrong — not because the translation is incorrect, but because the modernization has inherited the legacy's confusion about what the domain really is. Anthony Alcaraz has argued this point for agentic systems generally. The orchestration of agents — how they pass work to each other, when one defers to another, what state machine governs their interaction — is downstream of *ontological grounding*: the question of what the agents are reasoning *about*. Swap one orchestration pattern for another and the system still works if the grounding is right. Get the grounding wrong, and no orchestration pattern recovers it. The orchestration is interchangeable; the ontological substrate is not. The argument lands with particular force in legacy modernization, where the substrate has had decades to drift.

Forces

Behavioural recovery (Patterns 1, 2, 3) is necessary. It is also tractable: the legacy is right there, observable, queryable, runnable. Ontological recovery is harder. The domain the legacy was supposed to be about is not the same as what the legacy actually became. Recovering ontology requires sources the legacy alone does not provide — domain experts, business artifacts, regulatory definitions, the

conversations between teams that produced the implementations in the first place. These sources are partial, sometimes contradictory, and require human judgement to reconcile.

Skipping ontological recovery is cheaper in the short run and catastrophic in the long run. The modernized system inherits whatever ontological confusion the legacy had, now expressed in clean modern code that makes the confusion harder to detect and harder to fix.

Pattern

Treat the domain ontology — the formal articulation of what entities exist in the domain and how they relate — as an independent substrate of the modernization, separate from the structural graph and semantic index (Pattern 3), separate from the implementation artifacts of the legacy. The ontology specifies what entities exist in the domain, how they relate, what the canonical vocabulary is, where the boundaries between concepts lie. It is the foundation of the ubiquitous language. It is not derived from the legacy; it is *grounded* in conversations with the people who understand the domain, validated against business sources, and reconciled where the legacy disagrees with itself.

Recovery of the ontology is partial from the legacy. Vocabulary inference from comments, display literals, naming conventions, the intermediate representation, and the data layer's DDL — DB2 schemas, VSAM definitions, DCLGEN copybooks — provides candidate terms. The data layer often preserves domain vocabulary better than the procedural code does: column names in DDL frequently retain canonical business terms that working-storage variables in COBOL paragraphs have abbreviated, prefixed, or renamed for technical convenience. Semantic similarity over code units (the semantic index discussed in Pattern 3) surfaces clusters that may correspond to ontological concepts. These are starting points, not conclusions.

Workshop techniques like Event Storming (developed by Alberto Brandolini for collaboratively recovering domain understanding from systems and people) accelerate the human side of this work — domain experts, developers, and operators in a room mapping events, commands, and aggregates against shared vocabulary. The architect or domain expert validates each candidate against domain understanding, refines vocabulary, reconciles drift, articulates the canonical ontology that the modernized system should encode.

Eric Evans named this work *distillation* (Evans, 2003): separating what is essential about the business from what is incidental about how the legacy happened to express it. Vaughn Vernon (2013) elaborated the operational mechanics for in-life systems. What this catalog adds is the framing of ontology as a *substrate* of the modernization architecture, alongside the structural graph (Pattern 3), the IR (Pattern 7), and the heuristic catalog (Pattern 17). The ontology is not a mental model the team carries in their heads. It is not a glossary in a wiki that no one updates. It is a queryable, versionable artifact, stored in its own substrate, maintained on its own change cadence, consumed by every other substrate that needs canonical vocabulary. The legacy substrates feed it during recovery (vocabulary inference, semantic clustering, DDL analysis), but once recovered, the ontology lives separately from them — its lifecycle is its own. It does not change when the schema changes, when the architecture changes, when the framework changes. It changes only when the domain changes — when the business itself adopts new concepts or retires old ones.

This independence is what makes ontology durable across modernizations and across the system's lifetime. And the durability is what makes ontology, arguably, the most valuable asset the modernization produces.

The code will be rewritten again within ten or fifteen years — that is the nature of software. The framework will be replaced; the cloud architecture will evolve; the team will turn over. What survives across these transitions is the canonical vocabulary of the business: the recovered, validated, agreed answer to what the domain is actually about. The ontology is the bridge between business strategy and technical architecture, expressed in a form both sides can read. The business reads it as the authoritative description of its own concepts. The architecture reads it as the contract every modernized component must respect. Few artifacts in software earn that dual readership. Fewer still survive the lifetime of the systems they describe.

The modernization uses the ontology as a reconciliation reference. When the legacy has three definitions of "active customer," the ontology articulates the canonical one and the modernization team decides which legacy paragraphs implement which concept under the canonical definition. Twin Verification (Pattern 13) confirms behavioural equivalence within each canonical concept; the ontology decides which paragraphs belong to which concept in the first place.

Ontology recovery is not a one-shot activity at the start of the modernization. Nick Tune has documented how the target model itself drifts during migration: concepts that began as straightforward renames end up restructured as the team's understanding sharpens through contact with the legacy and with domain experts (see *Drifting Domain Model* in the glossary). Pattern 4 accommodates this: the ontology is a living substrate, versioned and revisable, and the harness (Pattern 18) records each revision as a first-class event in the modernization's audit trail.

Consequences

The modernization has a referent that is independent of the legacy. Disagreements within the legacy can be reconciled against the ontology rather than preserved by default. The vocabulary in the modernized C# matches the domain, not the historical accidents of how the legacy expressed the domain.

The ontology also functions as a defence against the *Behavioural Equivalence Without Ontology* antipattern. When the modernization preserves three behaviours that the ontology says should be one, the divergence is visible and addressable. When the modernization preserves vocabulary that the ontology says is wrong, the discrepancy becomes a deliberate decision (preserve for compatibility) or a refactoring target (rename to canonical), not an unnoticed inheritance of confusion. This is how the ontology becomes a tool for managing technical and cognitive debt. *Technical debt* is the cost of code that no longer matches the architecture the team would design today — well-understood, widely discussed. *Cognitive debt* is the cost of vocabulary that no longer matches the concepts the team would name today — less widely discussed, often more expensive. Each discrepancy between legacy vocabulary and canonical ontology is one of two things: a tech debt or cognitive debt item the team is choosing to carry deliberately, or a refactoring backlog item with a known target. Either way it is named, sized, and trackable. What the ontology prevents is the third option: debt that accumulates silently, never acknowledged, never paid down, eventually overwhelming the team that inherits the system.

The cost is that ontology recovery is genuinely hard work. It requires access to domain experts whose time is scarce. It requires reconciling sources that disagree. It requires judgement calls that are not derivable from the legacy and not obvious from any single domain artifact. Most modernizations do not do this work, which is why most modernizations inherit the ontological drift of their predecessors.

This pattern is *designed* in Rosetta. The principle is articulated; the substrates that feed ontology recovery (vocabulary inference, semantic clustering) are operational; what remains is the ontology substrate itself

as a first-class artifact — the operational machinery to validate, refine, and maintain canonical ontology independently of the structural and semantic substrates. The work is in design; the foundation it builds on is in place.

Related patterns

The *Behavioural Equivalence Without Ontology* antipattern names the failure mode this pattern protects against — without Pattern 4, modernization preserves the legacy's confusion under the appearance of fidelity. Pattern 2 (*The Legacy as Oracle*) is what this pattern complements: behavioural fidelity is necessary but not sufficient, and Pattern 4 names the missing piece. Pattern 3 (*The Graph as Projection*) provides structural and semantic input that ontology recovery can draw on — vocabulary inference from comments and naming and similarity clustering from the semantic index are starting points for ontology rather than substitutes. Pattern 7 (*The Intermediate Representation*) consumes the ontology when it is available — IR vocabulary aligns with canonical ontology rather than with whatever the legacy happened to use.

Pattern 5: Vertical Slice Discovery from Structural and Behavioural Signals

Status: in construction.

Context

A legacy codebase parsed as a graph (Pattern 3) and enriched with semantic signal. The graph contains thousands of paragraphs, hundreds of programs, dozens of bounded contexts. The modernization effort needs to identify *vertical slices* — coherent feature units that can be extracted, scaffolded, translated, and verified as a working whole. Each slice represents one user-visible behaviour from input through processing through output.

A note on vocabulary, since the catalog uses several related terms in close proximity. Capabilities (Pattern 1), bounded contexts, vertical slices, and aggregates are not synonyms — they are nested concepts of different scope. A **capability** is a business-recognised unit of value ("process insurance claim"); it typically spans multiple bounded contexts and many slices. A **bounded context** is a DDD boundary within which a single domain model and vocabulary apply; it contains many slices. A **vertical slice** is a coherent feature unit within a bounded context — one user-visible behaviour, end to end. An **aggregate** is a cluster of domain objects within a bounded context, treated as one unit for state changes; a slice typically touches one or more aggregates. The hierarchy is: capability → bounded context → vertical slice → aggregate. Each level is the unit of a different kind of work: capability mapping is strategic, bounded context identification is architectural, vertical slice discovery is tactical, aggregate design is implementation.

In DDD vocabulary, slices map to use cases within bounded contexts; the aggregates a slice touches define its consistency boundary.

Problem

A bounded context is too large to be the unit of work. A paragraph is too small to be a feature. The right unit is between — a slice that includes the side effects it produces (writes to DB2 or VSAM, messages onto MQ or CICS queues, calls to external systems, audit log entries, notifications dispatched), not just the logic that triggers them. A slice that excludes its side-effect tail is incomplete: extracting it would leave the side effects orphaned, attached to no one's modernized counterpart. But slices are not stated explicitly anywhere in the legacy. They have to be inferred.

Pure structural derivation produces slice candidates that are syntactically coherent but operationally meaningless. Pure observational derivation (which transactions actually run together) produces slices that reflect current usage patterns but miss the structural coherence. Either signal alone is insufficient.

Forces

The slice must be small enough that the agents can scaffold and translate it as a unit, but large enough that it represents real business behaviour. Structural cohesion is one signal — measured through community detection algorithms (Louvain, Leiden) over the graph (Pattern 3), or through more focused metrics like shared-data ratio and predicate-overlap within paragraph clusters. Behavioural cohesion is another. The two signals frequently agree but sometimes diverge — and the divergence is informative.

Pattern

Derive slice candidates from a multi-signal pipeline. Five signal sources contribute, each with its own engineering cost and confidence weight:

- **Structural cohesion in the graph** — paragraph clusters with high internal cohesion (shared data, predicates, ancestry from a common entry point), low external coupling, and bounded *side-effect surface* (the set of external resources the cluster writes to, queues it dispatches onto, or external systems it calls). The community-detection algorithms named in Pattern 3 (Louvain, Leiden, modularity-based methods) are the standard mechanism; for slice discovery specifically, the cut quality matters more than algorithm pedigree, and engagement-specific experience teaches which algorithm produces slice candidates an architect will recognise.
- **Linguistic cues encoded by the original programmer** — `CALL` and `EXEC CICS LINK` mark structural coupling within a slice; `XCTL` typically marks transition between bounded contexts; `START TRANSID` marks a new transactional unit; `EXEC CICS WRITE TS QUEUE` and `EXEC CICS WRITE TD QUEUE` mark asynchronous communication points where slices can split; shared copybooks mark data coupling. Each construct is a decision the original programmer made about how the system is composed.
- **The data layer's DDL** — DB2 schemas, VSAM file definitions, DCLGEN copybooks. Field names in DDL frequently preserve domain vocabulary better than working-storage names. Foreign keys reveal relationships hidden by procedural code. Constraints capture domain invariants. Where DDL boundaries align with proposed slice boundaries, confidence increases; where they diverge, the divergence is informative.
- **Operational observation when available** — paragraphs exercised together by real transactions, with shared inputs and shared outputs.
- **Synthetic execution when production telemetry is not available** — the Legacy Twin (Pattern 2)

is already runnable locally, and if instrumented for paragraph-level execution tracing, it produces the equivalent signal: synthetic test inputs exercise candidate slice boundaries and the trace reveals which paragraphs activate together. The instrumentation is part of the Twin's setup cost; without it, the Twin can verify behavioural equivalence but cannot contribute to slice discovery. The synthetic signal is not equivalent to production observation — it reflects test design, not real usage — but it catches many divergences between structural intuition and operational reality.

The heuristics that map signals to slice boundary candidates live as first-class artifacts, not as implicit knowledge in agent prompts. A heuristic catalog declares: "XCTL between paragraphs in different bounded contexts is strong evidence of slice transition; weight: 0.8." Specialized agents query the catalog when they need to interpret a cue in context. The catalog is queryable, versionable, and observable — when an agent proposes a slice boundary, the reasoning record (Pattern 18) cites the heuristics applied and the evidence supporting each.

For CICS specifically, pseudo-conversational boundaries (`RETURN TRANSID/COMMAREA` cycles) provide strong structural evidence: the system itself signals where one user-visible interaction ends and the next begins. Use this as the primary structural anchor.

The output is a set of candidate slices, each with: an entry point, a set of paragraphs that participate, a side-effect surface, an estimated tier classification (Pattern 8), and a confidence signal indicating how strongly the signals agree. Treat low-confidence slices as needing human validation; treat high-confidence slices as ready for scaffolding. The discovery process is not "find all slices automatically" — it is "propose slices with evidence and let architects validate or refine."

Consequences

The modernization gets units of work that are operationally meaningful, not just syntactically coherent. The agents work on real features; architects validate slice boundaries based on combined evidence — structural, linguistic, data-layer (DDL schemas and foreign keys), operational, and synthetic. Each slice carries its own provenance — traces back to the graph nodes, DDL fragments, and observation traces (production or synthetic) that grounded it. This makes the slice a queryable artifact: the control plane (Pattern 19) shows why a slice was proposed and where signals diverge.

Slices are not the unit of business value — capabilities are. A business capability ("process insurance claim," "underwrite policy renewal," "reconcile end-of-day") typically composes multiple slices. Slice discovery should produce slices that aggregate naturally into recognisable capabilities. When proposed slices do not compose into capabilities the business recognises, the discovery has fragmented something unitary or conflated something separable. Capability mapping is a validation lens for slice quality, not a substitute for slice discovery.

The cost is the multi-signal pipeline. Each signal has its own engineering cost; when a source is unavailable, slice discovery falls back to remaining signals with explicit confidence reduction. The audit trail records which signals validated each slice. Pattern 9 (*Pluggable Emitters*) — the catalog's pattern for rendering substrates into views through deterministic emitters — can render slices into specific views: slice maps showing the working set, communication maps showing queues and CALLs between slices, dependency diagrams showing the side-effect surface.

The pattern's status is *in construction*. Structural slice discovery — using graph queries, linguistic cues, and DDL evidence — is implemented and validated inside the Rosetta prototype today. What remains

in construction is the integration of *behavioural* signal: production observation through Witness (Pattern 14) once real engagement traffic is available, and synthetic execution traces from an instrumented Legacy Twin in dev mode. The structural side establishes the candidate; the behavioural side either confirms it (the paragraphs structurally clustered are the ones that actually run together) or surfaces a divergence the architect must resolve. The pattern works today on structural signal alone; it earns its full claim once behavioural integration is built.

The notion of *vertical slice* as architectural unit comes from Jimmy Bogard and Steven Smith's articulation of vertical slice architecture (Bogard, 2018; Smith, 2018). Alberto Brandolini's Event Storming (Brandolini, 2013) provides the workshop technique for collaborative slice discovery with domain experts. What this catalog contributes is the methodology for slice discovery in legacy mainframe systems — the linguistic cues, the multi-signal pipeline (structural, linguistic, DDL, observational, synthetic), the heuristic catalog as queryable artifact — applied at codebase scale where Event Storming alone wouldn't reach.

Related patterns

Pattern 3 (*The Graph as Projection*) provides the structural and semantic substrate. Pattern 4 (*Domain Ontology as Independent Substrate*) provides the canonical vocabulary slice boundaries align with — slices are coherent units of domain behaviour, and the ontology says what counts as a unit. Pattern 6 (*The Compiler Principle*) is why the heuristic catalog lives as deterministic infrastructure. Pattern 8 (*Tier-Aware Scaffolding*) consumes slice candidates and produces the appropriate scaffold. Pattern 9 (*Pluggable Emitters*) renders slice candidates into views the architect can validate, since documentation emitters live there. Pattern 14 (*Hypothesis-Driven Verification*) provides the behavioural signal that refines structural slice discovery. Pattern 15 (*Bounded MCP Servers*) is where specialized slice-discovery agents live with explicit access to the heuristic catalog. Pattern 17 (*Heuristics as Explicit Artifacts*) holds the catalog this pattern consumes. Pattern 18 (*The Harness as Self-Observing State Machine*) makes heuristic application observable through its reasoning telemetry.

Architectural Interlude: Foundations Across the Catalog

Legacy modernization fails when it tries to go from source to scaffold in one hop. Too much semantic loss. Too much implicit knowledge buried in runtime contracts rather than the source text. Too many architectural commitments smuggled in under the appearance of mechanical translation.

The patterns in Part I recover legacy systems from source; the patterns in Part II project them onto modern code. Between recovery and projection lives an architectural discipline this catalog has been articulating implicitly across the patterns above and the patterns ahead. Two pieces of that discipline operate across the whole catalog rather than belonging to any single pattern. They are the foundations every pattern presupposes.

The first foundation is the three-layer recovery architecture. The second is the source provenance discipline that holds the layers honest. Both are introduced here so the patterns that follow can refer to them without re-establishing the ground each time.

The three-layer recovery architecture

Recovery has to traverse **three distinct substrates**, each at a different abstraction level, with architect-gated transitions between them. None are implicit LLM leaps. The substrates are not a methodology imposed on the work — they emerge from the work itself, once recovery is taken seriously as a multi-stage activity.

L1 — Syntactic and Resource Graph

The bottom layer is what the legacy *literally says*. Pattern 3 (*The Graph as Projection*) produces it: a property graph of programs, paragraphs, copybooks, CICS resources, control flow, data flow, side effects, predicates, entry points. Provenance discipline (see below) is mandatory at every node — every L1 element traces back to specific file coordinates.

L1 is **evidence, not interpretation**. Two runs over the same source must produce the same graph. The graph is diffable against the source; the source is diffable against the graph. No layer above L1 can claim to know something L1 doesn't witness.

L1 is deterministic. No LLMs participate in its construction. The parser produces the graph; the graph is what it is.

L2 — Semantic Intent Graph

The middle layer is what the legacy *actually does*. This is the territory where legacy idioms get named, codified, and made queryable as patterns — and where most of the catalog's intellectual work lives.

L2 is the **specification the original team never produced**. It is the layer at which Pattern 4 (*Domain Ontology as Independent Substrate*) recovers canonical vocabulary; at which Pattern 5 (*Vertical Slice Discovery*) identifies coherent units of behaviour; at which Pattern 17 (*Heuristics as Explicit Artifacts*) holds the queryable rules that interpret legacy structural cues. The ontology, the heuristic catalog, and the slice working sets are all L2 substrates — different facets of the same intent layer.

L2 is where DDD's strategic design lives in this architecture. Bounded contexts, ubiquitous language, aggregate candidates, command and event boundaries — these are L2 concepts, recovered from L1 evidence and validated against domain understanding. L2 patterns are typed subgraphs over L1 with required structural signatures, disqualifying signatures, and explicit confidence scores. A `LatentSaga` is an L2 pattern: it requires specific L1 evidence (paired write/inverse-write paths, linguistic markers like *refund* or *reverse*) and excludes specific L1 evidence (intermediate writes without inverses, which would make it a different L2 pattern). The pattern is precise enough to be testable; the testability is what makes L2 reviewable.

LLMs enter L2 only for **linguistic disambiguation, and only to propose edges that an architect or oracle verifies**. The agents do not author L2; they propose candidates that the harness (Pattern 18) gates and that humans validate through the control plane (Pattern 19). L2 is the design document the original team never produced — and it cannot be produced by LLMs alone, because what counts as a valid pattern is a domain question, not a linguistic one.

L2 is what compounds across engagements. Every CICS modernization that passes through this catalog refines the L2 catalog: a `RecoverableSideEffect` pattern caught once sharpens its detection forever; a misclassified `LatentSaga` (Compensation Cargo Cult — see antipatterns) teaches the catalog where its disqualifying signatures need tightening. The L2 catalog is the durable artifact of the modernization team's accumulated CICS expertise, codified.

L3 — Architectural Target Graph

The top layer is how the legacy *should be expressed* in modern code. This is where projection happens.

L3 is the territory of Pattern 7 (*The Intermediate Representation*) — the typed WolverineIntentModel that encodes architectural commitments: handlers, aggregates, commands, events, sagas, ports. Pattern 8 (*Tier-Aware Scaffolding*) decides which architectural shape each bounded context receives in L3. Pattern 9 (*Pluggable Emitters*) renders L3 into target code.

L2 → L3 is **not 1:1**. It is a mapping function with architect-gated rules. A `DeferredCommitWizard` in L2 does not become a `Saga` in L3 — it becomes a `StatefulFlow + SingleCommand`, because the deferred-commit shape has no compensation semantics to operationalise. A `LatentSaga` in L2 does become a `Saga` in L3, but only after the recovered compensations have been validated as first-class steps. A `SyncConversation` (LINK/XCTL chain inside one task) typically collapses into a single command handler in L3 — the call chain is recovered as evidence, not preserved as structure.

The mapping rules are explicit policy, versioned, and architect-validated. **L3 invariants override L2 shape when they conflict.** The graph holds decisions; the architect validates them; the compiler renders

but does not decide. This is the compiler principle (Pattern 6) operating at the layer transition.

Architect-gated transitions

Each transition has a specific gate.

- **L1 → L2** is gated by the heuristic catalog (Pattern 17). Pattern detectors run as Cypher queries against L1; each match carries its evidence, its confidence, and the heuristics applied. Low-confidence matches surface for review; high-confidence matches proceed; disqualified matches are recorded so the catalog learns from the negative case.
- **L2 → L3** is gated by the mapping rules. Each L2 pattern has explicit projection rules onto L3; each rule carries its rationale and the architectural invariants it must satisfy. Architect overrides are recorded as graph nodes, becoming part of the evidence trail that future projections consult.
- **L3 → scaffold** is gated by `scaffold-meta.json` (Pattern 18). The scaffold defines constitutional boundaries: which fields are immutable, which extension points agents may fill, which invariants generated tests must verify. The agent never breaches these boundaries — the harness (Pattern 18) enforces them as code, not as prompts.

Why the three layers matter

The architecture earns four properties that single-hop modernization cannot:

Property	Layer that delivers it
Verifiable	L1 is diffable against source; two runs produce the same graph
Reviewable	L2 patterns are named, evidenced, disputable; each detection cites its L1 evidence
Governable	L3 mapping rules are explicit policy, versioned, architect-validated
Constitutional	The scaffold enforces L3 boundaries as code, not as prompts

Without L1, generation has nothing to ground claims in. Without L2, modernization silently inherits legacy structure as if it were legacy intent. Without L3, scaffolding becomes mechanical translation of L2 shapes that should have been transformed. Without architect-gated transitions, the layers collapse into the single-hop failure mode the architecture was designed to prevent.

How the three layers map to the DDD arc

The three-layer architecture is a cross-cutting lens, not a competing organization. Every pattern in the catalog lives in one of the four DDD-arc parts (strategic recovery, tactical generation, verification, governance) and operates on or between substrates of the three-layer architecture. Some examples:

- Pattern 3 (*The Graph as Projection*) lives in Part I and produces the L1 substrate.
- Pattern 4 (*Domain Ontology*) lives in Part I and produces an L2 substrate.
- Pattern 6 (*The Compiler Principle*) lives in Part II and operates the L2 → L3 transition.
- Pattern 7 (*The Intermediate Representation*) lives in Part II and is the L3 substrate.
- Pattern 13 (*Twin Verification*) lives in Part III and verifies L3 outputs against L1 evidence through the legacy oracle.
- Pattern 18 (*The Harness as Self-Observing State Machine*) lives in Part IV and governs all the transitions.

When a pattern is rooted in one part of the DDD arc but operates across multiple layers, the pattern body says so. The reader does not need to track both schemes in parallel — the patterns make their position in both visible where it matters.

Source provenance as cross-cutting discipline

The second foundation is not a pattern; it is a discipline every pattern presupposes. **Every artifact at every layer carries source coordinates back to the legacy code it derives from.**

The discipline is mechanical and unglamorous. It is also load-bearing: without it, every capability that depends on traceability collapses; with it, those capabilities become tractable. Compilers have always done this through debug symbols and source maps. Build systems do it through deterministic dependency tracking. Audit infrastructure in regulated industries does it through signed transaction logs. What this catalog adds is the framing of provenance as the architectural commitment of an *agentic modernization pipeline*: every representation the agents reason over carries source coordinates, not just the artifacts humans inspect.

What the discipline requires

Every graph node carries source coordinates (`source_file`, `start_line`, `end_line` in Rosetta — naming is implementation-specific, the discipline is not). Every IR element carries the same, plus references to the graph nodes it derived from. Every generated C# artifact carries comments or metadata pointing back to the IR element that produced it. Every semantic index entry carries the same source coordinates as the graph nodes it derives from. Every agent reasoning record carries citations to the substrates it consulted; those citations carry provenance forward.

Schema validators enforce provenance presence — a graph node without source coordinates is rejected at ingestion, an IR element without graph references is rejected at construction, a generated artifact without metadata pointers fails the build. The discipline is enforced by the substrates themselves, not by reviewer attention.

The audit trail is end-to-end. A modernization decision can be traced from the agent's reasoning back through the substrates the agent queried back to the specific legacy code those substrates derived from. A production exception in the modernized C# traces back through the IR back through the graph back to the source COBOL. A regulator asking how a particular decision was reached gets an answer of the form "the agent reasoned from these specific lines of COBOL, with these specific transformations applied, validated against these specific heuristics" — not "the agent decided."

Why the discipline must be cross-cutting

Provenance fails the moment any link in the chain is optional. A graph that mostly has source coordinates is not provenance-enabled; it is provenance-vulnerable. The audit trail breaks at the first node that lacks them, and the reviewer has to do manual archaeology from that point. Cross-cutting discipline means every layer of the pipeline enforces provenance locally, so that the chain holds globally.

The temporal asymmetry is what makes provenance discipline so frequently neglected. The cost is paid daily by everyone building the pipeline — every developer working on a new analysis pass must remember to thread provenance through, every emitter must carry it forward, every transformation must validate it.

The benefit accrues to a future operator, regulator, or debugging session that isn't in the room when the schema is designed. By the time you need provenance, it's too late to add. This is why the discipline must be enforced as code, not as convention.

Where provenance shows up in the patterns

Several patterns refer to provenance discipline because they consume or produce it. Pattern 3 (*The Graph as Projection*) is where source coordinates first appear — the graph schema is where provenance becomes mandatory. Pattern 7 (*The Intermediate Representation*) extends it into the IR. Pattern 18 (*The Harness as Self-Observing State Machine*) requires agent reasoning records to cite the provenance of their evidence. Pattern 19 (*The Control Plane*) is the operational surface that consumes provenance, surfacing side-by-side comparisons and audit traces to humans.

But these are local callouts to a global discipline. Provenance is not what any single pattern does; it is what every pattern respects.

The AsIs/ToBe ownership discipline

The third foundation across the catalog is not a substrate. It is a discipline about *who owns what kind of artifact*.

Recovery and projection produce two qualitatively different things, and the patterns above presuppose that the difference is held explicitly.

AsIs is the recovered specification — evidence about what the system actually does. Where the parser produced an L1 graph, where the heuristic catalog (Pattern 17) detected an L2 pattern with cited evidence, where a use case was recovered with structural anchors and linguistic vocabulary, the result is AsIs. No design decisions live in AsIs. Two runs over the same source produce the same AsIs. When the legacy changes, AsIs re-derives mechanically.

ToBe is the target design — judgment about how the legacy *should be expressed* in the modernized architecture. Where the tier is decided (Pattern 8), where the architectural pattern is selected (Pattern 9), where the seam between bounded contexts is drawn, where commands and events shape the logical boundary (Pattern 10) — the result is ToBe. ToBe is not derivable from AsIs alone. The same AsIs can ground multiple defensible ToBe designs; the choice between them is architectural judgment, validated by humans, recorded as decision.

The dichotomy is what makes the rest of the catalog tractable. Pattern 6 (the compiler principle) only holds if it is clear which substrates hold decisions and which substrates hold evidence. Pattern 13 (Twin Verification) only verifies meaningfully if what it tests against (AsIs behaviour, captured through the legacy oracle) is distinguishable from what it produces (ToBe artifacts, candidate translations). Pattern 18 (the harness) only governs coherently if its gates separate decisions from evidence rather than treating all artifacts as equally subject to revision.

The ownership pattern is the discipline:

- **AsIs artifacts are owned by deterministic infrastructure** — the parser, the pattern detectors, the heuristic catalog, the semantic index. Humans review the *infrastructure that produces AsIs*, not the AsIs itself, because AsIs is mechanical output.

- **ToBe artifacts are owned jointly by agents, the ontology, and humans** — agents propose, the ontology constrains, humans approve. Every ToBe artifact carries a decision record: which ontology version constrained it, which agent proposed it, which human approved it, what evidence from AsIs grounded the proposal.
- **The Intermediate Representation (Pattern 7) is the bridge contract.** It is not where decisions are made. It is what the scaffold renders from, after ToBe is finalized.

Where the dichotomy collapses, the catalog collapses with it. If agents are permitted to revise AsIs to make their ToBe proposals fit, evidence becomes negotiable and the pipeline degrades into rationalisation. If humans are asked to validate AsIs line by line, they exhaust their attention on mechanical output and have nothing left for the strategic decisions in ToBe. If ToBe is treated as derivable from AsIs without judgment, modernization becomes mechanical translation — and the antipatterns this catalog is built against (*Jobol*, *Frozen architecture*, *Behavioural equivalence without ontology*) re-emerge under the appearance of rigour.

The three foundations interact. The three-layer recovery architecture says *what substrates exist* (L1, L2, L3). Source provenance discipline says *how the substrates trace back to legacy code*. The AsIs/ToBe ownership discipline says *who owns which substrate, in what mode of authority, with what governance*. None of the three is sufficient alone; together they hold the catalog's structural commitments.

Scoping note

This catalog instantiates the three-layer architecture for CICS COBOL modernization. The L1 vocabulary is calibrated to CICS (programs, paragraphs, CICS resources, COMMAREA, TRANSID, syncpoints); the L2 catalog is populated with CICS-specific patterns (the conversational, transactional, domain-shape, and routing patterns the modernization recovers); the L2 → L3 mapping rules encode CICS-specific projection decisions.

Whether the same three-layer structure applies to other legacy substrates — PL/I, RPG, Natural, JADE, Smalltalk — is a question this catalog does not answer. The principle of architect-gated multi-stage recovery is substrate-independent in shape; the specific substrates that populate each layer would need to be developed for each legacy stack. The CICS case is what's been validated inside the Rosetta prototype. Other practitioners working with other stacks may find the architecture applies; that work is not done here.

The provenance discipline is substrate-independent. Any legacy modernization, regardless of stack, can apply it. The cost is the same; the benefit is the same; the discipline is mechanical.

Part II: Tactical Generation

The patterns in this group address tactical design as DDD uses the term: how each bounded context materialises in modern code. Aggregates, domain events, handlers, the architecture chosen for each context. They presuppose strategic recovery from Part I — without bounded contexts identified and ubiquitous language established, generation produces structurally plausible code that doesn't reflect the domain.

The tactical patterns here are calibrated to AI-assisted generation. They specify what humans decide, what agents produce, what deterministic infrastructure renders, and how the boundaries between these stay clean.

Pattern 6: The Compiler Principle

Status: prototype-validated.

Context

An agentic system for software engineering, where LLMs participate in code generation. The system has both deterministic decisions (what scaffold to render, what bounded contexts exist, what types to declare) and probabilistic decisions (how to translate idiomatic COBOL into idiomatic C#, how to handle edge cases, how to phrase things). How these two kinds of work are organised determines whether the system is reproducible and auditable, or opaque and unreliable.

Problem

When the LLM is asked to make decisions of both kinds, the system becomes opaque and unreliable. Architectural choices made by the LLM aren't reproducible: ask twice, get different scaffolds. Boundary placements drift. Structural commitments are made on shaky ground. The deterministic infrastructure (compilers, linters, type systems) ends up checking outputs from a process that should never have produced them in that form.

The instinct to constrain the LLM through prompt engineering — telling it what kinds of decisions to make and which to defer — fails as the work scales. Prompts are advisory; the LLM treats them as suggestions, not contracts.

Forces

LLMs are good at language transformation, idiom translation, and contextual disambiguation. They are bad at structural commitment, architectural integrity, and reproducible decision-making. Deterministic tools are the inverse. Both kinds of work are necessary in modernization. Mixing them in a single agent surface produces the worst of both.

Pattern

Separate deterministic decisions from probabilistic ones at the architectural level, not at the prompt level. Put deterministic decisions in deterministic infrastructure: the graph holds architectural decisions, the architect validates them, the Roslyn-based emitter renders C# scaffolds from validated decisions. Put probabilistic work inside the rendered scaffold: the agents translate paragraph-level COBOL into method bodies inside handler classes whose shape is already determined.

The agent doesn't choose architecture. The architecture is rendered from validated decisions, and the agent fills in the constrained space.

The deterministic side is itself made of explicit artifacts. The graph schema is queryable. The IR types are inspectable. The scaffold rendering rules are code, not prompt content. The heuristics that classify paragraphs into tiers, that propose slice boundaries, that detect anti-corruption layer candidates — these live as first-class catalogs that specialised agents query, not as implicit knowledge baked into agent prompts. Determinism doesn't end at "the compiler exists"; it extends through every rule the compiler applies.

In DDD terms, this is the boundary between strategic design (which the human architect performs, with agent assistance for analysis) and tactical execution (which agents perform, within the scaffold that strategic decisions produced). Strategic decisions about bounded contexts, subdomain types, and aggregate boundaries are not LLM decisions. Tactical decisions about how a particular paragraph translates to a method body, given the scaffold, are.

Consequences

The system becomes reproducible at the architectural level. The same graph produces the same scaffold every time. The same scaffold constrains agents to the same kinds of work. Failures are localised: when something goes wrong, it's clear whether the failure is in the architectural decision (wrong bounded context) or the agentic translation (wrong handler body). Each can be debugged independently. For regulated industries this matters operationally — auditors can review architectural decisions separately from generated code, and the audit trail at each layer is independently inspectable.

The cost is engineering discipline. The deterministic infrastructure has to actually be deterministic, which means a real compiler/emitter (Roslyn SyntaxFactory) rather than templated string interpolation. The graph has to actually hold the decisions, which means a schema rich enough to express them. The boundary between deterministic and probabilistic work has to be defended actively.

The principle isn't novel in absolute terms — compilers have always been deterministic infrastructure with creative work happening inside their constraints. What is new is applying this division to the architecture of agentic workflows themselves: agents as the creative work, deterministic infrastructure as the harness around them. Anthony Alcaraz has named this "Pierre Milet Llobet's Project Rosetta

principle" — naming it explicitly because the field has not yet articulated it as a principle in its own right. Although Rosetta operates in mainframe modernization specifically, the principle itself applies wherever AI-assisted code generation must coexist with architectural integrity.

Related patterns

Pattern 7 (*The Intermediate Representation*) is the contract between the deterministic and probabilistic sides. Pattern 8 (*Tier-Aware Scaffolding*) operates entirely on the deterministic side. Pattern 9 (*Pluggable Emitters*) is a corollary: if the principle is correct, the deterministic side should be replaceable without disturbing the probabilistic side.

Pattern 7: The Intermediate Representation

Status: prototype-validated.

Context

A pipeline that has both a flexible substrate where analysis happens (the graph, where bounded contexts emerge and uncertainty lives) and a strict surface where generation happens (the emitter, where C# is produced through Roslyn). The two need to communicate. Naively connecting them couples the analysis to the generation in ways that resist evolution.

Problem

Without a typed contract between analysis and generation, every change to either side propagates everywhere. A new pattern discovered in the analysis (say, a new kind of saga) requires changing the emitter to render it. A new architectural target for the emitter requires re-deriving everything from the graph. The pipeline stiffens.

Forces

Analysis must remain flexible because the field is still discovering what to look for in legacy code. Generation must remain strict because rendering requires every piece of information in a specific shape. Both pressures are real and pull in opposite directions.

A second tension: the IR's vocabulary must be stable enough that downstream emitters can rely on it, but evolutionable enough that new analysis patterns can extend it. Treating the IR as frozen produces tools that can't grow with what's discovered. Treating it as fluid produces tools that break with every analysis change. Both extremes fail.

Pattern

Place a typed intermediate representation between the graph and the rendered scaffold. The IR is not one substrate — it is two, separated by the same compiler discipline that governs the boundary between agentic and deterministic work.

IR-Domain holds architectural intent. It captures commands, events, handlers, sagas, aggregates, side-effect surfaces — the tactical-design vocabulary of DDD as articulated by Eric Evans (2003) and elaborated by Vaughn Vernon (*Implementing Domain-Driven Design*, 2013). Each IR-Domain element is grounded back to the graph nodes that derived it. It is the machine-readable form of the modernization's architectural commitments: what aggregates exist, what consistency boundaries they enforce, what events they emit, what commands they accept, what sagas coordinate them.

IR-Domain is the substrate agents reason about. When an agent proposes a translation, it is grounding its proposal in IR-Domain elements — citing the aggregate this paragraph touches, the command this CICS transaction expresses, the events that follow from successful processing. IR-Domain is queryable, inspectable, reviewable; humans validate architectural commitments at the IR-Domain layer before any scaffold is rendered.

IR-Scaffold holds the structural blueprint. It captures class layouts, file paths, project structure, namespace organisation, interface definitions, test stub locations, the `scaffold-meta.json` constitutional contracts that the harness (Pattern 18) enforces. IR-Scaffold is a deterministic projection from finalized IR-Domain plus target conventions. There are no decisions in IR-Scaffold; given the IR-Domain and the chosen emitter (Pattern 9), the scaffold is what it is.

Agents do not reason about IR-Scaffold and do not modify it. They receive its output — the rendered C# files, the project layout, the test stubs — and fill the bodies the scaffold has carved out for them. The boundary between IR-Domain (where architectural intent lives) and IR-Scaffold (where structural rendering lives) is the boundary between deliberative work and compilation. Collapsing the two is the most common way the compiler principle (Pattern 6) silently fails: when agents can revise scaffold structure to accommodate their translations, architectural commitments stop being commitments.

The vocabulary of IR-Domain draws from the domain ontology (Pattern 4), not from the legacy's accidental naming. When IR-Domain captures an aggregate as `Customer`, that name comes from the canonical ubiquitous language the modernization team has validated — not from whatever the legacy happened to call it (`CUST-MAST-REC`, `CMR`, `CUSTREC01`). Where the ontology is not yet established, IR-Domain carries provisional names from vocabulary inference, marked explicitly as provisional and awaiting validation. The data layer's DDL informs aggregate boundary discovery: foreign keys reveal containment relationships, NOT NULL and CHECK constraints capture invariants. IR-Domain is one of the principal consumers of ontology recovery — without it, the architecture encodes the legacy's confusion in modern syntax.

IR-Domain is not a structurally neutral intermediate. It encodes architectural commitments: when IR-Domain captures a `HandlerIntent` with typed commands and events, it has committed to Vertical Slice Architecture for that bounded context; when it captures a `PortIntent` with adapter slots, it has committed to Hexagonal Architecture; when it captures a `SagaIntent` with choreography steps, it has committed to event-driven coordination. These are hard decisions — they structure the important constructs of the generated code. They are not LLM-mutable; they are validated by the architect during strategic recovery (Part I) and frozen into IR-Domain before IR-Scaffold is rendered.

The rendering from IR-Domain to IR-Scaffold to C# is deterministic and explicit. The emitter is implemented as Roslyn code generators — Roslyn is Microsoft's open-source C# and VB compiler platform, which exposes the compiler as a service: parsers, syntax trees, semantic models, and code generators are all addressable as APIs. Each architectural commitment in IR-Domain has a corresponding

Roslyn code generator that knows how to render it into IR-Scaffold, which in turn drives the rendered files. Changing architecture style — moving a bounded context from VSA to hexagonal, or from hexagonal to layered clean architecture — is not a prompt change. It is writing new Roslyn code generators that consume the same IR-Domain types and emit different IR-Scaffold shapes. IR-Domain stays stable; the rendering changes. This is what makes Pattern 9 (*Pluggable Emitters*) tractable: architecture diversity at the rendering layer, not at the analysis layer.

The split extends beyond code. The same IR-Domain feeds documentation emitters (Pattern 9) — C4 model views, aggregate maps, context maps, ubiquitous language glossaries — through their own deterministic emitter chains. Each documentation view is its own deterministic projection from IR-Domain. The vocabulary of IR-Domain, once stable, supports code generation, documentation rendering, and architecture-decision records from the same substrate.

In Rosetta, IR-Domain is called WolverineIntentModel because the code-emission target it most directly serves is Wolverine handlers. IR-Scaffold is materialized through Roslyn SyntaxFactory operating on WolverineIntentModel inputs and producing the rendered C# project. The naming is specific to the implementation; the principle — IR-Domain as architectural-intent substrate, IR-Scaffold as deterministic structural projection, neither one elidable into the other — is not.

Consequences

Analysis and generation evolve independently. New patterns extend IR-Domain without touching the emitter; new targets add emitters without touching the analysis. IR-Domain itself becomes the durable artifact of what the modernization understands about the legacy. For audit purposes, IR-Domain is the bridge: any generated artifact traces back through its IR-Domain origin to the graph nodes and ontology terms that grounded it. Auditors can inspect architectural decisions at the IR-Domain layer separately from the rendered code; each layer is independently inspectable.

The split between IR-Domain and IR-Scaffold makes the agentic / deterministic boundary visible at the IR layer itself, not just at the prompt-to-scaffold transition. When something goes wrong, it is clear which substrate is implicated. Architectural confusion shows up in IR-Domain (wrong aggregate boundaries, missing events, malformed sagas) where it can be debugged at the level of commitments. Rendering errors show up in IR-Scaffold (wrong file structure, missing test stubs, malformed scaffold metadata) where they can be debugged at the level of emission. Conflating the two would force every diagnostic to traverse both concerns simultaneously, which is what makes monolithic IRs hard to maintain.

The cost is a layer that looks dull but isn't. The IR has to actually be typed, which means real schema with real validation on both sides of the split. The grounding back to graph nodes has to be maintained through all transformations. IR-Domain's vocabulary has to stay aligned with the architectural intent the analysis is trying to capture, which means the IR's design is itself an ongoing concern.

The intermediate representation as concept comes from compiler theory — LLVM IR (Lattner & Adve, 2004), GCC GIMPLE, and earlier compiler IRs articulated decades ago that separating front-end analysis from back-end generation requires a typed intermediate. What this catalog contributes is the framing of IR as the contract between agentic analysis and deterministic generation specifically: the surface where heuristic-driven recovery and rule-driven scaffolding meet, with each side enforcing its own discipline at the boundary. The naming WolverineIntentModel reflects the target this IR most directly serves; the principle generalises to any agentic modernization where what's discovered must be rendered into

something compilable.

Related patterns

Pattern 6 (*The Compiler Principle*) is what motivates the IR's existence. Pattern 4 (*Domain Ontology as Independent Substrate*) provides the canonical vocabulary the IR uses — without ontology, the IR risks encoding the legacy's confusion in modern syntax. Pattern 9 (*Pluggable Emitters*) is what the IR enables on the generation side: architecture diversity at the rendering layer and documentation diversity at the view layer, with the IR as stable contract for both. Pattern 19 (*The Control Plane*) surfaces IR fragments to architects during review, making the architectural commitments visible before generation begins. The source provenance discipline (architectural interlude) extends through the IR — every IR element carries source coordinates back to the graph nodes that derived it.

Pattern 8: Tier-Aware Scaffolding

Status: prototype-validated.

Context

A modernization pipeline producing C# scaffolds for many bounded contexts within a single legacy system. Each bounded context has different domain weight — some are core to the business and likely to evolve significantly, others are generic supporting logic that should be cheap to maintain.

Problem

Most modernization platforms apply a single architectural template across the entire codebase. The result is that generic domains get over-engineered (hexagonal architecture for a lookup table is masochism) and core domains get under-engineered (vertical slices for a pricing engine collapse under their own weight in a few years).

The instinct to apply uniform architecture is operationally simpler — the scaffold renderer has one mode, every team works the same way, training is consistent. But the resulting code carries inappropriate complexity for most of the codebase and inadequate complexity for the parts that matter most.

Forces

Architectural complexity has real costs (more abstractions to maintain, more code to read, more places to introduce bugs) and real benefits (clearer boundaries, easier evolution, better long-term durability). Both costs and benefits scale with how much the code matters and how often it changes. Uniform architecture forces a single trade-off across the entire codebase.

Pattern

Annotate each bounded context with a domain tier based on importance and likely evolution. Use the tier to drive scaffold selection. The taxonomy is calibrated to CICS COBOL modernization but the underlying idea comes directly from Eric Evans' classification of subdomains in *Domain-Driven Design* (2003):

some subdomains are **core** to what differentiates the business, some are **supporting** of the core, some are **generic** capabilities the business needs but does not differentiate on. Each subdomain type deserves different architectural investment.

In Rosetta, this maps to three scaffold tiers:

- **Tier 0–1** (generic and supporting subdomains): vertical slice architecture. Easy to read, easy to change, no shared abstractions for thin domain logic. Generic capabilities — reference-data lookups, validation rules, audit logging — do not benefit from heavy structure.
- **Tier 2** (core subdomains with moderate complexity): hybrid scaffold. Vertical slices for request handling, lightweight domain models for the underlying business logic. The middle ground for business logic that has weight but isn't deeply differentiated.
- **Tier 3** (strategic core, the crown jewels): full hexagonal architecture. Domain at the centre, ports and adapters, the structure that pays its tax over the system's lifetime. This is where Evans' insistence on protecting the core domain from infrastructural concerns earns its keep.

The tier classification lives in the graph as an annotation on each bounded context. It can be derived heuristically (cyclomatic complexity, coupling, change frequency from version control if available) and validated by an architect. It is not an LLM decision.

Consequences

The architecture earns its complexity. Where complexity is justified, you pay for it. Where it isn't, you don't. Generic supporting code stays simple and cheap to maintain. Core code gets the structural investment it deserves.

The cost is the tier classification itself. Heuristic derivation is approximate; an architect must validate. The taxonomy may not transfer cleanly across all domains — some codebases may need different tiers, different boundaries, different scaffold shapes. The principle (match architecture to domain weight) is more general than the specific three-tier policy.

In the engagements I've examined, tier 2 dominates. Most code is moderately important business logic that doesn't deserve full hexagonal architecture but isn't trivial enough for pure vertical slices. The hybrid scaffold has earned its place by being the most-used.

Related patterns

Pattern 6 (*The Compiler Principle*) is what makes tier-based scaffolding tractable: the deterministic emitter can select scaffolds based on tier annotations without LLM involvement. Pattern 9 (*Pluggable Emitters*) is what makes the scaffold variants implementable as separate emitters.

Pattern 9: Pluggable Emitters

Status: prototype-validated.

Context

A modernization pipeline whose target architecture is one of multiple possible architectures. The choice of target depends on the engagement — some teams want vertical slice architecture (VSA) with Wolverine handlers, others want hexagonal architecture with explicit ports and adapters, others want a Java target instead of C#, others may want a hybrid approach for tier-2 bounded contexts that balances VSA velocity with hexagonal protection. The target may evolve as understanding of the legacy improves, or as the team's priorities shift across the lifetime of the modernization.

The same pipeline must also serve constituencies that consume something other than code. Architects reviewing strategic design need C4 model views. Data engineers validating ER models need entity-relationship diagrams. DDD practitioners checking aggregate boundaries need aggregate maps and context maps. Business analysts reading workflows need data flow diagrams. Operators tracing state need event maps. Regulators auditing decisions need architecture decision records. Each constituency wants a different view of the same architecture.

Problem

If the target architecture is hard-coded into the pipeline, switching targets requires rebuilding the pipeline. Teams either commit to one architecture upfront and live with the consequences, or they fork the pipeline per target and maintain divergent toolchains. Both options are operationally expensive.

If the target architecture is configured but the configuration is shallow — a few flags, a few template variants — the pipeline supports limited variation around a single target shape but can't address fundamentally different targets. A pipeline configured to produce vertical slice variants can't suddenly produce hexagonal scaffolds; the configuration vocabulary is too narrow to express the structural difference.

The deeper issue is that target architecture is a first-class decision that should live somewhere identifiable in the pipeline's design, not as a side effect of how the pipeline was constructed.

The same problem applies to documentation. Hand-authored documentation drifts. The diagrams produced for kickoff are inaccurate by month three; the wiki page that explained the architecture is stale by month six; the C4 model the architect drew on the whiteboard exists only in someone's screenshot folder. When the system changes — and it always changes — the documentation lags or fragments until the gap between system and documentation becomes unbridgeable, and the team gives up.

Auto-generated documentation from code has a different problem: it produces documentation at the wrong level of abstraction. Class diagrams that mirror code structure are useless for strategic understanding. Sequence diagrams generated from method calls capture mechanics but not intent. The strategic constituencies need views that exist *above* the code, not below it.

Forces

Different engagements have different correct targets for code. The same legacy may be modernized for different audiences with different architectural preferences — a bank's risk-engine team may want hexagonal for strategic-core contexts; the same bank's commodity reporting team may want lift-and-shift to VSA. Forcing a single target across all uses limits the pipeline's reach.

Multiple constituencies each want a different view of the architecture. Each view changes when the architecture changes. Manual maintenance is unsustainable across constituencies and over time. Auto-generation from code produces views at the wrong abstraction level for most constituencies.

But supporting multiple targets — code or documentation — requires architectural discipline: the pipeline must factor cleanly enough that the target-specific logic is replaceable. Every concession to target-agnosticism in the analysis side is a constraint on what the IR can express. Every concession to target-specificity in the emitter side is duplication when targets share structure.

The substrates that produced the code (graph, IR, ontology) already encode the strategic and tactical decisions — but they encode them as data, not as visual or narrative documentation. The same substrates that feed code generation can feed documentation generation if the architecture allows it.

Pattern

Make the emitter pluggable. The graph and the IR (Pattern 7) are target-agnostic; they describe the legacy and its architectural intent without committing to a specific output shape. The emitter is target-specific; it consumes the IR and produces an output in whatever target form is appropriate.

A new target requires a new emitter, not a new pipeline. The same graph, fed to the same IR, fed to a different emitter, produces a different output. The architectural decision lives in the choice of emitter — a deliberate, visible, replaceable choice — not in the pipeline's foundations.

Emitters fall into two categories that share the same compiler discipline.

Code emitters generate the modernized system itself. An emitter is responsible for the whole shape of the scaffold: directory structure, project files, dependency wiring, test scaffolds, build configuration, deployment metadata. A VSA emitter generates one shape; a hexagonal emitter generates another; a hybrid emitter generates a third. Each is its own Roslyn code generator — or, for non-C# targets, its own equivalent in the target language's tooling.

In Rosetta today, code emitters exist for vertical slice architecture (Wolverine, C#) and for hexagonal architecture (Wolverine + explicit ports, C#), with a hybrid emitter for tier-2 contexts that combines VSA velocity with hexagonal protection at specific seams. Future emitters could target Java with Spring Boot for organizations standardised on JVM stacks; or Jeremy Miller's Jade for event-sourced workloads where Wolverine isn't the best fit; or even non-handler architectures like CQRS-with-separate-read-models for analytical bounded contexts.

Documentation emitters render architectural views from the same substrates. The principle generalises: the same compiler architecture that allows a vertical-slice emitter and a hexagonal emitter to consume the same IR allows a C4-diagram emitter and an ER-diagram emitter to consume the same substrates. The emitters are different; the substrates are the same. Documentation is not authored in parallel with the system; it is rendered as a view of the system.

Specific views the catalog of documentation emitters can produce:

- **C4 model views** — rendered from bounded-context boundaries (graph community detection), deployment intent (IR), and external system relationships.
- **Entity-relationship diagrams** — rendered from data structures in the graph and aggregate boundaries in the IR; one ER diagram per bounded context.

- **Aggregate maps** (DDD-specific) — rendered from IR aggregate definitions and graph relationships; shows aggregate roots, contained entities, value objects, emitted events.
- **Context maps** (DDD-specific, Evans 2003) — rendered from bounded-context relationships in the graph and command/event flows in the IR; shows upstream/downstream relationships, anti-corruption layers, shared kernels, integration patterns.
- **Data flow diagrams** — rendered from side-effect surfaces in the graph and command flows in the IR.
- **Ubiquitous language glossaries** — rendered from the domain ontology (Pattern 4) and IR vocabulary; one glossary per bounded context.
- **Architecture Decision Records (ADRs)** — linked from spec deltas (Pattern 19); when a spec delta articulates an architectural change, an ADR is produced as part of the change.

The documentation lives in markdown and standard diagram formats (PlantUML, Mermaid, SVG) so it integrates with the engineering workflow — versioned in Git alongside the code, viewable in pull requests, included in pipelines. Each view is a file generated by its emitter; regenerating is idempotent and fast.

The IR's job is to remain stable across emitter additions, code or documentation. When a new emitter is added, the IR shouldn't need new types — the existing IR types (HandlerIntent, AggregateIntent, CommandIntent, EventIntent, BoundedContextIntent) should be expressive enough that the new emitter can interpret them in its own architectural or documentation idiom. If a new emitter requires new IR types, that's a signal the IR was implicitly biased toward existing targets.

When humans need to *change* the architecture, they don't edit the documentation directly. They edit the substrates — refining the ontology, updating the IR, adjusting the graph annotations — and the documentation re-renders to reflect the change. This is what Cyrille Martraire (*Living Documentation*, 2019) called the discipline of making documentation a byproduct of doing the work. This pattern operationalises that discipline through the same compiler architecture that produces the code.

Consequences

The architectural decision lives in the choice of emitter, not in the graph or the IR or the agents. That decision becomes explicit, reviewable, replaceable. Teams modernize their codebase against the architecture they want, not against the one the pipeline imposed by accident.

Documentation never goes stale relative to the system it describes. Reviewers validate decisions at the abstraction level appropriate to them: architects check the C4 model, data engineers check the ER diagrams, DDD practitioners check aggregate and context maps. Each constituency engages with documentation that is canonical for them, generated from the same source of truth that produced the code. The substrates themselves become the canonical "documentation" — diagrams and markdown documents are convenience views rendered for human consumption. This inverts the traditional relationship: substrates are not artifacts derived from documentation; documentation is artifacts derived from substrates.

Emitters compose into a library that grows over time. The first engagement contributes the first emitter; the second engagement may reuse it or add a variant. After several engagements, the catalog of emitters covers the most common code targets and documentation views, and new engagements typically need to extend an existing emitter rather than write a new one from scratch. The investment compounds across the engagement portfolio.

The cost is the discipline of keeping the IR target-agnostic. Every IR concept must be expressible in any reasonable target architecture and renderable in any reasonable documentation view, which constrains the IR's vocabulary. Adding a new target may reveal that the IR has been quietly target-specific in places, requiring refactoring. The IR's stability across emitter additions is something to maintain deliberately, not assume.

Reviewers in the control plane (Pattern 19) can request a specific view to validate a decision. Spec deltas (Pattern 19) include the documentation re-renders as part of the change package; reviewers see what the architecture looked like before, what it looks like after, and which substrate changes produced the difference.

The status of documentation emitters specifically is *designed*. Code emitters are *prototype-validated* in Rosetta — the VSA and hexagonal emitters work today. The documentation emitter library is the natural extension of the same architecture but has not been built yet. The principle follows directly from the code emitter discipline; the gap is in the catalog of emitter implementations, not in the underlying approach. The combined pattern is reported as *prototype-validated* because the architectural principle has been validated through the code emitter side; the documentation emitter side is implementation work that the principle authorises.

Pluggable target architectures are a long-established practice in compiler back-ends — LLVM (Lattner & Adve, 2004) is the canonical modern example, with a single front-end feeding multiple architecture-specific back-ends for x86, ARM, RISC-V, and so on. What this catalog contributes is two things. First, the application of the back-end pluggability principle to *architectural styles* rather than to *machine architectures*: the back-end isn't a CPU instruction set, it's an architectural pattern (VSA, hexagonal, event-sourced). Second, the extension of the same compiler discipline to *documentation* as another emitter target: the principle that what gets emitted is replaceable, code or documentation, as long as the substrates upstream stay stable.

Related patterns

Pattern 6 (*The Compiler Principle*) is what makes pluggable emitters possible: deterministic emission is what's being plugged in. Pattern 7 (*The Intermediate Representation*) is what the emitters consume — and is what must remain target-agnostic for pluggability to work. Pattern 8 (*Tier-Aware Scaffolding*) is one application of the pluggable-emitter pattern: tier policy is implemented as emitter selection (tier-0/1 uses VSA emitter; tier-3 uses hexagonal emitter; tier-2 uses hybrid). Pattern 3 (*The Graph as Projection*) and Pattern 4 (*Domain Ontology as Independent Substrate*) are the substrates documentation emitters consume alongside the IR. Pattern 19 (*The Control Plane*) is where rendered documentation surfaces for human consumption and where spec deltas integrate documentation re-renders into change packages.

Pattern 10: Commands and Events as Logical Boundary, Independent of Physical Deployment

Status: prototype-validated.

Context

A modernized system structured as a set of bounded contexts, each generated through the patterns above. The bounded contexts must communicate with each other to implement business workflows that span them. The communication may need to be synchronous in-process initially (for performance, for transactional simplicity, for deployment minimalism) but may need to evolve toward asynchronous messaging or distributed services later (for scaling, for team boundary alignment, for operational independence).

Problem

Most modernization projects pick a deployment model — monolith or microservices — and bake it into the architectural structure. The communication between bounded contexts is modeled as direct method calls (in monoliths) or as REST/RPC boundaries (in distributed systems). Either choice is hard to reverse: switching from method calls to messaging is a major refactor; switching from messaging back to method calls is also a major refactor.

The deeper problem is conflating two distinct decisions. *What contexts communicate, and what they communicate* is a logical question — which slice produces what command, which slice handles which event, what business intent is being expressed. *How they communicate, and where they're deployed* is a physical question — sync or async, in-process or over the network, monolith or microservices. The two questions have different answers at different times in the system's life. Conflating them couples the logical model to the deployment model in ways that resist evolution.

Forces

In-process synchronous communication is operationally simpler — fewer moving parts, easier debugging, stronger transactional guarantees. Distributed asynchronous communication is more scalable — independent deployment, fault isolation, team autonomy. Most systems start in conditions where the simpler choice is correct, but evolve toward conditions where the more scalable choice becomes necessary. The transition shouldn't require an architectural rewrite.

CICS transactional guarantees specifically must survive any transition. The legacy system has been preserving consistency semantics for decades; the modernized system loses something important if those semantics drop quietly during decomposition.

Pattern

Model communication between bounded contexts as commands and events at the logical level, regardless of how they're physically implemented. A command expresses *something the system should do*; an event expresses *something the system has observed*. Both are first-class artifacts in the architecture: typed, named, owned by specific contexts, traceable through the audit trail.

The physical implementation is a separate concern. Initially, commands and events flow as in-process method calls within a modular monolith — the contexts share a process, communication is synchronous, transactional boundaries are simple. The framework underlying the implementation (Wolverine, Jade, or equivalents) handles the dispatching, but the public surface of each context is the command/event vocabulary, not method signatures.

When operational pressure justifies extraction — scaling demands, team-boundary alignment, deployment frequency divergence, regulatory isolation — the transition is mechanical. The same commands and events that flowed in-process now flow through messaging infrastructure. The receiving context's handlers are unchanged; the sending context's call sites are unchanged. What changes is the transport. The transactional boundary changes shape (eventual consistency replaces immediate consistency where appropriate), but the vocabulary stays the same.

The discipline: never let method-level coupling sneak in between contexts. Every cross-context interaction must go through a command or an event. The framework enforces this by making cross-context method calls inconvenient or impossible. Bounded contexts communicate only through their command/event surface.

Consequences

The architectural decision of *what to deploy where* becomes operational and reversible. Teams can extract a context to its own service when operations justify it, without rewriting the consumer code. Teams can pull a struggling service back into the monolith if the operational complexity isn't paying for itself. The decomposition is a flywheel, not a one-way ratchet.

Where the modernized system must coexist with parts of the legacy that are not yet migrated, an **anti-corruption layer** (Eric Evans' term for a translation layer that prevents legacy concepts from polluting the modernized domain model) intercepts the legacy interaction and translates between the legacy's vocabulary and the canonical ubiquitous language of the modernized side. The anti-corruption layer is itself a bounded context with a single responsibility — protect the modernized domain from inheriting whatever the legacy still calls things.

CICS-style transactional guarantees survive the transition because the framework provides them. Wolverine's transactional outbox pattern, for example, ensures that commands published as part of a transaction are delivered reliably even when the recipient is now a different service. The consistency semantics are preserved through infrastructure, not through hoping nothing breaks.

Architectural decisions become localized. The choice of in-proc vs. distributed lives in the deployment configuration, not in the source code. The choice of which contexts share a process can change without code changes. Architectural evolution becomes operational, which is what mature engineering organizations need.

The cost is the discipline. Every cross-context interaction must be modeled as a command or event, which adds ceremony compared to just calling a method. New developers must learn the discipline; tools must enforce it; reviews must check it. The framework handles most of the mechanical cost, but the cognitive cost of thinking in commands and events is real, especially for teams used to method-level coupling.

This pattern is what allows the decomposition flywheel — the post-cutover architectural evolution from modular monolith toward selective service extraction — to work as a continuous process rather than as a major project. The architectural shift is mechanical because the logical boundaries were always there; only the physical boundaries change.

The same logical-boundary discipline supports two transitional techniques Nick Tune has named: *Expose Legacy Asset*, in which the legacy publishes events that modernized subsystems consume as their integration surface; and *Republishing Legacy Events*, in which an autonomous bubble consumes those

legacy events and re-emits them in the canonical target vocabulary, allowing downstream consumers to decouple from the legacy model before the migration is complete. Both are forms of the same idea — the logical contract carries across the physical transition — applied at the legacy boundary rather than at the modernized boundary. Definitions and pointers are in the glossary.

Related patterns

Pattern 7 (*The Intermediate Representation*) captures commands and events as first-class IR concepts; the IR's vocabulary is what makes this pattern possible at scaffold-rendering time. Pattern 9 (*Pluggable Emitters*) is what allows different physical implementations to be selected — an emitter for in-process Wolverine, an emitter for distributed Wolverine, an emitter for Jade for event-sourced workloads. Pattern 18 (*The Harness as Self-Observing State Machine*) governs the transition from one physical deployment to another, ensuring that extraction or consolidation is auditable and reversible.

Part III: Verification

The patterns in this group address how the modernization knows the generated code is right. They presuppose Pattern 2 (*The Legacy as Oracle*) — without a live ground truth to compare against, verification reduces to test suites that the agents wrote.

Pattern 11: Transactional Boundaries as First-Class Migration Concern

Status: in construction.

Context

A modernization translating CICS COBOL transactions to C# handlers. The legacy system has implicit transactional boundaries everywhere: a CICS unit of work spanning from `EXEC CICS RETURN TRANSID` to the next `RETURN`, an IMS transaction surrounding a series of database updates, a batch JCL job step with checkpoint/restart semantics, an `EXEC CICS SYNCPOINT` marking a commit boundary. These boundaries are not decorative — they preserve invariants the business depends on. When the unit of work commits, related changes commit together; when it rolls back, related changes roll back together; when it fails, the system is in a known recoverable state.

The modernization must preserve these invariants. But the modern target — C# handlers, distributed services, event-driven workflows — has very different transactional primitives. Naive translation produces code that looks structurally similar but loses the transactional guarantees the legacy provided.

Problem

Transactional boundaries in mainframe systems rarely survive migration intact. The default failure mode is silent: the modernization preserves syntactic structure but loses transactional semantics. The customer believes they have atomicity and discovers in production that they don't — partial updates persist, compensating transactions never fire, edge-case failures leave the system in inconsistent states that the legacy would have prevented.

The silent failure is the danger. If transactional boundaries were preserved or visibly violated, the team could see the problem. Because the violations are invisible — the code compiles, the tests pass, the happy path works — they accumulate until production traffic exposes them, often months after cutover.

Three categories of failure dominate. First: lost atomicity, where two changes that committed together in CICS now commit independently in the modernized system, opening windows where intermediate states leak. Second: lost rollback semantics, where a failure that would have aborted the entire transaction in the legacy now leaves partial state behind in the modern. Third: lost isolation, where reads that saw a consistent snapshot in CICS now see partial updates from concurrent transactions in the modern system.

Forces

CICS transactional guarantees are strong — pessimistic locking, distributed two-phase commit across DB2 and VSAM, recoverable units of work survived process restarts. Modern systems often relax these guarantees for scaling reasons — eventual consistency across services, optimistic locking within a service, sagas instead of distributed transactions. The relaxation is sometimes correct (the legacy's guarantees may have been over-engineered for the actual business need) and sometimes catastrophic (the business genuinely needs the guarantee, and silently losing it produces production failures).

Distinguishing the two cases requires explicit analysis, not implicit translation. The team must articulate what each transactional boundary guarantees, decide whether the guarantee is essential to the business, and design the modern equivalent — strict preservation, controlled relaxation, or saga compensation — with the trade-offs visible.

There is also a regulatory force. Banks, insurers, and government systems often have audit requirements about transactional behaviour: a settlement is atomic; an audit log is durable; a regulatory report sees a consistent snapshot. Losing these guarantees silently is not just a technical bug; it is a compliance failure.

Pattern

Treat transactional boundaries as first-class migration artifacts. During strategic recovery (Part I), identify every transactional boundary in scope: CICS units of work, IMS transactions, batch JCL job steps, `EXEC CICS SYNCPOINT` commit points, database transactional scopes. The graph (Pattern 3) captures these as annotated edges and nodes; analysis surfaces them as candidate boundaries for review.

For each transactional boundary, document the invariants it preserves: which writes commit together, which reads see consistent snapshots, what rolls back together when a failure occurs. These invariants are part of the domain ontology (Pattern 4) — they are statements about what the business considers atomic, not statements about how the implementation happens to enforce atomicity.

Decide explicitly how each boundary maps in the modernized system. Three categories of decision:

- **Preserved strict** — the modern system preserves the legacy's transactional guarantees through equivalent mechanisms (a single-aggregate transaction in C#, a Wolverine handler with implicit transactional scope, a distributed transaction with two-phase commit where the data layer supports it). Used when the business genuinely requires the guarantee.
- **Relaxed deliberately** — the modern system intentionally loosens the guarantee, with explicit documentation of what's been relaxed and why. For example, eventual consistency across two bounded contexts where the legacy had distributed transactions. Used when analysis shows the strict guarantee was unnecessary and the relaxation enables better scaling.
- **Replaced with saga compensation** — the modern system implements the legacy's atomicity through saga choreography: each step is locally atomic, with explicit compensating actions if

a later step fails. Used when distributed transactions are not viable and the business requires effective atomicity.

The decision is documented in the IR (Pattern 7) as part of the architectural commitment for each bounded context. Generated code includes the chosen mechanism — Wolverine's transactional outbox, MartenDB's session-scoped transactions, saga state machines. Twin Verification (Pattern 13) specifically tests transactional behaviour: simulate failures at every commit point, verify that the modernized system's recovery matches the legacy's.

CICS-specific cues guide the analysis. `EXEC CICS SYNCPOINT` marks a commit point; `EXEC CICS ROLLBACK` marks an explicit abort; `EXEC CICS START TRANSID` marks the beginning of a new transactional unit; pseudo-conversational boundaries (`RETURN TRANSID / COMMAREA`) mark transitions between transactional contexts. Each is a decision the original programmer made about what should be atomic. The modernization team must consciously confirm or revise each decision.

Consequences

Transactional behaviour becomes a designed property of the modernized system, not an accident. When production exposes a transactional issue, the team can trace it back to a specific recovery decision — "we chose relaxed eventual consistency here; the production traffic shows that decision was wrong, and we need to redesign as a saga." Without the explicit decision trail, the team can only debug the symptom.

The pattern surfaces over-engineering. The legacy may have wrapped operations in transactions that did not need to be atomic — a habit of CICS programming rather than a business requirement. Explicit analysis distinguishes "this must be atomic" from "this happened to be atomic," allowing the modernization to relax constraints where appropriate and gain scaling room.

The cost is the analysis discipline. Every transactional boundary in scope must be reviewed, documented, and decided. For systems with thousands of transactions, this is non-trivial work. It cannot be automated entirely — analysis can surface the boundaries and suggest classifications, but business decisions about which guarantees are essential require human judgment. The capability map (Pattern 1) helps prioritise: strategic-core capabilities get deep transactional analysis; commodity capabilities can default to preserved strict for simplicity.

Vaughn Vernon's *Implementing Domain-Driven Design* (Vernon, 2013) articulates aggregate consistency boundaries as the canonical DDD anchor for transactional discipline. Pat Helland's work on eventual consistency and the limits of distributed transactions (Helland, 2007) informs the relaxation decision. Saga compensation patterns trace back to Garcia-Molina and Salem (1987) and have been extensively elaborated in microservices literature. What this catalog contributes is the framing of transactional boundaries as first-class migration artifacts in mainframe modernization specifically, where CICS conventions encode transactional decisions that must be consciously confirmed or revised — not silently translated.

Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines which capabilities deserve deep transactional analysis. Pattern 3 (*The Graph as Projection*) captures the transactional cues in the legacy. Pattern 4 (*Domain Ontology as Independent Substrate*) holds the invariants as canonical statements about the business. Pattern 7 (*The Intermediate Representation*) encodes the chosen transactional mechanism per

bounded context. Pattern 10 (*Commands and Events as Logical Boundary*) provides the cross-context communication primitives that sagas use. Pattern 13 (*Twin Verification*) verifies transactional behaviour, not just functional behaviour. Pattern 14 (*Hypothesis-Driven Verification*) categorises transactional violations as a specific divergence class. The *Silent Semantics Loss* antipattern names what happens when transactional boundaries are translated implicitly rather than explicitly.

Pattern 12: Transitional Architecture: The Modular Monolith as Migration Vehicle

Status: prototype-validated.

Context

A modernization that will run across many months — sometimes years — moving capabilities from mainframe to modern stack. The team has classified capabilities (Pattern 1), identified bounded contexts, decided the target architecture for each context. The temptation now is to design the *target state* — the eventually-distributed microservices architecture, the cloud-native deployment, the service mesh, the event bus — and start building toward it directly.

This temptation needs resisting. The target state is the destination, not the path. The architecture *during* the migration matters as much as the architecture at the end — sometimes more — because the migration's duration is when the team learns whether the boundaries it drew on a whiteboard survive contact with real code.

Problem

Modernization teams fragment too early. Microservices feel modern; the cloud-native deployment is what the customer paid for; the architecture diagrams show separate services with their own databases. The team begins extracting bounded contexts into separate services from day one — each with its own deployment pipeline, its own data store, its own operational overhead. The result: an immature decomposition prematurely committed to physical separation, with all the operational complexity of distributed systems and none of the scaling benefits.

The operational cost is real. Each extracted service brings: a deployment pipeline, observability stack, alerting rules, on-call rotation, network configuration, security policies, data store, backup strategy, integration tests. For a team modernizing 30 bounded contexts, that is 30 services to operate during the migration period — when the team is *also* keeping the legacy mainframe running, *also* learning the modern stack, *also* discovering that the bounded contexts they drew on a whiteboard need refinement once they encounter the real code.

The deeper failure is decomposing before the boundaries have earned their independence. A bounded context whose boundary is provisional should not be a service yet; it should be a module inside a larger deployable unit, where boundary changes are cheap. Premature service extraction freezes provisional boundaries into operational reality.

Forces

The target architecture may genuinely be distributed services — that's the modernization's eventual destination. But the migration period needs different optimization criteria than the destination. During migration: minimise operational overhead, maximise boundary flexibility, accelerate learning about which boundaries are real. After migration: optimise for independent scaling, independent deployment, independent team ownership where those properties are needed.

The modular monolith resolves the migration-period forces. Bounded contexts are explicit — separate modules, separate vocabularies, separate data ownership within the deployable — but they share a single deployment, a single observability surface, a single database (with schema separation), a single set of operational concerns. Boundary changes are cheap because they don't cross network boundaries. Operational overhead is minimal because there is one thing to deploy.

When a bounded context's boundary stabilises and its scaling/deployment/ownership profile diverges from the rest, *then* it earns extraction into its own service. The extraction is informed by operational evidence, not by architectural aspiration.

Pattern

Default to the modular monolith as the transitional architecture for mainframe modernization. Bounded contexts are first-class within the monolith — each has its own module, its own ubiquitous language, its own data ownership (enforced through schema separation or row-level ownership rules), its own aggregate roots. Communication between contexts happens through commands and events (Pattern 10), with the same logical contracts that a distributed system would use. The contracts survive any future extraction.

Within the monolith, the boundaries are enforced through deliberate mechanism, not through hope. .NET's `internal` access modifier combined with assembly boundaries provides one level of enforcement: each bounded context is its own assembly; cross-context access goes through public interfaces; internals are invisible across the boundary. ArchUnit-style architecture tests (NetArchTest in the .NET ecosystem) provide a second level: rules that fail the build when a forbidden dependency is introduced. Custom Roslyn analyzers provide a third level: source-level enforcement that surfaces violations as compile-time errors with specific diagnostics. The three layers together approximate the compile-time enforcement that a language with stronger module systems (Rust crates, Java's JPMS, Swift modules) would provide natively. The discipline is mechanical and the enforcement is real, but the engineering cost of building it is real too.

Operational concerns are unified during the migration. One deployment pipeline, one observability stack, one database (with schema separation), one set of integration tests. The team's operational attention is spent keeping the legacy mainframe running and learning what the modern stack actually requires — not on managing 30 services.

Extraction comes later, when specific bounded contexts have earned their independence. The extraction criteria are operational:

- **Scaling profile diverges** — this bounded context's traffic shape is different from the rest, and shared scaling produces waste (over-provisioning the quiet contexts, under-provisioning the busy one).
- **Deployment cadence diverges** — this bounded context changes frequently while others are stable, and shared deployment produces unnecessary risk for the stable contexts.

- **Team ownership diverges** — this bounded context belongs to a team that wants independent deployment authority, separate from the modernization team's release cycle.
- **Compliance boundary diverges** — this bounded context has regulatory or security requirements distinct from the rest, and shared deployment confuses the audit boundary.

When one of these criteria applies, the bounded context is extracted into its own service. The extraction is straightforward because the contracts already exist — commands and events flow over network instead of through in-process method calls, but the logical boundary is unchanged. Pattern 10 (*Commands and Events as Logical Boundary*) makes this extraction mechanical, not architectural.

The modular monolith is also what the agentic platform (Pattern 15) is. Bounded MCP servers are modules within a single platform, not separate services. The recursion is deliberate: the discipline that works for the modernized business system works for the platform that produces it.

The modular monolith is one transitional shape; other shapes are sometimes appropriate within the same modernization. Nick Tune has documented two complementary forms: the *Bubble*, in which a new subsystem with a clean target model accesses legacy data exclusively through an anti-corruption layer without local persistence; and the *Autonomous Bubble*, in which the new subsystem holds a local data store populated asynchronously from legacy events. The modular monolith is the right default when the modernization owns both code and data; the Bubble forms are useful when the target model can be designed independently but the underlying data must remain in the legacy during the transition. Definitions and pointers are in the glossary.

Consequences

Operational complexity during migration is minimised. The team can focus on the modernization work itself — slice discovery, scaffold generation, behavioural verification — without simultaneously absorbing the cost of operating distributed systems. The migration completes faster because fewer things compete for operational attention.

Boundary refinement is cheap. As the team discovers that a bounded context drawn on a whiteboard needs to be split, or that two contexts should be merged, or that an aggregate boundary needs to move, these changes are local refactorings inside the monolith — not cross-service migrations with their attendant data movement and contract negotiation.

The path to distributed services remains open. Bounded contexts that earn their independence are extracted with minimal architectural disruption because the contracts are already explicit. The modular monolith is not a permanent destination; it is a way station that preserves optionality.

The cost is the discipline of resisting the fragmentation instinct. Teams trained in cloud-native architectures may feel that the modular monolith is "not modern enough" — that microservices are what 2026 looks like. This is wrong for migration contexts but socially difficult to articulate. The catalog must make the case explicitly, repeatedly: distributed services are an *outcome* of validated boundaries, not a *starting point*.

The discipline also requires that the modular monolith be built correctly. A monolith with all modules sharing a database freely is not modular; it is a big ball of mud with hopeful naming. Schema separation must be enforced, public interfaces must be the only cross-module access path, the compile-time enforcement must actually compile.

Sam Newman's *Monolith to Microservices* (Newman, 2019) provides the canonical articulation of incremental extraction strategies. Simon Brown's modular monolith advocacy (Brown, multiple talks 2017-2023) names the architectural pattern this catalog applies. What this catalog contributes is the framing of the modular monolith specifically as the *transitional architecture for mainframe modernization* — where the migration period's operational pressures are extreme and the boundary uncertainty is high, both of which favour deferred decomposition. The principle (don't fragment before boundaries earn independence) generalises beyond mainframe; the specific application to long-running brownfield mainframe migrations is what's articulated here.

Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines which contexts will eventually need independent extraction. Pattern 8 (*Tier-Aware Scaffolding*) operates within the modular monolith — tier-2 and tier-3 contexts may eventually be extracted; tier-0 and tier-1 may remain modules permanently. Pattern 9 (*Pluggable Emitters*) renders each module's scaffold; extraction changes the deployment target but not the emitter. Pattern 10 (*Commands and Events as Logical Boundary*) is what makes future extraction mechanical — the logical boundary survives the physical change. Pattern 15 (*Bounded MCP Servers*) applies the same discipline to the agentic platform itself. The *Frozen Architecture* antipattern names what happens when modernization adopts the legacy's accidental decomposition rather than designing the right transitional architecture.

Pattern 13: Twin Verification

Status: prototype-validated.

Context

An agentic workflow generating C# code from COBOL, paragraph by paragraph, slice by slice. The agents need to know whether each candidate they produce is correct — behaviourally equivalent to the original legacy paragraph it replaces. Verification has to be fast enough to be part of the agent's inner loop (the iteration cycle in which agents produce, evaluate, refine, repeat), not a downstream activity that happens after the agent has committed to a candidate. The legacy is available as oracle (Pattern 2).

The inner loop is where most of the work happens. An agent typically produces multiple candidates per paragraph — initial translation, refinement after first verification, response to divergence diagnostics, alternative approaches if initial ones fail. Each candidate needs verification. If verification is slow, the agent's exploration is constrained by latency; if verification is fast, the agent can explore aggressively.

Problem

When verification is slow (running against a remote mainframe, running through a CI pipeline, running against a shared test environment with queueing), the agents can't iterate effectively. They produce a candidate, wait minutes for verification, get a verdict, try again. Most of the wall-clock time is spent waiting. The quality of the iteration suffers because each step is expensive — the agents conserve attempts,

miss exploratory opportunities, settle for early candidates that pass rather than refined candidates that pass cleanly.

There is also a correctness problem. When verification is slow, teams cut corners: they verify against test suites the agents themselves wrote (creating the homework-grading problem Pattern 2 addresses), or they verify against sampled inputs rather than representative inputs, or they skip verification at intermediate stages and verify only at the end. Each shortcut reduces confidence in the final result.

Forces

Real verification against the actual legacy mainframe environment is authoritative but slow and expensive — network latency to the mainframe, cost of mainframe cycles, queueing for shared resources, security and access restrictions. Synthetic verification (test suites, mocks, fixtures) is fast but disconnected from ground truth — it tests what the team thought to test, not what the legacy actually does.

Local approximations of the legacy can be both fast and authoritative if the approximation is faithful enough. The faithfulness requirement is strict: the local oracle must produce outputs indistinguishable from the legacy's for the inputs being exercised, or the verification verdicts are unreliable. A local approximation that's 99% faithful is worse than no local approximation, because the agents trust it.

Pattern

Compile the legacy to a portable runtime form and package it as a local container — the Legacy Twin. Run the candidate C# and the Legacy Twin against the same inputs. Compare outputs in-process. Treat any divergence as failure until proven otherwise.

Make the loop fast — milliseconds, not minutes — by keeping everything local: the Twin runs in a Docker container on the same machine as the agent; inputs come from local fixtures; comparison happens in-memory; verdicts return synchronously. The agent generates a candidate, the candidate runs against the Twin, the verdict is immediate, the agent has direct evidence of what's right and what's wrong.

The Twin's faithfulness is non-negotiable. The compilation from legacy to portable form must preserve the legacy's exact semantics — every edge case, every error path, every transactional guarantee, every type coercion. The Twin is not an emulation; it is the legacy code running in a different runtime environment. Its outputs are the legacy's outputs.

Divergence diagnostics are first-class. When the candidate's output differs from the Twin's, the system doesn't just report "FAIL" — it reports a structured diff: which field diverged, what the candidate produced, what the Twin produced, what input caused the divergence, what provenance trail (architectural interlude) leads back to the legacy code that defines the expected behaviour. The agent uses the diagnostic to refine the candidate; the diagnostic becomes the next input to the inner loop.

In Rosetta, the Twin is Raincode-compiled COBOL packaged as a Docker container. Raincode is one of the few tools that compiles COBOL to .NET IL with sufficient fidelity to make this approach practical. The implementation is specific to CICS COBOL; the principle (local containerised oracle in the inner loop) is more general — any legacy stack with a faithful-enough portable compilation target can apply it.

Consequences

The agents iterate against evidence in real time. The cost of being wrong drops, which lets the agents explore more aggressively — try multiple translations, evaluate alternatives, refine until the result is good rather than settling for the first thing that passes. The cost of being right stays the same, which keeps verification honest. The agents converge faster and more honestly because they're matching behaviour the legacy actually exhibits, not behaviour someone wrote down.

Behavioural equivalence becomes the verification standard. Tests check what we thought to test; behavioural equivalence checks the actual output for whatever inputs we exercise. When you have the legacy as oracle, you don't need to imagine in advance what the edge cases are — the Twin exhibits them when exercised.

The cost is the Twin itself. Raincode is one of the few tools that can compile COBOL to .NET IL well enough to make this practical. For other legacy stacks, equivalent tooling may or may not exist; without faithful compilation, the pattern doesn't apply. The Twin must be kept in sync with the legacy if the legacy evolves during the modernization. The container must be packaged, distributed, and maintained — operational concerns that don't exist if verification runs only against the original mainframe.

There is also a coverage caveat. Twin Verification confirms behavioural equivalence for inputs the agents exercise; it does not exhaustively prove equivalence for all possible inputs. Production traffic includes inputs no dev-time corpus anticipated. This is why Pattern 14 (*Hypothesis-Driven Verification*) extends Twin Verification from dev mode into production — production becomes the corpus that catches what dev-time exercise missed.

Related patterns

Pattern 2 (*The Legacy as Oracle*) is the foundational principle this pattern operationalises — the legacy is the source of truth, and the Twin is how the agents access that truth in the inner loop. Pattern 14 (*Hypothesis-Driven Verification*) extends Twin Verification from dev mode to production mode using Witness telemetry. Pattern 18 (*The Harness as Self-Observing State Machine*) treats Twin Verification as a deterministic gate the agents must pass before promotion. The source provenance discipline (architectural interlude) is what makes divergence diagnostics traceable back to specific legacy code.

Pattern 14: Hypothesis-Driven Verification

Status: in construction.

Context

A modernization that has succeeded against dev-time verification (Pattern 13) and is moving to production. In dev mode, the candidate C# matches the Legacy Twin's behaviour over the test corpus. The question is whether it will match in production, where the corpus expands to whatever traffic the system actually receives.

The dual-run period — during which legacy and modernized C# both process real traffic — is finite. Eventually one is decommissioned. Whatever the modernization learns about production must be

captured during this window or it's lost. And the captures, once accumulated, become an artifact in their own right: a corpus of how the system actually behaves under real workloads, potentially valuable beyond the immediate question of behavioural equivalence.

Problem

Dev-mode verification covers what was exercised. Production reveals what the dev-time corpus didn't anticipate: temporal coupling, end-of-month patterns, edge cases triggered by specific data shapes, environmental dependencies. The behaviour is real, the legacy handles it correctly, and the dev-mode verification has no way to know about it.

Charity Majors has argued for years that you can't stage your way to production knowledge. Distributed systems are hostile to mirroring. What production reveals isn't in the tests or the spec — it's in the running system itself.

A second problem appears once production verification is operational. The captured behaviour is rich evidence, but it's not yet a specification. It's traffic patterns, request-response pairs, scenarios. The traditional approach to specifications is to write them before development: someone articulates what the system should do, and developers implement it. In legacy modernization this is doubly broken. The original specification was never written; whatever specs exist are reconstructions. And the modernized system is replacing something that already runs, which means the running system itself is more authoritative than any reconstruction.

What to do with the captures as a long-term artifact becomes its own question. They are evidence during the dual-run. They could become specifications afterwards. The modernization team has to decide whether to extract that value or leave the captures as transient logs.

Forces

Production verification needs the same tight feedback as dev-time verification but operates on a different scale and at different stakes. The dual-run period is finite; eventually one system is decommissioned. The traditional approach (write more tests after production reveals issues) is reactive. By the time tests are written, the divergence has already happened in production.

Manually translating production captures into specifications is expensive and arbitrary — different humans would produce different specs from the same captures. Skipping the translation altogether leaves the captures as evidence but not as contracts; they don't survive as a specification once the legacy is gone. Asking an LLM to generate specifications from raw captures produces inconsistent results because the captures themselves don't carry intent.

The captured behaviour does carry intent implicitly — recurring traffic shapes, repeated transformations, consistent pre-conditions. The intent is recoverable if the captures are processed with discipline.

Pattern

Deploy a verification framework alongside the modernized C# in production. Capture legacy behaviour on real traffic. Compare it to the modernized C#'s behaviour on the same inputs. Record divergences as evidence. Replay captured scenarios in dev mode to refine the modernized code.

Distinguish three kinds of drift: *semantic drift* (the modernized C# produces a genuinely different result), *architectural artifact* (the new architecture handles something differently than the legacy did, but the result is equivalent in business terms), and *temporal artifact* (the test fixture in dev mode didn't capture a time-dependent input). Each kind requires different action. Semantic drift is a defect — fix it. Architectural artifact is a recorded difference — document it as intentional and update the comparison rules. Temporal artifact is a corpus gap — extract the new input pattern into the dev-mode fixture library so future runs catch it.

The drift typology is not exhaustive. Other categories appear: *data drift* (the input distribution shifted between dev and production), *clock drift* (timing-dependent behaviour the test fixture didn't reproduce), *infrastructure drift* (failure handling that differs between mainframe and cloud stack). The three named above are the most common; the framework should be extensible to others as engagement experience teaches them.

In Rosetta, this framework is called Witness. Witness is the production-mode counterpart to Twin Verification. Specialist observer agents deploy alongside the modernized C# as part of Witness, watching, surfacing anomalies, capturing scenarios. The capture-replay lineage extends from a project I built in 2016 (pmilet/playback, an open-source HTTP capture-replay middleware), now reframed for the agentic era.

Captures as future specifications

The captured behaviour, once accumulated, is evidence that the modernized system can build on after the legacy is decommissioned. The accumulation can be processed into executable behavioural specifications — Given-When-Then statements grounded in what the system actually did rather than in what someone wrote in a planning meeting.

The process is not automated end-to-end. Captures are pattern-matched to surface recurring traffic shapes across the corpus. The preconditions that lead to each shape are extracted. The outcomes that follow are extracted. Candidate specifications are generated and surfaced to a human reviewer through the control plane (Pattern 19) for validation against domain understanding — the captures may include noise, errors, or behaviour that shouldn't survive into the modernized system.

Validated specifications become executable: they form a regression suite the modernized system runs against. They are traceable: each specification points back to the captures that grounded it, which trace back through source provenance (architectural interlude) to the legacy code that produced the observed behaviour. They are reviewable: humans can read them, validate them against domain understanding, refine them where the captures were ambiguous.

The regression suite grows from what the system actually does. Tacit SME knowledge — the kind that lives in domain experts' heads and is never formally written down — becomes externalised as a byproduct of verification. The modernization produces a durable artifact that survives the legacy decommissioning: a behavioural specification grounded in production reality.

This extension is *designed*, not *prototype-validated*. The verification framework itself is in construction; the capture-to-specification machinery is the natural follow-on but has not been built. The principle (specifications can grow from production rather than precede it) is consistent with the prototype's broader posture (the legacy is the spec). What remains to be tested is whether the operational machinery to

make this efficient can be built, and whether the resulting specifications are useful enough to justify the engineering cost.

Consequences

Production becomes a source of evidence rather than a source of incidents. The modernization continues learning past the cutover. The drift typology gives the operations team a vocabulary for triaging divergences — not every difference between legacy and modernized output is a bug, and the typology makes the right response visible.

Captures accumulate into a corpus that has value beyond immediate divergence detection. When the legacy is decommissioned, the modernization is not left without a reference point — the captured behaviour, processed into specifications, is the modernization's testimony about what the legacy did. Future development against the modernized system has executable contracts to validate against. Future modernizations of adjacent systems have a precedent to follow.

The cost is operational complexity. Witness must be deployed in production, which requires the platform engineering to support it. The drift typology must be operationalised — distinguishing semantic from architectural from temporal drift requires discipline that the verification framework can't fully automate. Some divergences will always require human judgement.

The specification-generation path adds further engineering cost. Pattern-matching at scale requires technique — naive grouping produces too many specifications or too few, and the threshold isn't obvious. The quality of the resulting specification depends on the quality of the capture corpus, which depends on Witness running long enough across enough traffic shapes. Teams that want the specification artifact must budget for both the capture infrastructure and the processing pipeline.

Whether Witness works at scale is the prototype's most uncertain bet. The principle (production reveals what dev mode hides, and that revelation should feed back into the modernization) is sound; whether the operational machinery to capture it efficiently can be built is what's being tested. The follow-on (captures become specifications) depends on Witness working first; that follow-on is even less validated.

Related patterns

Pattern 2 (*The Legacy as Oracle*) is the foundational principle, now extended into production. Pattern 13 (*Twin Verification*) is the dev-mode counterpart and the receiver of replayed captures. Pattern 19 (*The Control Plane*) is where humans review captured divergences and approve action, and where candidate specifications surface for validation. Pattern 21 (*Rollout and Cutover at Bounded Context Granularity*) is the operational frame this pattern serves during dual-run — Witness watches for divergences as bounded contexts cut over progressively. Pattern 22 (*Dual-Run Coexistence*) is where Witness sits in the dual-run architecture, monitoring across the bridge period.

Part IV: Governance

The patterns in this group address how the modernization stays coherent across the agentic system. They presuppose verification (Part III) and generation (Part II) — without a working pipeline, governance has nothing to govern.

Pattern 15: Bounded MCP Servers

Status: prototype-validated.

Context

An agentic system for software engineering, composed of multiple capabilities that the agents need to invoke: graph queries, legacy oracle invocation, scaffold generation, verification execution, telemetry recording, hypothesis testing. The capabilities have distinct concerns and could be implemented independently, but the agents need coordinated access to all of them.

The Model Context Protocol (MCP) — introduced by Anthropic in 2024 — provides a standardised way for AI agents to discover and invoke capabilities through declared tool interfaces. An MCP server exposes a set of tools with typed inputs and outputs; agents discover those tools, decide which to call, and receive results back through the protocol. MCP is the substrate this pattern builds on; what's articulated here is how to decompose capabilities across MCP servers, not the MCP protocol itself.

Problem

The default approach is to give a single agent broad tool access — direct file system access, direct database access, direct shell commands. This works for small systems and fails at scale.

Cross-cutting access produces unauditable behaviour: when something goes wrong, there's no way to localise the failure because the agent could have touched anything. Implementations become coupled because any agent can reach into any subsystem; replacing one capability requires understanding how every agent interacts with it. Security boundaries collapse: granting an agent access to "the system" means granting it access to everything. Cost accounting becomes impossible: telemetry can't attribute work to specific capabilities because the work is scattered.

The deeper issue is that the agentic system itself is a domain that hasn't been modelled. Without bounded contexts inside the agentic platform, the platform is a monolith with broad surface area — the same

architectural smell DDD has been correcting in business systems for two decades.

Forces

The agents need access to multiple capabilities. The capabilities need to evolve independently — a new graph analysis pass should not require updating every agent that uses graph data. The system needs to be debuggable and auditable. Security and access control need to be enforceable per capability. Cost and performance need to be attributable.

These pressures pull in different directions. Tight coupling between agents and capabilities makes the agents simpler but the system fragile; loose coupling makes the system robust but requires explicit contracts everywhere. The contracts themselves have to be expressive enough to support agent needs without exposing internals — a hard balance.

Pattern

Apply bounded contexts to the agentic system itself. Expose each capability through a specialised MCP server with a single, coherent responsibility. The agents access capabilities through server interfaces; they don't reach into implementations directly.

Each MCP server has its own ubiquitous language, its own data model, its own consistency boundary. The Discovery server speaks the language of graphs, ontologies, slices, communities. The Twin server speaks the language of scaffolds, paragraphs, candidates, verification fixtures. The vocabularies don't bleed across boundaries — when the Twin server needs graph information, it queries Discovery's tools and receives data in Discovery's terms, which the Twin server then translates into its own vocabulary internally.

In Rosetta, four MCP servers structure the agentic platform: a Discovery server owns the graph layer (graph queries, ontology lookups, slice discovery); a Legacy server owns the Legacy Twin (oracle invocation, behavioural verification, divergence diagnostics); a Twin server owns the C# generation pipeline (scaffold rendering, paragraph translation, IR construction); a Witness server owns the verification lifecycle (hypothesis generation, production-mode verification, certification). Each server's implementation is private; agents see only the tools the server exposes.

Cross-server interaction follows the same disciplines DDD applies to bounded contexts. When one capability needs another's data, it goes through the public interface, not through shared databases or back-channel access. The integration patterns are explicit: published events (when one server's work emits state changes others consume), conformist consumers (when one server simply accepts another's output as canonical), anti-corruption layers (when one server needs to translate another's vocabulary into its own internal model). The Legacy server is itself an instance of what Nick Tune has called *Expose Legacy Asset* (see glossary): the legacy is wrapped in an explicit interface that modernized subsystems consume, rather than being accessed directly through database connections or in-process coupling.

Consequences

Implementations evolve independently — a new graph analysis pass changes the Discovery server without disturbing Twin or Witness. Audit trails are clean because every agent action goes through a server boundary and is recorded with structured input/output. Failures localise: when something goes wrong,

the audit shows which server saw which inputs and produced which outputs, and the investigation starts at the relevant boundary instead of tracing through unbounded agent activity.

Security becomes per-capability: an agent that needs graph queries gets credentials to the Discovery server only, not blanket access. Cost attribution becomes possible: each server reports its own resource usage, and the cost of a modernization can be decomposed by capability. Performance debugging becomes tractable: latency profiles are localised to specific servers, and bottlenecks are identifiable.

The cost is contract design. Each server's interface must be expressive enough to support the agents' needs without exposing internals. Cross-cutting concerns — orchestration, policy enforcement, end-to-end telemetry — live above the servers (Pattern 16), not within them. The servers themselves can't bypass each other's contracts, even when bypassing would be convenient.

The agentic system, structurally, is a modular monolith of bounded servers — same DDD discipline applied to the agentic platform itself that this catalog applies to the modernized business system. The recursion is not coincidental: bounded contexts are how complex systems remain comprehensible, whether the system is a bank's core ledger or an AI platform for legacy modernization.

My guess is that this pattern will become more common as the field matures. The current default of giving a single agent broad tool access has clear limits at scale, and the discipline of bounded MCP servers is the natural correction.

Related patterns

Pattern 16 (*Durable Orchestration Above Bounded Capabilities*) is what coordinates across the bounded servers. Pattern 18 (*The Harness as Self-Observing State Machine*) governs *what* agents do; this pattern governs *how* they do it — which capabilities they can reach and through which interfaces. The reasoning telemetry layer of Pattern 18 is what each server emits about its own decisions, which makes the audit trail end-to-end. Pattern 19 (*The Control Plane*) is the human-experience surface above the MCP layer, surfacing what each server has done and what's pending. Pattern 20 (*Team Topology and Bounded Context Alignment*) determines which team has authority over which server's evolution.

Pattern 16: Durable Orchestration Above Bounded Capabilities

Status: prototype-validated.

Context

An agentic system structured around bounded capabilities (Pattern 15), where each capability has a coherent responsibility and a defined interface. The agents need to accomplish goals that span capabilities — discovering bounded contexts in the graph, then using those contexts to scaffold C#, then verifying the scaffolds against the Legacy Twin, then refining based on the verdict. No single capability owns this end-to-end work.

The end-to-end work runs across weeks or months. Agents work asynchronously, humans intervene at gates, decisions get revisited as evidence accumulates. The platform must keep state across this entire

span — across infrastructure restarts, across human absences, across deployment changes, across model upgrades.

The cross-cutting work has both a logical shape (what gets coordinated, in what order, with what authority) and a physical shape (how that coordination survives the realities of long-running operation). Both shapes are first-class concerns and they have to be designed together.

Problem

Without an explicit orchestration layer, the agents themselves end up coordinating across capabilities. This produces brittle behaviour: every agent has to know which capabilities exist, which order to invoke them in, what to do when one capability's output needs to be reshaped before another can consume it. Cross-capability coordination becomes scattered through the agent population, which makes the overall flow opaque and hard to govern.

The instinct to fold orchestration into one of the bounded capabilities — making the Discovery server, say, responsible for invoking Twin and Witness — violates the bounded-context principle. The capability that orchestrates is no longer a bounded context; it's a god object dressed up as a server. Its vocabulary expands to include other capabilities' vocabularies; its state expands to track other capabilities' states; its responsibility expands to include other capabilities' decisions. The bounded discipline that Pattern 15 establishes collapses if any single server absorbs orchestration.

There is a choice between orchestration and choreography that the catalog needs to make. Choreography (capabilities reacting to events from each other without a central coordinator) is appealing because it preserves bounded discipline, but it makes the overall flow harder to reason about and harder to gate with human approval. Orchestration (a coordinating agent that drives the workflow) is more legible and more governable, at the cost of introducing a coordinator that must be carefully constrained.

A separate but inseparable problem: agentic workflows that exist only in process memory don't survive the realities of long-running work. A platform restart loses the workflow state. A model upgrade mid-flight loses context. A human reviewer who steps away for a week returns to find the workflow expired. The traditional approach — checkpoint everything to a database periodically and reconstruct on resume — is fragile because the workflow has rich state (open agent conversations, in-flight tool calls, accumulated reasoning) that doesn't reduce cleanly to database rows.

The problem compounds with replay. When an agent's tool call fails partway through, the workflow needs to recover without redoing the work that already succeeded. Without replay-aware infrastructure, partial failures cascade — every tool call that succeeded before the failure has to be redone, every decision that was reached has to be re-derived.

The two problems are facets of one larger problem: cross-cutting coordination that survives the operational realities of multi-month agentic work.

Forces

Coordination is inherently cross-cutting. It can't be eliminated. But where it lives matters: scattered across agents, it's chaotic; folded into a bounded server, it corrupts that server's coherence. There needs to be a place for cross-cutting work that doesn't compromise either the agents or the bounded servers.

The orchestrator must coordinate without bypassing. If it accesses capabilities directly — through database connections, through internal APIs — the bounded discipline of Pattern 15 collapses. The orchestrator's power must come from broader scope, not from privileged access. It sees the whole workflow but it interacts with capabilities through the same MCP interfaces other agents use.

The orchestrator must also accommodate human-in-the-loop gates. Not every cross-cutting decision is automatable — some require human judgment (validating a hypothesis, approving a scaffold, accepting a divergence as intentional). The orchestrator routes work to humans through the control plane (Pattern 19) and waits for the response, the same way it routes work to capabilities and waits for their response.

The workflow's logical model wants to be simple: states, transitions, gates, with agents operating inside steps. The workflow's physical reality is complex: long durations, partial failures, infrastructure restarts, version skew between models and tools, human intervention timing that the system can't predict. The simple logical model needs to survive the complex physical reality, which requires infrastructure designed for the complexity, not retrofitted to it.

Pattern

Place an orchestrating agent above the bounded capabilities, with explicit responsibility for cross-cutting decisions. The orchestrator coordinates without bypassing — it accesses bounded capabilities only through the same interfaces other agents use, but it has the broader view that lets it make decisions about sequencing, escalation, and revision.

Run the orchestrator on infrastructure designed for durability: durable workflow infrastructure that persists state automatically, supports replay semantics for failed steps, recovers cleanly from restarts, and tolerates the time scales the work actually requires. The infrastructure handles state persistence, replay, and recovery; the orchestration code expresses the logical model and stays close to the harness definition.

The harness (Pattern 18) is the workflow's *definition* — what states exist, what transitions are valid, what gates apply. The durable workflow infrastructure is the *execution substrate* that runs the harness across time. The orchestrating agent is the *coordinator* that drives transitions through the harness on top of the durable substrate. The three concerns are separable: the same harness can run on different durable infrastructures depending on the engagement scale and operational constraints; the orchestrator's policies can evolve without changing the harness definition.

In Rosetta, this orchestrator is called Cortex. Cortex makes decisions like: when to escalate to a human, when to switch focus from one bounded context to another, when to revisit prior decisions in light of new evidence, when one capability's output needs reshaping before another can consume it. Cortex runs a retrospective sub-agent that learns across modernization sessions, accumulating patterns about which sequences work well and which often produce escalation, feeding those patterns back into future workflow decisions.

Cortex maintains the workflow state — what slice is currently being worked on, what stage it's at, what gates remain, what humans are waiting for. The state lives in durable workflow infrastructure (Azure Durable Functions in the current Rosetta implementation), surviving process restarts and human absences. Other agents query Cortex about workflow state; Cortex queries them about capability state.

GitHub-native primitives (branch protection, required status checks, GitHub Actions, issue templates) materialize a subset of gates as enforceable mechanisms in the development workflow. These are

useful where they apply — code review gates, deployment gates, scaffold validation gates. They're not the durable execution substrate; they're a complementary layer for gates that happen to align with version-control workflow. The orchestrator coordinates *across* the GitHub-native gates and the durable infrastructure rather than choosing one over the other.

The orchestrator is itself an agent, not a service. Its behaviour is bounded by the harness (Pattern 18), observed through the control plane (Pattern 19), and emits reasoning telemetry like any other agent. It coordinates the others, but it doesn't escape the governance that applies to the whole system. The orchestrator's authority comes from its scope, not from privileged status.

Consequences

Cross-cutting decisions have a place. Agents working in bounded capabilities focus on their own work; the orchestrator handles the coordination. The system as a whole becomes legible — there's a specific place to look for "why did the work move from this bounded context to that one," and the answer is in the orchestrator's decision log.

Workflow coherence becomes a queryable property. At any moment, the orchestrator can answer: which slices are in flight, which have stalled, which are blocked on humans, which are blocked on capabilities. This visibility makes the control plane's role tractable — the control plane doesn't have to reconstruct the workflow from scattered evidence; it queries the orchestrator.

Workflows survive infrastructure realities. Restarts don't lose state. Long human absences don't expire workflows. Tool-call failures don't cascade. Model upgrades during a workflow don't corrupt prior decisions. The platform becomes operationally credible at the time scales real engagements require.

The audit trail becomes durable. Every state transition is recorded by the infrastructure, available for retrospective analysis. Compliance reviewers in regulated industries have explicit records of the workflow's progression — not just the final state, but every intermediate decision and the conditions under which it was reached.

The cost is twofold. The orchestrator's complexity is the first. It has to know enough about every bounded capability to coordinate them, but it must not duplicate their internals. This is a delicate balance — orchestrators that grow too smart start to absorb logic that belongs in the capabilities; orchestrators that stay too simple end up unable to coordinate effectively. Maintaining the right level of abstraction in the orchestrator is ongoing work, often informed by what the harness's self-observation (Pattern 18) surfaces about which decisions the orchestrator gets right and which it doesn't.

The durable infrastructure dependency is the second cost. Running on durable workflow infrastructure adds operational complexity: it has to be hosted, maintained, monitored, scaled. The workflow code has constraints — certain operations don't replay cleanly, certain patterns don't survive serialization — that pure in-memory workflows don't have. Developers must learn the infrastructure's semantics and design within them.

Whether the durable infrastructure is hosted (a cloud platform's managed service) or self-hosted is a separate operational decision that depends on the engagement's compliance posture, scale, and integration requirements. The pattern is agnostic to that choice; what matters is that the substrate is designed for durability, not retrofitted for it.

There is a choreography alternative this pattern explicitly rejects for modernization contexts.

Choreography — bounded capabilities reacting to each other's events without a coordinator — works well when the workflow shape is stable and the participants are autonomous. Modernization workflows are exploratory, human-gated, and reshape as engagements teach what works. Orchestration's central coordination is the right tradeoff for that context. The durable substrate makes the orchestration tractable; without it, central coordination would be too brittle to operate at the time scales the work requires.

Related patterns

Pattern 15 (*Bounded MCP Servers*) is what the orchestrator coordinates across. Pattern 18 (*The Harness as Self-Observing State Machine*) is the workflow definition that runs on the durable substrate; it governs the orchestrator's behaviour the same way it governs other agents — and watches its decisions for refinement opportunities. Pattern 19 (*The Control Plane*) is where humans observe and intervene in the orchestrator's decisions. Pattern 20 (*Team Topology and Bounded Context Alignment*) is what the orchestrator must respect when routing work to humans: routing to "the team that owns this context" requires the orchestrator to know which team owns which context.

Pattern 17: Heuristics as Explicit Artifacts

Status: in construction.

Context

An agentic system where specialised agents make decisions across many contexts: slice-discovery agents propose where slice boundaries belong; tier-classification agents decide which scaffold shape to apply to which bounded context; ontology-recovery agents score candidate domain terms; anti-corruption layer detectors identify integration points needing translation; similarity agents weigh whether two paragraphs implement the same intent. Each of these decisions involves applying heuristics — rules about what counts as evidence for what.

The heuristics are not optional. Slice boundaries cannot be inferred without rules about what makes a structural cluster cohesive enough to be a slice. Tier classification cannot proceed without thresholds on coupling, change frequency, and strategic value. Ontology recovery cannot weight candidate terms without scoring rules. The question is not whether the heuristics exist — they do, in every agentic system that makes such decisions — but where they live and how they evolve.

Problem

The dominant practice in agentic systems is to bake heuristics into agent prompts. The slice-discovery agent's prompt explains what to look for: "paragraphs that share data and predicates are likely to belong to the same slice; XCTL between paragraphs typically marks a bounded-context transition; …" The tier-classification agent's prompt encodes the thresholds: "if cyclomatic complexity is over X and coupling is over Y, classify as tier 3…"

This works briefly and fails as the system scales. The failures are characteristic:

- **Implicit knowledge becomes invisible.** Heuristics buried in prompts are not findable. When the team needs to know "what rule did we apply for slice boundary detection?" they must read prompts — sometimes scattered across dozens of agent definitions — and reconstruct the rule from natural language. The reconstruction is unreliable.
- **Refinement drifts through prompt-engineering.** When a heuristic needs adjustment, the change is made by editing a prompt. The change has no version history that distinguishes it from other prompt edits. It cannot be A/B tested. Its effect is observable only through the downstream behaviour of the agent.
- **Audit trail breaks.** When an agent makes a decision and a reviewer asks "why?", the agent cites its reasoning in natural language, but the heuristic it applied is not a named artifact. The reasoning is post-hoc rationalisation of prompt-embedded knowledge, not citation of a queryable rule.
- **Composition fails.** When two agents must apply the same heuristic — slice-discovery and slice-validation, for instance — the heuristic must be duplicated across prompts. Drift between the copies is inevitable.

The deeper failure is treating heuristics as implicit context rather than as first-class artifacts. The compiler principle (Pattern 6) articulates that deterministic decisions belong in deterministic infrastructure; heuristics are deterministic decisions, but they have been hiding in probabilistic prompt content.

Forces

Heuristics need to be observable. When an agent applies a heuristic, the audit trail must record which heuristic was applied and what evidence supported it. Heuristics need to be versionable. When a heuristic is refined, the refinement must be reviewable, reversible, and traceable to its rationale. Heuristics need to be composable. Multiple agents may apply the same heuristic; refining once must update everywhere.

But heuristics also need to be contextually applied. A heuristic that holds for tier-3 bounded contexts may not apply in tier-0. A heuristic for slice discovery in CICS COBOL may not apply for batch JCL. The catalog of heuristics must support specialisation by context.

Pattern

Treat heuristics as first-class queryable artifacts, not as implicit knowledge in agent prompts. Build a heuristic catalog where each heuristic is a named, structured entry:

- **Heuristic identifier** — a stable name (`xctl-bounded-context-transition`, `tier-3-coupling-threshold`, `ontology-candidate-vocabulary-overlap`).
- **Conditions of application** — what context the heuristic applies to (tier, language, bounded context, slice characteristic).
- **Evidence weights** — for each piece of evidence the heuristic considers, the contribution to the decision (XCTL between paragraphs in different bounded contexts → 0.8 confidence of slice transition).
- **Version and validation status** — when the heuristic was introduced, when it was last refined, what evidence validated it.
- **Observability hooks** — what telemetry the heuristic emits when applied.

Specialised agents access the catalog through queries: a slice-discovery agent encountering an XCTL queries `heuristics:by-cue("XCTL", context:current_tier)` and receives the relevant heuristics

with their weights. The agent applies the heuristics, emits a reasoning record (Pattern 18) citing which heuristics were applied with which evidence, and the audit trail is intact.

The catalog itself is built with the same compiler discipline as the rest of the modernization pipeline. Heuristic definitions live in structured files (YAML, JSON, or domain-specific schema); validators enforce required fields; versioning is git-native; refinements go through review like any other architectural change. The catalog is queryable through an MCP server (Pattern 15): agents discover heuristics through tool calls, not through prompt content.

Refinement happens through structured evolution. When the harness's self-observation (Pattern 18) detects that a heuristic is producing high error rates in a specific context, it surfaces the heuristic for review. An architect (or a retrospective agent operating under human authorisation) refines the heuristic — adjusts weights, narrows conditions of application, adds new evidence types — and the refinement is recorded. The next time the heuristic is applied, agents see the refined version.

The heuristic catalog is itself a substrate that the compiler architecture treats as first-class, alongside the graph (Pattern 3), the IR (Pattern 7), and the domain ontology (Pattern 4). Documentation emitters (Pattern 9) can render the catalog as documentation: which heuristics exist, what they decide, what their evidence weights are, what their refinement history is. The catalog becomes legible to humans without requiring them to read agent prompts.

Catalog entries themselves can be partially derived from operational history. Uberto Barbini has reported experimenting with generating rule files for AI assistants from git history and pull-request comments — the team's own corrective behaviour over time becomes input to the rules the next agent applies. The same idea operates at platform scale here: when reviewers reject scaffolds or flag candidates with consistent rationale, that rationale is signal about which heuristics need refinement or which new heuristics need authoring. The retrospective agent (Pattern 18) can consume PR conversation telemetry as one of its evidence streams, surfacing candidate catalog updates that an architect then validates. The lesson does not stay in the head of the reviewer who wrote the comment; it propagates into the catalog.

The layered structure of the catalog

The heuristic catalog is not flat. It has four layers organised by stability — by how rarely each layer changes, and by which engagements have authority to change it. Each layer constrains the layer below; each layer is fed by promotion from the layer below.

Universal heuristics are substrate-agnostic and target-agnostic. They encode the antipattern categories the catalog is built against, the meta-rules about how heuristics are structured, the cross-cutting evidence weights that hold regardless of legacy stack or modernization target. Universal heuristics change rarely; changes are reviewed centrally and propagate across every engagement that consumes the catalog.

Substrate-specific heuristics are calibrated to a particular legacy stack. The CICS COBOL substrate has its own L2 catalog: pattern detectors for conversational shapes (LINK/XCTL chains, COMMAREA conversations, pseudo-conversational TRANSID cycles), for transactional shapes (SYNCPOINT scope, deferred-commit wizards, latent sagas), for routing shapes, for domain-shape recovery from data layer evidence. A PL/I substrate or an RPG substrate would have its own catalog, structurally parallel but populated with different patterns. Substrate-specific heuristics are sharpened per engagement and curated by substrate experts; they change at the cadence of accumulating substrate experience.

Target-specific heuristics are calibrated to a particular target architecture. The Wolverine + C# target has its own rules: when a `LatentSaga` pattern in L2 projects to a `Saga` in L3, when it projects to a `StatefulFlow + SingleCommand`, when it projects to choreographed events. A Java + Axon target would have different projection rules consuming the same substrate-specific L2 patterns. Target-specific heuristics evolve with the target ecosystem — when Wolverine introduces a new construct, the target layer extends; when the target stack itself changes, the target layer is replaced wholesale.

Engagement-specific heuristics are the volatile layer. Each engagement accumulates overrides: this specific customer's domain has vocabulary the canonical ontology doesn't yet cover; this specific codebase has an idiom the substrate catalog hasn't seen; this specific architect made a non-canonical projection decision that needs to be recorded with its justification. Engagement-specific heuristics live for the duration of one engagement and serve one customer's modernization.

Promotion gates connect the layers. When an engagement-specific override appears repeatedly across engagements with consistent justification, it becomes a candidate for promotion to substrate-specific or target-specific status. When a substrate-specific pattern proves substrate-agnostic — when the same shape recurs in PL/I and RPG with structurally equivalent dynamics — it becomes a candidate for promotion to universal status. The promotion is not automatic; it is a review event with explicit governance, the same way changes to the canonical layers themselves are reviewed.

The promotion mechanism is what lets the catalog learn. Without it, engagement-specific lessons stay engagement-specific and the modernization team starts each new engagement from the same baseline. With it, the catalog absorbs accumulated experience: the team's expertise about how CICS conversations recover, how Wolverine targets render, how specific customer domains pattern, gets crystallised into the catalog as durable artifacts that compound across engagements.

The layering also clarifies who has authority. Engagement-specific changes are made by the engagement team and require only engagement-level approval. Substrate-specific changes require substrate-expert curation. Target-specific changes require target-architecture review. Universal changes require central review with cross-engagement evidence. Each layer's stability is proportional to the breadth of its consumers and the cost of getting it wrong.

This layered structure is what distinguishes the heuristic catalog from a flat configuration file. Configuration files don't learn — they're updated. The catalog learns because its promotion gates make learning visible, reviewable, and reversible. When a promoted heuristic turns out to be substrate-specific after all (it appeared universal because the first three substrates that triggered it happened to share an idiom), demotion is possible — the heuristic moves back to substrate-specific and stops constraining engagements that should never have been bound by it.

Consequences

Agent reasoning becomes auditable at the rule level. When a slice is proposed, the trace cites the specific heuristics that drove the proposal; when a reviewer challenges the proposal, the conversation can address whether the heuristic is correct, not whether the agent "got it right." The unit of disagreement is the rule, not the outcome.

Refinement compounds across the engagement. Lessons learned from one bounded context — that this specific cue is more informative than the catalog initially weighted — become catalog updates that benefit

every subsequent bounded context. Without the catalog, lessons stay in individual prompts and don't propagate.

Composition works. Multiple agents apply the same heuristics by querying the same catalog; refinement updates all consumers simultaneously. There is no prompt-copy drift because there are no prompt copies.

The cost is the discipline of structured heuristic authoring. Heuristics that would have been one sentence in a prompt now require schema-conformant catalog entries with named fields, validated conditions, and observability hooks. The authoring cost is higher per heuristic; the system-wide cost — across many agents and many engagements — is lower.

There is also a discovery cost. When a new kind of decision arises (a new agentic capability needs to make a new kind of judgment), the team must decide what heuristics apply and add them to the catalog. This work is more visible and slower than adding sentences to prompts. The visibility is the point.

Adjacent disciplines have solved related problems: rule engines (Drools, Camunda) in business systems articulate decision logic as data; OPA / Rego articulates policy as code with versioning and audit; ML feature stores treat feature definitions as queryable artifacts. These are precedents the pattern stands on rather than departures from. What this catalog contributes is the application of the same discipline specifically to *agentic decision-making in modernization* — the rules agents apply when classifying paragraphs, proposing slice boundaries, scoring ontology candidates, detecting anti-corruption layer points. The territory is new; the underlying principle (decision rules as queryable, versionable artifacts) is not.

Related patterns

Pattern 6 (*The Compiler Principle*) is the broader principle this pattern operationalises — deterministic decisions live in deterministic infrastructure. Pattern 5 (*Vertical Slice Discovery*) is one of the principal consumers — slice-boundary heuristics live in the catalog. Pattern 8 (*Tier-Aware Scaffolding*) consumes tier-classification heuristics. Pattern 14 (*Hypothesis-Driven Verification*) consumes divergence-categorisation heuristics. Pattern 15 (*Bounded MCP Servers*) hosts the heuristic catalog as a queryable capability. Pattern 18 (*The Harness as Self-Observing State Machine*) detects when heuristics need refinement based on operational evidence, and its reasoning telemetry layer cites which heuristics each agent applied.

Pattern 18: The Harness as Self-Observing State Machine

Status: prototype-validated. (Harness and self-observation: validated. Retrospective agent and reasoning telemetry: in construction.)

Context

An agentic system performing modernization work over weeks or months. Agents make many decisions: which slice to work on next, which heuristic to apply, when to escalate, when to retry, when to abandon a candidate. The system must remain governable across the entire span — humans must be able to intervene, decisions must be auditable, the work must converge rather than drift.

The system also has to *learn*. The same kinds of agentic decisions recur across many slices, many bounded contexts, many engagements. If every decision is independent, every engagement starts from zero. If the system observes its own behaviour and feeds those observations back into refinement, the work compounds.

The decisions themselves have to be examinable. When a human reviewer asks "why did the agent choose this translation rather than that one," the answer should not be lost. Without traceable reasoning, debugging is forensic — reconstructing intent from outcomes — rather than direct.

Problem

Three failure modes recur in agentic platforms that operate without the discipline this pattern names.

The first failure is governance through prompts. The agents are told what to do in natural language: "respect the scaffold boundaries; do not modify the architectural skeleton; escalate to human when uncertain." Prompts are advisory; the agents treat them as suggestions when convenient, and the violations are detected only post-hoc through review.

The second failure is opacity about behaviour. The platform produces work, but the *patterns* in how that work was produced — which gates the agents trip most often, which transitions stall, which escalations recur — are invisible. The platform cannot improve because it cannot see itself. Engineering teams optimise based on intuition rather than evidence.

The third failure is opacity about reasoning. Even when the *what* of agentic decisions is recorded, the *why* is lost. The agent chose a particular translation; the trace doesn't say why. The agent rejected an alternative; the trace doesn't say what alternative or why. When the same decision is revisited weeks later, the rationale must be reconstructed rather than retrieved.

All three failures share a root: agentic behaviour that should be *constitutional, observable, and explainable* is instead *advisory, opaque, and silent*.

Forces

Agentic systems must be both flexible (so agents can do useful work the architects didn't anticipate in detail) and constrained (so agents don't violate architectural invariants the modernization depends on). The flexibility lives in *what* the agents do within a step; the constraint lives in *which steps* are valid and *which conditions* must hold at every transition.

The platform must operate at long time scales: weeks for a single slice through its full lifecycle, months for a bounded context through full modernization, years for a substantial mainframe estate. Across that span, conditions change — model versions evolve, tool implementations shift, organisational priorities reshape. The platform's behaviour must remain coherent regardless.

Self-observation is itself an architectural decision. Observing the platform's behaviour costs storage, query infrastructure, retention discipline. The benefit accrues over time — refinements based on operational evidence compound over engagements — but the upfront cost is real and the value is deferred.

Observation also creates pressure. Once agents are evaluated against specific measures — gate passage rates, heuristic application frequency, reasoning trace quality — Goodhart's law applies immediately:

when a measure becomes a target, it ceases to be a good measure. Reasoning traces can be performed; gates can be pattern-matched without genuine engagement; metrics can be optimised without the underlying behaviour improving. The self-observation must anticipate this pressure rather than ignore it.

Pattern

The pattern has three parts that compose into one architectural commitment: a constitutional harness, a self-observation layer over that harness, and a reasoning telemetry layer over each decision.

Part 1: The harness as typed state machine with hook-based guards

Build the agentic workflow as a typed state machine in code, not as natural-language instructions. States are explicit (`SliceProposed`, `ScaffoldRendered`, `TranslationCandidate`, `TwinPassed`, `TwinFailed`, `WitnessDeployed`, `Certified`, `DarkLaunched`, `Cutover`). Transitions are explicit (which states can follow which). Each transition has a guard that runs as code — not as a prompt, not as an advisory note — and the transition only fires if the guard returns true.

The guards encode constitutional invariants. The agent cannot modify the scaffold's immutable fields (defined in `scaffold-meta.json`, per-project, validated at every PreToolUse hook). The agent cannot promote a translation that hasn't passed Twin Verification (Pattern 13). The agent cannot escalate to deployment without explicit human approval at the certification gate. The constitution is enforced as code; the agent's instructions don't have to be trusted because the harness doesn't allow violations.

PreToolUse and PostToolUse hooks fire around every tool invocation. PreToolUse hooks check whether the proposed action is permitted in the current state; PostToolUse hooks verify that the action's effects respect the constitutional contract. Both hooks have full access to the workflow's state, the agent's history, the scaffold metadata, and the heuristic catalog (Pattern 17). They are not lightweight check-and-pass mechanisms; they are where constitutional enforcement actually lives.

Human governance gates are first-class states that agents *cannot transition out of without explicit human action*. The state machine encodes this as agent-inaccessible transitions — only specific external authority (a human approving through the control plane, Pattern 19) can move the workflow forward from these states. The agent can prepare the work for review, can articulate why the work is ready, can request the gate's opening; but it cannot open the gate itself. Issue templates and labels function as human workflow triggers: when an agent's work is ready for review, an issue is created from a template that specifies what the human is being asked to decide.

GitHub-native primitives complement code-level enforcement. Branch protection ensures generated code cannot be merged without passing required status checks. GitHub Actions execute deterministic validations (does the project compile, do the tests pass, does the scaffold's hash match the expected metadata). Required reviewers attach to specific labels: certification PRs require an architect, dark launch PRs require a designated approver. The GitHub layer is not the only enforcement, but it is the layer most engineers already understand, which makes the harness legible to developers without requiring them to learn new tooling.

Part 2: Self-observation as architectural property

Once the harness exists, observe it. Every state transition emits a structured event: which state was entered, which guard was evaluated, which heuristic was applied, what evidence the agent cited, how long the

agent spent in the previous state. Every gate evaluation emits a record: which gate, what condition, what outcome, what data the gate consulted. Every escalation emits a record: which agent escalated, what condition triggered it, what authority resolved it.

The records accumulate as queryable telemetry — not as logs intended for human reading, but as structured events intended for analytical query. The harness's own behaviour becomes inspectable through the same kinds of queries the modernization uses for the legacy code: aggregate, filter, group, correlate, surface patterns.

What the queries surface drives refinement. If a particular gate is failing 40% of the time and most failures are recovered through a specific kind of retry, the gate's logic should be adjusted: either tightened (the agent should not have proposed something that fails this often) or relaxed (the gate is over-strict for current evidence). If a heuristic from the catalog (Pattern 17) is being cited but the resulting decisions are reversed 30% of the time, the heuristic needs refinement. If certain state transitions are stalling repeatedly with humans not responding, the routing to humans needs investigation — perhaps the gate is being routed to the wrong team (Pattern 20), or the artifacts the human needs to decide are not surfacing correctly through the control plane (Pattern 19).

A retrospective agent runs over the accumulated telemetry, surfacing candidate refinements: heuristics that need reweighting, gates that need tightening, transitions that need additional guards, agent roles whose escalation patterns suggest they need narrower scope. The retrospective agent does not apply refinements autonomously — it surfaces them to architects through the control plane, with evidence, and architects validate or reject. The refinement loop is human-gated. The substrate of evidence is automated.

Goodhart pressure applies the moment self-observation becomes evaluative. If the harness optimises for gate passage rates, agents will learn to produce candidates that pass gates without doing the underlying work. If it optimises for heuristic application frequency, agents will cite heuristics performatively. If it optimises for reasoning trace richness, agents will produce verbose reasoning traces that don't reflect actual reasoning.

The mitigation is layered:

- **Action-and-outcome is the primary signal.** Reasoning traces and heuristic citations are *supplementary*; what the agent actually produced, and how that production performed against ground truth (Pattern 13's Legacy Twin, Pattern 14's Witness in production), is the authoritative signal. Outcome metrics are harder to game than process metrics.
- **The retrospective agent reads telemetry as evidence, not as performance.** When patterns appear that suggest gaming — convergent reasoning-trace shapes that don't track outcome quality, heuristic citations that don't correlate with the heuristic's actual evidentiary basis — those patterns themselves become surfaceable as concerns to architects.
- **Self-observation is observed by architects, not just by retrospective agents.** The control plane (Pattern 19) surfaces self-observation summaries to architects regularly; humans look at the telemetry directly. The Goodhart pressure on the retrospective agent itself is checked by direct human inspection of the underlying telemetry.

The pattern does not eliminate Goodhart pressure — it cannot. Any measure can be gamed by sufficiently capable agents. The pattern names the pressure and structures the observation to anticipate it.

Part 3: Reasoning telemetry as first-class signal

Each agentic decision emits a structured reasoning record alongside its action. The record cites which substrates the agent consulted (graph nodes, IR elements, ontology terms, semantic index matches), which heuristics from the catalog it applied with what weights, which alternatives it considered and why it rejected them, what evidence supported the chosen path.

The records live in observability infrastructure designed for queryable analytical workloads — the same kind of infrastructure that handles production telemetry generally, not bespoke agent-specific logging. This integration matters: reasoning telemetry becomes part of the broader operational observability story, queryable with the same tools the engineering team uses for performance debugging, security investigation, and incident response.

What this enables: agentic behaviour becomes inspectable at the decision level. When a human reviewer asks "why did the agent choose this translation," the agent's reasoning record is the answer — not reconstructed from prompts and outputs, but retrieved from a queryable telemetry store. Patterns across decisions become surfaceable: which heuristics from the catalog are most frequently applied, where ontology terms are repeatedly cited as ambiguous, where escalations to humans concentrate, where alternative-considered counts spike (suggesting genuine uncertainty rather than mechanical decision).

The interpretability caveat is essential, however. Reasoning telemetry surfaces what the agent *reports* about its decision-making. This is not necessarily what the agent's underlying computation actually did. Research on LLM faithfulness (Lanham et al. on chain-of-thought faithfulness; work from Anthropic and DeepMind on model self-explanation) suggests that LLM-emitted reasoning traces frequently rationalize rather than reflect — the model emits a plausible explanation post-hoc rather than reporting its actual internal state.

Reasoning telemetry is therefore valuable as *supplementary* signal, not as authoritative ground truth. The primary signal remains action and outcome: what the agent did, and how it performed against deterministic ground truth (Twin Verification, Witness in production). Reasoning telemetry adds context for human review, supports pattern detection by the retrospective agent, captures the agent's articulated rationale for audit purposes — but it does not displace action-and-outcome as the authoritative basis for decisions about the agent's behaviour. When reasoning trace and outcome diverge — when an agent confidently articulates an approach that consistently fails verification — outcome wins; the trace becomes another diagnostic input, not a rebuttal.

Consequences

Agentic behaviour becomes governable as architecture rather than as supervision. Architectural invariants survive regardless of which agent runs, which model version is deployed, which prompt is active. The harness is the contract; agents operate within it; humans approve at gates the harness defines.

The platform observes its own behaviour. Engineering decisions about which agents to refine, which heuristics to update, which gates to adjust become evidence-based — the telemetry shows where the system is working and where it is not. Refinement compounds across engagements rather than starting from intuition each time.

Reasoning becomes legible. Audit conversations move from "reconstruct what the agent must have been thinking" to "retrieve what the agent said it was thinking, and compare to what it actually produced." The

discrepancies become first-class signal — when the trace says one thing and the outcome says another, the divergence is itself informative.

Three classes of cost are real and persistent:

- **The state machine and hook discipline must be maintained continuously.** As the workflow evolves, the state machine evolves with it. New states, new transitions, new guards, new hooks. The cost is engineering work that does not produce new features but produces continued governability.
- **Self-observation infrastructure has real cost.** Storage for telemetry, query infrastructure, retention policy, the retrospective agent itself, the control plane surfaces that present self-observation summaries. None is large in isolation; all add up across the lifetime of the modernization platform.
- **Reasoning telemetry pays a per-decision cost.** Every agentic decision emits a structured record. The volume can be significant — large modernizations involve millions of agentic decisions over their lifetime. The retention and query infrastructure must scale to this volume; the integration with broader observability tooling has to be operationally sustainable.

The Goodhart pressure does not go away. The mitigations described above reduce its impact but cannot eliminate it. Architects working with this pattern must remain alert to the pattern of convergent agent behaviour that looks too good to be honest — and must be willing to revise gates, heuristics, and telemetry schemas when gaming is suspected. The pattern is not a one-time architecture; it is an ongoing practice.

The interpretability limit also persists. Reasoning telemetry will always be the agent's report about itself, not direct observation of the agent's computation. Architects working with this pattern must internalise that the telemetry is one source of evidence, not the truth, and must weight it accordingly when making judgments about agentic behaviour.

The pattern's first two parts (the harness as constitutional state machine; the harness as observable substrate) are validated in the Rosetta prototype. The third part (reasoning telemetry as integrated signal) is in active construction — the telemetry schema is designed, the integration with broader observability infrastructure is being built, the retrospective agent's analytical capabilities over reasoning telemetry are being implemented. The pattern is reported as *prototype-validated* with these components called out as *in construction* because the architectural decision (treating reasoning as queryable telemetry rather than as prompt-time context) is validated through the rest of the pattern's operation; the specific machinery to operationalise it at scale is what's being built.

Charity Majors' framing of production telemetry as the specification informs this pattern in foundational ways. Birgitta Böckeler's harness engineering vocabulary — feedforward guides, feedback sensors, harnessability — informs the constitutional dimension. What this catalog contributes is the synthesis: the harness as architecture, self-observation as queryable property, reasoning as first-class telemetry, all composed into one ongoing practice. Self-observing systems exist in adjacent disciplines (eBPF for kernel observability, distributed tracing in microservices); the application to agentic modernization platforms is what is articulated here.

Related patterns

Pattern 6 (*The Compiler Principle*) is the broader principle this pattern operationalises in the governance layer — agentic decisions live within deterministic harness constraints. Pattern 15 (*Bounded MCP Servers*) is what the harness governs at the capability layer; pattern 16 (*Durable Orchestration*) is what

coordinates across capabilities and runs the harness as a long-lived workflow on durable infrastructure. Pattern 17 (*Heuristics as Explicit Artifacts*) is what the agents query when deciding, and what the retrospective agent refines based on this pattern's telemetry. Pattern 19 (*The Control Plane*) is the human-experience surface over the harness — where humans observe agentic behaviour, approve at gates, validate refinement candidates the retrospective agent surfaces, and inspect reasoning telemetry directly. Pattern 20 (*Team Topology and Bounded Context Alignment*) determines which humans the gates route to. The *Agent Army* antipattern names what happens when scale is pursued through agent multiplication rather than harness engineering.

Pattern 19: The Control Plane

Status: in construction.

Context

A modernization platform where many agents do work across many slices, generating substantial output every day. Humans participate at specific gates: validating slice boundaries, approving scaffold variants, certifying behavioural equivalence, authorising deployment. The humans cannot — and should not — read every line of generated code; their attention is the scarcest resource in the platform.

The platform must surface what humans need to decide, with the evidence to decide it, at the moment the decision is needed. Anything less wastes human attention. Anything more drowns it.

There is also a structural problem the control plane has to address. Modernization decisions accumulate across the engagement: which bounded contexts were identified, which slices were defined, which scaffolds were chosen, which translations were approved, which heuristics were applied, which gates were tripped, which divergences were accepted as intentional. The accumulation is the modernization's living architecture. But without a way to review the *deltas* — what changed since last week, what the agents proposed today, what's queued for review — the humans interact with the accumulation as a flat document instead of as an evolving system.

Problem

Without a designed human surface, the control plane collapses into one of two failure modes.

The first is *log fatigue*: the platform produces logs, dashboards, notification streams, all of which the human reviewer is expected to monitor. The signal-to-noise ratio collapses; humans miss what matters because everything looks like it matters. Engagement reviews degrade into "let me catch up on three weeks of notifications," which means decisions go unreviewed.

The second is *opacity*: the platform makes decisions internally and surfaces only summaries to humans. The humans are expected to trust the summaries because the underlying evidence is too dense to review. Trust erodes when summaries turn out to be wrong, and the humans can't see *why* a decision was made, only *that* it was made. Audit conversations become forensic rather than direct.

The deeper problem is that the control plane's job is volume discipline, not visibility maximisation. The instinct to surface "everything that happened" is wrong. The right discipline is to surface *only what*

requires a decision, with *only the evidence the decision needs*, at *only the moment the decision is needed*. Everything else is noise.

A specific symptom of getting this wrong: spec deltas (changes to behavioural specifications, architectural commitments, ontology vocabulary, heuristic catalog) get buried in commit history. The team can see *that* something changed but cannot easily see *what* the architectural significance is. Reviewers approve scaffolds without seeing the architectural decisions those scaffolds embody. Architects discover during retrospectives that decisions they would have intervened in were made and committed weeks earlier, with no signal to flag the moment of decision.

Forces

Human attention is scarce and expensive; the platform's output is abundant and cheap. The asymmetry must shape the control plane's architecture — every interaction surface costs human attention, and the control plane must ration that cost ruthlessly.

But humans also need *enough* context to decide well. Decisions made on summaries-without-evidence are worse than decisions made on no summary at all, because they feel informed but aren't. The control plane must provide *just enough* evidence to ground each decision — and *no more*.

Spec deltas are a specific case of the broader tension. Surface every change and the volume drowns reviewers. Surface only summaries and the architectural significance is lost. The right discipline is to surface deltas that *change the modernization's architectural commitments*, with the underlying diff available on request but not surfaced by default.

Pattern

Build the control plane as the human-experience layer over the entire platform. Its job is to surface, at the right moment, the artifacts a human needs to decide — and to make decisions easy when the evidence is in front of the reviewer.

Specifically, the control plane surfaces:

- **Graph decisions** — bounded context boundaries the discovery layer proposes, with the structural and behavioural evidence supporting each. Architects can accept, refine, or reject; the reasoning is captured.
- **Translation candidates** — agentic translations of paragraphs, with the side-by-side diff against the legacy and the Twin Verification (Pattern 13) verdict. Reviewers approve, request alternatives, or escalate.
- **Gate transitions** — the harness's (Pattern 18) state transitions, especially those that require human gating. The reviewer sees what state the workflow is in, what's pending, what evidence the agents have prepared.
- **Spec deltas** — changes to behavioural specifications (Pattern 14), architectural commitments (Pattern 7), ontology vocabulary (Pattern 4), heuristic catalog (Pattern 17), bounded context structure. Each delta surfaces with: what the change is, what evidence supported it, which agent (or which human) proposed it, what the architectural significance is. The delta is reviewable, dismissible, or escalable; significant deltas can be tagged for inclusion in periodic architectural reviews.

- **Audit trail** — the historical record of decisions: which agent did what, when, citing which heuristics, with what evidence. Queryable, filterable, exportable for compliance review.
- **Diff views** — side-by-side, three-way, structural comparisons. Different reviewers need different lenses; the control plane supports them all without prescribing.
- **Documentation re-renders** — the C4 model views, aggregate maps, context maps, ER diagrams (Pattern 9) regenerated whenever the substrates change. Reviewers see the architectural shape, not just the code shape.

The Alignment Record

The control plane's audit trail surface materializes as a specific artifact at each significant decision: the **Alignment Record**. Where a spec delta captures *what changed*, an Alignment Record captures *what was decided, by whom, against what evidence, under what constraints*.

An Alignment Record is created at each architectural decision point and carries the essential fields a decision must capture to remain auditable:

- A reference to the AsIs evidence that grounded the decision — the recovered use case, the L2 patterns detected, the source provenance trail.
- The ToBe outcome — the architectural pattern selected, the tier chosen, the seam drawn, the module assigned.
- The ontology version (Pattern 17) under which the decision was made — so that future engagements can identify which heuristics constrained the choice.
- The agent or agents that proposed candidates, and the alternatives they considered before the chosen path.
- The human who approved, with their reasoning when an override was applied.
- The attestation contract — what behavioural equivalence the oracle (Pattern 13) and Witness (Pattern 14) must subsequently verify.

The Alignment Record is the control plane's most consequential artifact. It is not just an audit log entry — it is the working contract between recovery and generation, the input that feeds Pattern 17's promotion gates, and the evidence trail that lets a regulator, an auditor, or a future modernization team retrace exactly why a decision was reached.

The Alignment Record is also where the AsIs/ToBe ownership discipline (introduced in the architectural interlude) becomes operationally visible. Each record names which substrates were AsIs (deterministic, evidence-bearing) and which were ToBe (deliberative, judgment-bearing). When a decision is challenged later, the record makes clear what was discovered versus what was chosen — and which side of that line a particular dispute lives on.

The control plane surfaces Alignment Records the same way it surfaces other decisions: at the moment the human is asked to validate them, with the evidence in front of them, with the option to dive into specifics on demand. Once approved, the record becomes immutable: the architectural decision it captured is preserved as a fact about the engagement, not as a state that can drift silently. Future revisions are themselves new Alignment Records that supersede previous ones — the trail compounds rather than collapses.

The control plane is not a one-size-fits-all surface. Different reviewers need different views — architects look at strategic decisions and spec deltas; developers look at translation candidates and Twin Verification

verdicts; operators look at gate transitions and rollout status; compliance reviewers look at the audit trail. The control plane supports these roles with role-specific views over the same substrates, not with separate tools each role must learn.

The spec delta surface deserves particular discipline. Deltas should be batched into reviewable units, not surfaced individually as they happen — the reviewer should see "the architectural significance of yesterday's work," not a constant stream of micro-changes. Significance is itself a judgment the control plane must make: trivial deltas (formatting changes, comment updates) should not surface; significant deltas (a new bounded context, an ontology refinement, a heuristic update, an aggregate boundary change) should. The line between trivial and significant is heuristic and evolves with engagement experience; it is itself a catalog entry (Pattern 17) subject to refinement.

In Rosetta, the control plane is Rosetta Studio. It surfaces all of the above through a unified interface, with role-aware views and configurable subscriptions to specific delta categories.

Consequences

Human attention is spent on decisions, not on monitoring. The control plane's job is to make the right artifact appear at the right moment; the reviewer's job is to engage with the artifact, not to discover what artifacts exist. Decision quality improves because reviewers have the evidence in front of them; decision latency drops because the right decision-maker is notified when their input is needed.

Spec deltas remain reviewable rather than disappearing into commit history. Architectural commitments don't drift without the architect noticing; ontology refinements don't slip through without validation; heuristic updates surface for review before they propagate through the agent population. The modernization's architecture stays governable as it evolves.

Audit conversations become direct rather than forensic. When a regulator or auditor asks "why was this decision made," the control plane can show: the artifact that surfaced, the evidence presented, the reviewer who approved, the rationale captured. The trail is reconstructible because it was constructed at the moment of decision, not assembled later.

The cost is the discipline of *not surfacing more than needed*. Every surface is a temptation to add more information, more notifications, more dashboards. The control plane's quality is in what it omits as much as in what it presents. This requires ongoing curation, evidence-based pruning, and a willingness to remove features that nobody uses.

A second cost is the integration burden. The control plane consumes evidence from every other part of the platform — the graph, the IR, the harness, the heuristic catalog, the Twin, Witness, the orchestrator. Each integration must be designed, maintained, and evolved as the underlying systems change. The control plane is structurally a downstream consumer of the entire platform; its release cadence must accommodate that dependency.

The control plane's status is *in construction*. The principle is validated — humans engaging with curated decision surfaces is more effective than humans engaging with raw output — but the operational machinery to do this consistently across the full range of decisions is still being built. The hardest parts (significance heuristics for spec deltas, role-aware view configuration, multi-modal diff rendering) are works in progress; the parts that exist today are sufficient to demonstrate the value but not yet sufficient to fully realise it.

Related patterns

Pattern 7 (*The Intermediate Representation*) provides the architectural commitments the control plane surfaces as spec deltas. Pattern 9 (*Pluggable Emitters*) renders documentation views the control plane displays. Pattern 13 (*Twin Verification*) provides translation candidate verdicts. Pattern 14 (*Hypothesis-Driven Verification*) provides production-derived specifications and divergence categorisations. Pattern 15 (*Bounded MCP Servers*) is what the control plane queries to access platform state. Pattern 17 (*Heuristics as Explicit Artifacts*) provides the significance heuristics the control plane applies when batching spec deltas. Pattern 18 (*The Harness as Self-Observing State Machine*) provides gate transitions and reasoning telemetry the control plane surfaces for human review. Pattern 20 (*Team Topology and Bounded Context Alignment*) determines which roles see which views. The *Agent Army* antipattern names the volume problem the control plane's discipline is designed to address.

Pattern 20: Team Topology and Bounded Context Alignment

Status: prototype-validated.

Context

A modernization engagement with a strategic capability map (Pattern 1), bounded contexts identified, scaffolds chosen, governance machinery operational. The platform can generate, verify, and govern modernized code at scale. What remains under-articulated is *who* operates the modernization — the team structure, the ownership boundaries, the handoff between legacy and modernized sides, and the relationship between the modernization team and the vendor providing the platform.

Most modernization writing treats team structure as a downstream concern: get the architecture right and the team structure will follow. This is wrong. Conway's Law applies whether the modernization acknowledges it or not: the system's structure will mirror the team's communication structure. If the team structure is wrong, the modernization will silently reshape bounded contexts to match it, undoing the strategic recovery work.

Problem

Three failure modes dominate when team structure is unaddressed.

The first is *misaligned ownership*. The bounded contexts identified through strategic recovery don't map cleanly to existing teams. The modernization either reshapes the contexts to fit current team boundaries (preserving the legacy's accidental structure under a modern surface) or reshapes the team boundaries to fit the contexts (organisationally disruptive, often resisted, often impossible in the short term). Without explicit design, the team defaults to whichever is operationally easier in the moment, accumulating debt either way.

The second is *missing legacy-side ownership*. The modernized C# has a team; the legacy mainframe also has a team, often with different reporting lines, different release cadences, different cultural norms. During the dual-run period (months or years), these teams must coordinate continuously: bug fixes that need replication across both sides, behavioural divergences that need triage, capability changes that need

synchronised rollout. Without explicit handoff design, coordination devolves into ad-hoc channels — Slack threads, recurring meetings, individual heroics — and the work that depends on coordination stalls.

The third is *vendor relationship ambiguity*. The modernization platform is typically provided by a vendor partner (in Rosetta's case, Microsoft via the SMM AIM offering). The vendor is not part of the customer's team but is operationally essential — they own the platform, evolve it, support engagements, provide expertise the customer doesn't have. Without explicit framing, the relationship oscillates between "vendor as supplier" (transactional, friction-prone) and "vendor as embedded team" (high coordination cost, unclear accountability). Neither extreme works for multi-year modernizations.

Forces

Conway's Law operates regardless of intent — Melvin Conway's 1968 observation that organizations design systems whose structure mirrors their communication structure has been validated repeatedly across software engineering. In modernization, the force is asymmetric: the modernized system inherits the team's communication structure, but the legacy system already encodes a *historical* team structure that may differ from current organization. The modernization is the moment of explicit choice about which structure persists.

Team boundaries must align with bounded context boundaries to maintain the strategic recovery's integrity. But team boundaries are organisationally expensive to change — they involve reporting lines, performance evaluation, career paths, and political capital. Aligning teams to ideal contexts is sometimes impossible; aligning contexts to existing teams undermines the modernization's value.

The legacy-modernized handoff is structurally complex. Two teams, two stacks, two release cadences, two sets of operational practices. The handoff is not a one-time event at cutover — it's a continuous coordination over the duration of the dual-run period.

Pattern

Apply the team-topology framework (Skelton and Pais, *Team Topologies*, 2019) to the modernization. Three team types compose the modernization's organizational structure:

- **Stream-aligned teams** own bounded contexts and have end-to-end responsibility for delivering business value through them. In modernization, one stream-aligned team per major bounded context cluster (claims, underwriting, billing). Each team operates its modernized contexts in production and is the authority for capability evolution within those contexts.
- **Platform teams** provide internal services and capabilities that stream-aligned teams consume. The modernization toolchain, cloud infrastructure, observability stack, security tooling. Platform teams reduce cognitive load on stream-aligned teams by offering well-defined services with clear interfaces. The vendor partner providing the modernization platform is structurally an external platform team — the customer's internal platform team is the integration surface to the vendor.
- **Enabling teams** transfer expertise into stream-aligned teams without taking ownership. In modernization, enabling teams typically operate at the boundary between legacy operations and modernized development, transferring institutional knowledge in both directions until the modernized team can operate the system independently. Enabling teams are time-bounded; their goal is to make themselves unnecessary.

The bounded context map (Pattern 1's capability mapping, refined through Patterns 3 and 4) becomes the input to team boundary design. Where bounded contexts and existing teams misalign, the modernization makes the misalignment explicit: either the team boundaries evolve to match the contexts (planned organizational change, with timeline and authority), or specific contexts are bridged by enabling teams during the transition (with explicit exit criteria for when the enabling team disbands), or the misalignment is documented as accepted debt (with explicit acknowledgment that this context will not realise the benefits the strategic recovery promised).

The legacy-modernized handoff is structured through enabling teams and explicit ownership transitions:

- During strategic recovery, the legacy operations team contributes domain knowledge to the modernization team through enabling-team facilitation. The legacy team knows how the system behaves under load, what the operational quirks are, where the historical incidents accumulated.
- During tactical generation and verification, the modernization team consumes the legacy team's domain knowledge but does not yet own the modernized system in production.
- During dual-run, both teams operate their respective sides with explicit coordination protocols. The control plane (Pattern 19) surfaces divergences to both teams; the orchestrator (Pattern 16) routes work to the team that owns the affected context.
- At cutover, ownership transfers from legacy team to modernized team for the specific bounded context. The legacy team's responsibility shrinks as cutover progresses. The enabling team facilitates the transfer and disbands when no longer needed.

The vendor relationship is explicit and bounded. The vendor (Microsoft, in Rosetta's case) is a platform team external to the customer. The customer's internal platform team is the integration surface — they consume the vendor's platform, escalate issues, provide feedback, participate in roadmap shaping. Stream-aligned teams consume platform capabilities through the internal platform team, not directly from the vendor. This avoids the failure modes of both supplier-vendor (transactional friction at every escalation) and embedded-vendor (accountability blur, coordination tax).

The harness (Pattern 18) and the control plane (Pattern 19) make team-context alignment operationally legible. When an agent needs human approval at a gate, the harness routes the work to the team that owns the affected context — not to a generic queue, not to whoever is on call. The routing is configured per bounded context and is part of the modernization's architectural metadata. The control plane surfaces decisions to the relevant team, not to all teams. Team boundaries become enforceable through tooling rather than aspirational in documentation.

Consequences

Bounded contexts and team ownership stay aligned. Strategic recovery's value survives organizational reality because the organizational reality has been designed alongside the architecture. Conway's Law works in the modernization's favour: the team structure reinforces the bounded contexts rather than undermining them.

The legacy-modernized handoff has explicit structure. Both teams know what they're responsible for at each stage, who escalates what to whom, when ownership transfers. The dual-run period (Pattern 22) becomes operationally manageable because coordination has design rather than being improvised.

The vendor relationship has explicit shape. The customer's internal platform team owns the integration with the vendor; stream-aligned teams consume capabilities without bilateral vendor coordination.

Vendor evolution (new platform features, deprecations, version upgrades) flows through one channel rather than disrupting every stream-aligned team.

The cost is upfront design and ongoing maintenance of the team-context map. The map is not static — bounded contexts evolve as understanding sharpens, team structures evolve as people move and priorities shift. Keeping the map current requires deliberate work; letting it drift undoes the pattern's value within months.

A second cost is organizational. The pattern often surfaces the truth that current team structure is not aligned with the bounded contexts the modernization needs. This can require organizational changes that are politically expensive — reorganizing teams, shifting reporting lines, redefining career paths. The pattern surfaces the need; the organization must decide whether to pay the cost.

The pattern is *prototype-validated* in a specific sense. The Skelton-Pais framework is established practice with substantial industry validation. Rosetta's own team structure follows the pattern — modernization team as stream-aligned, internal platform team integrating with Microsoft as external platform team, enabling team facilitating legacy-modernized handoff in the prototype's exercises. The pattern has not yet been tested against the scale and political complexity of a real customer modernization at full enterprise scope; that test is part of the next phase.

Matthew Skelton and Manuel Pais's *Team Topologies* (2019) is the canonical source for the team-type framework. Melvin Conway's 1968 paper "How Do Committees Invent?" remains the foundational text on socio-technical alignment. What this catalog contributes is the application of the team-topology framework specifically to mainframe modernization — the legacy-modernized handoff as enabling-team work, the vendor relationship as external platform team, the team-context routing through harness and control plane as enforcement mechanism. The principle is established; the modernization-specific application is what's articulated here.

Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) provides the capability map that feeds team-context design. Pattern 3 (*The Graph as Projection*) and Pattern 4 (*Domain Ontology as Independent Substrate*) refine the bounded context boundaries the team structure aligns to. Pattern 15 (*Bounded MCP Servers*) is the platform-team surface that internal platform teams own and stream-aligned teams consume. Pattern 16 (*Durable Orchestration*) routes work to the team that owns each context. Pattern 18 (*The Harness as Self-Observing State Machine*) enforces team-context alignment at gate transitions. Pattern 19 (*The Control Plane*) surfaces decisions to the team that owns the affected context. Pattern 21 (*Rollout and Cutover*) sequences ownership transfer from legacy team to modernized team at bounded context granularity. Pattern 22 (*Dual-Run Coexistence*) is the period during which legacy and modernized teams coordinate most intensively.

Part V: Safe Transition and Coexistence

The patterns in this group address the disciplined movement of bounded contexts from legacy authority to modernized authority, and the dual-run period during which both sides operate. They presuppose strategic recovery, tactical generation, verification, and governance — all four of the previous parts must be in place before a bounded context is a candidate for transition. The patterns here are where the modernization meets operational reality.

Transition is not an event. It is a sequence of evidence-gated stages, with explicit criteria for when each stage is complete and when the next can begin. Coexistence is not a problem to be minimised. It is a designed operational state, often lasting months per bounded context, with its own infrastructure and its own discipline. The patterns in this part address both.

A small note on numbering: Pattern 23 appears first in this part, followed by Patterns 21 and 22. The numbering reflects the order in which patterns were added to the catalog rather than the order in which they apply during transition. Logically, Pattern 23 (*Completion Criteria*) gates entry into Pattern 21 (*Rollout and Cutover*), which gates entry into the Coexistence period that Pattern 22 (*Dual-Run Coexistence*) operates over. Read them in that order.

Pattern 23: Completion Criteria as Designed Property of Each Bounded Context

Status: designed.

Context

A modernization that has reached the late stages of work on a particular bounded context. Twin Verification (Pattern 13) is producing verdicts in dev mode. Witness (Pattern 14) is capturing production behaviour during dual-run. Cutover stages (Pattern 21) are progressing. The team has invested months on this bounded context. The question now confronting them is the one the catalog has not yet answered: *is it done?*

Not "does it work" — that's the question Patterns 13 and 14 address. Not "is it deployed" — that's the question Pattern 21 sequences. The question is whether the modernization of this bounded context has reached the point at which further work is no longer earning value, and the team should declare completion, transfer ownership, and move on.

Problem

In every modernization at scale, completion is the question teams answer worst. Three failure modes recur:

The first is *endless modernization*. Without explicit completion criteria, work continues until external constraints force it to stop — budget exhausted, executive patience worn out, team rotated away. The team finishes when something outside the modernization decides for them, not when value is delivered. The bounded context that has been "almost done" for three months absorbs disproportionate cost while delivering diminishing returns; nobody can articulate why except "we're not quite ready yet."

The second is *premature declaration*. Cutover pressure mounts. Stakeholders ask when the legacy can be decommissioned. The team declares the bounded context complete before evidence justifies the claim. Production traffic reveals what dev-mode verification missed. Witness catches divergences that should have surfaced during dual-run but didn't because the corpus was incomplete. The team learns, painfully, that "feels done" and "is done" are different statements.

The third is *threshold drift*. Completion criteria, if set at all, are set informally — "good enough" tested against intuition rather than evidence. Each bounded context's "good enough" turns out to mean different things at different times depending on who's reviewing. Two bounded contexts ostensibly modernized to the same standard exhibit different operational behaviour because the standard was never explicit.

The underlying failure is that completion is treated as a *judgment made at the end of the work* rather than as a *designed property declared before the work begins*. The catalog has many patterns for how to do modernization work; it has not yet articulated the pattern for naming when modernization work for a bounded context is finished.

Forces

Completion criteria need to be strict enough that meeting them constitutes real evidence the modernization is done, and loose enough that meeting them is actually achievable within the engagement's lifetime. Set the bar too high and no bounded context ever completes; set it too low and "complete" means nothing.

Completion is multi-dimensional. Behavioural equivalence (Patterns 13 and 14) is necessary but insufficient — a bounded context can pass every behavioural test and still be incomplete if its canonical ontology (Pattern 4) has not stabilised, if ownership has not transferred to the receiving team (Pattern 20), or if its verification corpus does not exercise representative legacy behaviour. Different dimensions of completion live in different parts of the catalog; the catalog itself has not synthesised them.

Completion criteria must be set before they can be applied. Naming them at the end of the work — when teams are tired and stakeholders are impatient — produces criteria that rationalise the current state rather than measuring it. The criteria have to be set during strategic recovery, when the team can think clearly about what completion means for *this* bounded context given its tier, capability classification, and operational stakes.

Completion criteria must also vary by bounded context. A tier-3 strategic core (Pattern 8) deserves stricter completion than a tier-0 generic supporting subdomain. A capability the business classifies as a core differentiator (Pattern 1) deserves stricter completion than one classified as commodity. Uniform criteria across the modernization estate either over-engineer the cheap work or under-engineer the strategic work.

Pattern

Treat completion criteria as a first-class artifact of strategic recovery, declared per bounded context, before generation begins. The criteria are an Alignment Record (Pattern 19): proposed during recovery, validated against domain understanding, approved by the architect with sign-off recorded, immutable once set. Subsequent revisions are themselves new Alignment Records that supersede earlier ones, with explicit rationale for the revision.

Completion criteria span five dimensions, each grounded in evidence the catalog already produces:

Behavioural equivalence (anchored in Patterns 13 and 14). Divergence rate below a threshold set per bounded context, sustained across a window appropriate to the context's traffic pattern. For a tier-3 core differentiator with continuous traffic, the threshold may be near-zero divergence sustained over weeks. For a tier-0 generic context with intermittent batch traffic, the threshold may tolerate categorised drift over a few cycles. The threshold is named explicitly; "no divergences" is rarely the right answer.

Coverage (the dimension the catalog otherwise leaves implicit). The fraction of the legacy's behavioural surface that the verification has actually exercised. This includes paragraph reachability through Twin Verification fixtures, predicate coverage across the recovered slices, transactional-shape coverage across the recovered patterns, and failure-mode coverage across the legacy's recoverable behaviour. Coverage is queryable telemetry from the harness (Pattern 18), not an estimate. A bounded context with 95% behavioural equivalence over 30% coverage is not nearly as complete as the same context with 85% equivalence over 95% coverage. Coverage matters; the threshold for it is named.

Ontological alignment (anchored in Pattern 4). The canonical ontology for this bounded context is stable; the modernized system uses the canonical vocabulary; legacy drift has been reconciled (either preserved deliberately with documented justification or refactored away). The criterion: the ontology has not changed in N weeks of work, and the modernized C# names match the canonical terms. An ontology still drifting when completion is claimed is a signal that strategic recovery is not finished, regardless of how clean the code looks.

Operational evidence (anchored in Pattern 14, observed through production). The modernized system has handled real production traffic at scale, with divergences (if any) categorised and resolved per Pattern 14's typology. The criterion includes sustained operation: not a single successful day but a window sufficient to expose temporal patterns the dev-mode corpus didn't capture. End-of-month, end-of-quarter, year-end if relevant. Operational evidence is what production reveals that dev mode cannot.

Team ownership transfer (anchored in Pattern 20). The receiving stream-aligned team operates the modernized bounded context independently of the modernization team. Enabling teams have disbanded for this context (Pattern 20's exit criterion). The team's operational runbooks, incident response, observability dashboards, and on-call rotation are functional and exercised. Completion is not just code correctness — it is *organisational completion*, and a bounded context whose receiving team cannot operate it has not finished modernization regardless of how clean the verification verdict reads.

Each dimension has a threshold set per bounded context, calibrated to its tier and capability classification. Tier-3 strategic core gets strict thresholds across all five dimensions. Tier-0 generic supporting context gets thresholds calibrated to the cost of getting it slightly wrong (often: strict on behavioural equivalence and ontology, looser on coverage breadth, since uniform exhaustive coverage of a CRUD reference-data context is waste). The capability map (Pattern 1) and the tier classification (Pattern 8) inform the

thresholds.

The completion criteria are surfaced through the control plane (Pattern 19) continuously — not as a one-time check at the end, but as standing telemetry through the bounded context's late-stage work. The team sees, at any time, which criteria are met and which remain open. When all criteria are met and have been stable for the declared sustain-window, the bounded context is *eligible* for completion. The completion declaration itself is a gate the harness (Pattern 18) routes to the architect, with the evidence assembled for review. The architect approves completion or identifies what remains.

After completion is declared, the bounded context's Alignment Records become part of the permanent engagement record. The legacy code for that bounded context enters its decommission sequence per Pattern 21. The receiving team's ownership becomes operational. The modernization team's attention reallocates to other bounded contexts.

Consequences

The team has a clear answer to the question "is this done?" — not feelings, not pressure, but evidence assembled against criteria set in advance. Completion becomes a designed property rather than a judgment made under duress. Endless modernization is structurally prevented because the criteria explicitly say what done looks like. Premature declaration is structurally prevented because the criteria require evidence, not assertion. Threshold drift is structurally prevented because the thresholds are named, immutable, and surfaced through the control plane throughout the work.

The completion criteria themselves become valuable engagement artifacts. Each bounded context's completion criteria are an articulated theory about what made *that specific context's modernization* succeed or fail. Across many bounded contexts within one engagement, patterns emerge: which thresholds were met easily, which proved sticky, which had to be revised mid-flight and why. The retrospective agent (Pattern 18) consumes these patterns; the heuristic catalog (Pattern 17) absorbs lessons about what completion criteria work in practice; the next engagement starts with better defaults.

For regulated industries, completion criteria with their Alignment Records constitute regulator-ready evidence that the modernization met explicit standards. The criteria, the thresholds, the evidence each criterion was measured against, the architect who approved completion — all are queryable through the control plane. Audit conversations become direct rather than reconstructive.

The cost is upfront discipline. Strategic recovery now includes naming completion criteria for each bounded context — explicit threshold-setting per dimension, calibrated to tier and capability. This is real work, requiring architect attention before the bounded context's generation work has even begun. Teams used to "we'll know it when we see it" will resist; the pattern's value depends on holding the discipline anyway.

A second cost is coverage telemetry as a queryable surface. Coverage as a named dimension requires instrumentation in the verification stack — Twin Verification (Pattern 13) and Witness (Pattern 14) must emit coverage telemetry, not just pass/fail verdicts. The instrumentation is engineering work that doesn't deliver new features; it delivers the ability to make completion criteria meaningful. Without coverage instrumentation, the coverage dimension collapses into estimation and the pattern loses one of its five anchors.

A third cost is the willingness to declare completion when criteria are met, even if the team's instinct

is to keep working. This is a cultural shift more than a technical one. Engineering teams trained to perfectionism will find ways to identify additional work after criteria are met. The pattern's discipline includes accepting that completion means *the criteria are met*, not *the team has stopped finding improvements*. Continuous improvement is the modernized system's normal state; modernization itself is finished when the criteria say so.

The pattern is *designed*, not prototype-validated. The prototype has the substrates required (verification telemetry, Alignment Records, the heuristic catalog), but completion criteria as a synthesised pattern across the five dimensions has not been operationalised. Real engagements will teach which thresholds work at what tier, which dimensions weigh most in practice, which failure modes the criteria miss. Until that experience accumulates, this pattern stands on architectural reasoning grounded in named gaps the field has documented — endless modernization and premature cutover are not theoretical risks; they are reported failure modes across the industry. The pattern is the catalog's articulated response.

The notion of explicit completion criteria for software work is not novel in isolation. Acceptance criteria in agile methodologies, Definition of Done in Scrum, formal correctness conditions in mathematical specification — each is a related practice in adjacent territory. What this pattern contributes is the synthesis specifically for *AI-assisted mainframe modernization at bounded-context granularity*: the five-dimensional structure calibrated to tier and capability, the integration with the catalog's existing verification and governance substrates, the discipline of declaring completion before generation begins.

Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) provides the capability classification that informs completion thresholds. Pattern 4 (*Domain Ontology as Independent Substrate*) is the anchor for the ontological-alignment criterion. Pattern 8 (*Tier-Aware Scaffolding*) provides the tier classification that calibrates threshold strictness across all five dimensions. Pattern 13 (*Twin Verification*) and Pattern 14 (*Hypothesis-Driven Verification*) anchor the behavioural-equivalence and operational-evidence criteria. Pattern 17 (*Heuristics as Explicit Artifacts*) absorbs lessons about which completion thresholds work in practice through its layered promotion gates. Pattern 18 (*The Harness as Self-Observing State Machine*) surfaces coverage telemetry as queryable evidence and gates the completion declaration as an architect-approval transition. Pattern 19 (*The Control Plane*) surfaces completion criteria continuously through the bounded context's late-stage work and presents the assembled evidence for the architect's completion decision. Pattern 20 (*Team Topology and Bounded Context Alignment*) anchors the team-ownership-transfer criterion. Pattern 21 (*Rollout and Cutover at Bounded Context Granularity*) is what completion *gates into* — the bounded context's cutover stages run only when completion criteria are met for the appropriate stage.

Pattern 21: Rollout and Cutover at Bounded Context Granularity

Status: prototype-validated.

Context

A modernization where bounded contexts have been generated, verified through Twin Verification (Pattern 13) and Hypothesis-Driven Verification (Pattern 14), and are ready for deployment. The legacy mainframe is still running and processing real business traffic. The modernization must move from "modernized C# exists and behaves correctly in test" to "modernized C# is the production system for this bounded context" without interrupting business operations or losing transactional integrity.

The legacy and the modernized system will coexist during the transition. Some bounded contexts will be on the modernized side while others remain on the legacy. Some traffic will route to one, some to the other, sometimes shadowed for validation, sometimes split for incremental confidence. The coexistence period may last days for a small bounded context, months for a large one, years across the full modernization estate.

Problem

The default failure mode is "big bang" cutover: at a planned moment, traffic switches from legacy to modernized across the entire system. Big bang cutovers are catastrophic because the modernization's verification is inherently incomplete (no test coverage matches production scope) and the rollback path requires reversing a system-wide change under operational pressure. Outages cascade because dependencies fail in unexpected ways; rollback is messy because data has already been written to the modernized side; customer-visible impact is severe because the failure surface is the entire system.

A second failure is "all or nothing per bounded context": even if cutover happens per bounded context rather than per system, each context's cutover is itself a big bang within its scope. The transition for that context is sudden, and the same failure modes apply at smaller scale.

A third failure is missing rollback infrastructure. Many modernizations plan the forward path carefully and treat rollback as theoretical. When divergence is detected post-cutover, the team discovers that rolling back requires data movement and contract reversal they hadn't designed for. The rollback path is unrehearsed and unsafe.

Forces

The modernization team needs deployment confidence to grow incrementally as evidence accumulates. The business needs the modernized system to handle real production traffic without interruption. The operations team needs clear rollback paths if anything goes wrong. Regulatory environments often require explicit cutover plans, evidence-based progression, and documented rollback procedures.

Cutover speed and cutover safety pull in opposite directions. Fast cutover gets the modernization to value sooner; slow cutover reduces risk. The right answer is not on either extreme — it's progressive cutover with explicit safety gates at every stage.

Pattern

Treat rollout and cutover as a first-class engagement design concern. Cutover happens at bounded context granularity — never at the whole-system scope — with explicit progression through five stages:

1. **Parallel run** — modernized C# deployed alongside legacy, both processing real traffic, results compared continuously through Witness (Pattern 14). No production decisions depend on

modernized output yet. The goal is to accumulate evidence that the modernized side handles real traffic correctly.

2. **Shadow validation** — modernized C# receives every input the legacy receives but its outputs are discarded for business purposes. The comparison continues but at higher volume and broader scope. Shadow validation surfaces divergences that parallel run missed because of scope; it stress-tests the modernized system at full production load without business impact.

3. **Incremental routing** — a small percentage of real traffic (typically starting at 1% or 5%) is routed to the modernized side as the authoritative response. Legacy still receives and processes the same traffic in parallel for divergence detection. The percentage grows as evidence accumulates — 1%, 5%, 25%, 50% — with explicit hold-points where evidence is reviewed before progression.

4. **Canary monitoring** — at each percentage threshold, monitoring intensifies: divergence rates, error rates, latency distributions, downstream system behaviour. The modernized side is the canary; if metrics degrade, traffic returns to legacy automatically.

5. **Cutover** — modernized C# becomes the authoritative production system for the bounded context. Legacy continues to receive traffic in shadow mode for some retention period (typically 30-90 days) as final divergence detection. After the retention period, legacy traffic stops.

6. **Decommission** — legacy code for the cutover bounded context is removed from active deployment. Source code is archived (it remains part of the historical record and the audit trail). The bounded context's modernization is complete.

Rollback rehearsal is required before cutover. The team explicitly tests the rollback path in pre-production: shutdown of modernized side, restoration of full legacy authority, reconciliation of any data written to modernized side during the test window. The rollback must work end-to-end in pre-production before cutover is authorized in production. Untested rollback paths are not rollback paths; they are aspirations.

The harness (Pattern 18) encodes the stages as a state machine with explicit gates between them. The progression from parallel-run to shadow-validation to incremental-routing requires explicit human approval at each transition; the agent prepares the evidence for the gate, the human reviews and approves. Control Plane (Pattern 19) surfaces the evidence: divergence rates by category, error rates, latency distributions, business metrics. The decision to progress is human; the evidence to ground it is agentic.

For systems with many bounded contexts (typical of mainframe modernizations), cutovers are sequenced rather than parallel. The capability map (Pattern 1) and team topology (Pattern 20) inform the sequence: high-business-value contexts cut over earlier (to deliver value faster), commodity contexts cut over later (lower risk, less business pressure), tightly-coupled contexts cut over together (to avoid integration complexity across the cut boundary).

Consequences

Cutover risk is bounded by bounded context size, not system size. A divergence detected during incremental routing affects a small percentage of traffic for a single bounded context; rollback restores that bounded context's traffic to legacy while the rest of the system continues unaffected. The blast radius of any failure is contained.

Evidence accumulates progressively. By the time cutover happens, the modernized side has processed real

production traffic at scale, with divergences detected and resolved at each stage. Confidence is grounded in operational evidence rather than in test coverage.

Rollback is real, not theoretical. The team has practiced the rollback procedure end-to-end; the operations team knows what to do if a post-cutover divergence appears; the data reconciliation path is designed and tested. Rollback becomes an operational option, not a crisis response.

The cost is engagement duration. Progressive cutover takes longer than big bang — months of dual-run per bounded context, with the legacy remaining operational throughout. The infrastructure to support dual-run (routing layers, divergence detection, reconciliation tooling) must be designed and built; this is the work Pattern 22 (*Dual-Run Coexistence*) addresses.

The cost is also operational complexity during the transition. Two systems running in parallel for months or years is harder than one system. The team's attention is split between modernizing new contexts and operating the cutover contexts. The capability map (Pattern 1) and the sequencing it informs help manage this — sequencing cutovers so that operational burden never exceeds team capacity.

The five-stage progression is what's validated in Rosetta's prototype. Real engagement experience may teach that some stages can collapse (shadow validation may not be necessary for low-risk commodity contexts) or expand (a high-criticality core differentiator may need finer-grained percentage increments). The pattern is the principle of explicit-staged progression with rollback rehearsal; the specific stages are a starting point.

Sam Newman's *Monolith to Microservices* (Newman, 2019) and the broader microservices migration literature articulate similar progressive deployment patterns. Martin Fowler's strangler fig (Fowler, 2004) is the architectural ancestor. What this catalog contributes is the framing of progressive cutover at *bounded context granularity* specifically — not per-feature, not per-service, but per the strategic bounded contexts that the modernization has invested in identifying. The bounded context is what the business recognizes as a coherent capability; cutting over at that granularity is what makes the modernization legible to stakeholders.

Related patterns

Pattern 1 (*Business-Aligned Capability Strategy*) determines cutover sequencing — high-value capabilities first. Pattern 13 (*Twin Verification*) is the dev-mode gate before any cutover stage begins. Pattern 14 (*Hypothesis-Driven Verification*) is the production-mode evidence engine that grades divergences during cutover stages. Pattern 18 (*The Harness as Self-Observing State Machine*) encodes the cutover stages as a workflow with explicit human gates. Pattern 19 (*The Control Plane*) surfaces cutover evidence for human review. Pattern 20 (*Team Topology and Bounded Context Alignment*) determines which team owns each cutover and ownership transfer. Pattern 22 (*Dual-Run Coexistence*) is the infrastructure pattern that supports the dual-run period this pattern operates over.

Pattern 22: Dual-Run Coexistence: CDC, Reconciliation, and the Bridge Period

Status: in construction.

Context

A modernization that has chosen progressive cutover (Pattern 21) rather than big bang. For weeks or months per bounded context, legacy and modernized systems both operate against the same business operations. The systems share data (or maintain corresponding copies), serve overlapping user populations, and must produce mutually consistent results.

The dual-run is unavoidable for any modernization at meaningful scale. The infrastructure to support it is what this pattern addresses.

Problem

Naive dual-run produces three categories of failure.

The first is *data drift*: legacy and modernized systems both write to their respective data stores, and over time the stores diverge. Some divergence is expected (the modernized side may have richer schemas, may capture new metadata, may store data in different forms); some is dangerous (the same business operation produces different financial balances on each side, leading to reconciliation failures and audit issues). Without explicit synchronization, the divergence accumulates silently until reconciliation reveals it.

The second is *write conflicts*: when both legacy and modernized sides accept writes for the same business operation, the order of writes matters and may differ between sides. The customer updates their address on the legacy mainframe at the same moment a system process updates it on the modernized side; both writes succeed, but the final state may differ. Resolution requires explicit conflict resolution rules; without them, the systems can become permanently inconsistent.

The third is *vocabulary mismatch at the boundary*: legacy and modernized systems use different vocabulary for the same concepts. The legacy calls a record `CUST-MAST-REC`; the modernized side calls it `Customer`. The legacy stores dates as `YYYYMMDD` integers; the modernized side stores them as ISO 8601 timestamps. Every integration point requires translation; without explicit translation infrastructure, the translation logic scatters across application code and becomes unmaintainable.

Forces

Both systems must function for the duration of dual-run. The business cannot wait for the modernization to complete before deriving value; it cannot accept periods of system unavailability. The legacy must continue working as it has been; the modernized side must work correctly against real traffic.

Data must remain consistent enough that business operations don't fail. Perfect consistency at every instant is too expensive (would require distributed transactions across legacy and modernized data stores, which most legacy stacks don't support); zero consistency is unacceptable (the business depends on records matching). The right answer is a controlled level of eventual consistency with explicit reconciliation infrastructure.

Synchronization mechanisms have engineering cost and operational risk. Each mechanism (CDC streams, outbox-pattern dual-writes, periodic batch reconciliation) has its own failure modes. The mechanisms must be chosen per data domain based on the consistency requirement, the legacy stack's capabilities, and the operational budget.

Pattern

Build dual-run infrastructure that addresses the three failure modes explicitly.

For data consistency, use **Change Data Capture (CDC)** as the default mechanism: changes on the legacy side stream to the modernized side; changes on the modernized side stream back to the legacy. The CDC mechanism depends on the legacy stack — IBM InfoSphere CDC for DB2, Oracle GoldenGate for Oracle, Debezium-style readers for systems where direct stream capture is available, journaling-based readers for VSAM, DPROP or equivalent for IMS. The choice of mechanism is per legacy data store; the architecture above the mechanism is the same.

For write authority, define **per-domain ownership** with explicit transition: at any time during dual-run, one side is the authoritative source for each data domain. The authority may transfer at cutover stages — initially legacy is authoritative, then writes flow to modernized side with legacy receiving the CDC stream, then modernized becomes fully authoritative. The transition is explicit, scheduled, communicated to all consumers. Bidirectional writes during the transition are minimized; where they are necessary, conflict resolution rules are defined upfront (last-write-wins, business-rule-based merge, escalation to human).

For dual-write scenarios within the modernized side, use the **transactional outbox pattern**: writes to the local data store and outbound events are committed in a single transaction; a separate process publishes events from the outbox. This pattern, well-supported by frameworks like Wolverine in .NET, prevents the inconsistency window between data update and event publication that would otherwise compromise CDC reliability.

For vocabulary translation, use **bridge APIs** that wrap legacy data access and translate at the boundary. The bridge API is structurally an anti-corruption layer (Pattern 10's Evans-derived translation discipline) applied at the data plane rather than at the application plane. The bridge translates legacy schemas into canonical ubiquitous language (Pattern 4) on read; it translates canonical language back to legacy schemas on write. The translation logic lives in one place rather than scattered through application code; both legacy-aware and modernized consumers can use the bridge.

For reconciliation, build **continuous and scheduled reconciliation**. Continuous reconciliation samples writes on both sides and compares them in near-real-time; divergences surface as alerts. Scheduled reconciliation runs periodically (nightly, end-of-month, end-of-quarter) and compares larger windows of data; the granularity depends on the business cadence. Each reconciliation run produces a divergence report; each divergence has a remediation playbook specific to its data domain. Witness (Pattern 14) provides the underlying machinery; reconciliation is a specialized application of the same evidence-capture infrastructure.

The bridge period — the full duration of dual-run for a bounded context — is a designed operational state, not a transition to be minimized. Some bridge periods are weeks (small, low-risk bounded contexts); some are months (large core-differentiator contexts requiring extended validation); some are years (estate-wide modernizations where the cutover wave is itself long). The infrastructure must support all of these durations without operational degradation.

Consequences

The dual-run becomes operationally credible. Data stays consistent within explicit tolerance; vocabulary translates at the boundary rather than scattering through application code; reconciliation surfaces

divergences early enough to remediate before audit exposure. The progressive cutover (Pattern 21) becomes possible at scale because the infrastructure to support it exists.

Cutover risk decreases because the dual-run period has surfaced and remediated divergences in advance. By the time a bounded context cuts over, the team has weeks or months of evidence that the modernized side handles its data correctly under real production load. Confidence is grounded in operational evidence rather than in pre-deployment testing.

The cost is the dual-run infrastructure itself — CDC pipelines, bridge APIs, outbox processors, reconciliation jobs, remediation playbooks. This is substantial engineering work that delivers no direct business value (it exists only to enable the cutover, after which it is decommissioned). For organizations that have done many migrations, the infrastructure can be reused across projects, amortizing the cost; for first-time modernizers, it is engagement-specific investment.

The cost is also operational complexity during the bridge. Two systems running in parallel with continuous synchronization and reconciliation requires operational attention: monitoring the CDC streams, triaging reconciliation divergences, responding to bridge API failures. The operations team must be staffed for this complexity for the full duration of the bridge.

The pattern is *in construction*. The principles (CDC for synchronization, transactional outbox for dual-write integrity, bridge APIs for vocabulary translation, continuous and scheduled reconciliation for divergence detection) are established in the broader microservices and integration literature. Their composition specifically for mainframe modernization dual-run, with the CDC mechanism choices calibrated to legacy stacks and the reconciliation tied into Witness's evidence engine, is what's being built in the Rosetta prototype.

This pattern addresses what Nick Tune has documented as the synchronization antipatterns that wreck incremental migrations when applied naively: *Bi-directional Model Sync* (both sides translating to each other's models continuously, fragile and resistant to retirement), *Asymmetrical Validation* (modernized side enforcing stricter rules than legacy holds historically), *Tri-directional Sync* (multiple legacy systems compounding the synchronization complexity). The corrective is structural: define data authority per domain on an explicit schedule, use bridge APIs (anti-corruption layers) to translate rather than synchronize models, resist the instinct to keep all sides equally authoritative throughout the transition. Definitions and pointers to Tune's articulation are in the glossary and antipatterns sections.

Pat Helland's *Life Beyond Distributed Transactions* (Helland, 2007) informs the explicit eventual-consistency framing. The CDC pattern itself has decades of articulation in data integration literature (Kimball and Caserta, 2004; subsequent work in stream processing). The transactional outbox pattern is canonical microservices engineering (Richardson, *Microservices Patterns*, 2018). What this catalog contributes is the synthesis specifically for mainframe modernization dual-run, with the calibration to specific legacy stacks (DB2, VSAM, IMS) and the integration with Witness (Pattern 14) as the evidence layer for reconciliation.

Related patterns

Pattern 4 (*Domain Ontology as Independent Substrate*) provides the canonical vocabulary that bridge APIs translate to and from. Pattern 10 (*Commands and Events as Logical Boundary*) is the architectural pattern bridge APIs implement at the data plane. Pattern 14 (*Hypothesis-Driven Verification*) is the evidence engine that reconciliation depends on. Pattern 18 (*The Harness as Self-Observing State*

Machine) encodes the bridge period as a long-lived workflow state. Pattern 19 (*The Control Plane*) surfaces reconciliation divergences for human triage. Pattern 20 (*Team Topology and Bounded Context Alignment*) determines which team owns each bridge API and reconciliation job. Pattern 21 (*Rollout and Cutover at Bounded Context Granularity*) is the operational frame this pattern serves. The *Synchronisation antipatterns* in the antipatterns section name what this pattern is correcting.

Antipatterns

A pattern catalog is most useful when it names what it's built against. The antipatterns below are failure modes the field has encountered, articulated briefly. Naming them helps clarify what the patterns above are correcting.

Jobol. Generated C# that has been mechanically translated from COBOL but retains COBOL's structure, idioms, and shape. The result reads like COBOL written in C# syntax: deeply nested control flow, primitive obsession, monolithic procedural style. The compiler principle (Pattern 6) and tier-aware scaffolding (Pattern 8) are correctives.

Silent semantics loss. A modernization that produces syntactically reasonable C# but loses behavioural detail in the translation. Edge cases, error paths, transactional guarantees disappear quietly. The user doesn't know until production reveals the loss. The legacy as oracle (Pattern 2) and Twin Verification (Pattern 13) are correctives.

False clean code. Generated C# that follows modern idioms (dependency injection, clean architecture, async/await) without preserving the legacy's actual behaviour. The code looks like what a senior developer would write today; it just doesn't do what the legacy does. Hypothesis-driven verification (Pattern 14) is the corrective: behavioural equivalence is the standard, not aesthetic alignment with current best practice.

Frozen architecture. A modernization that lifts the legacy's architectural decisions into the modernized system, treating them as preserved value rather than as historical artifacts. The result is modern code with thirty-year-old architectural commitments embedded in it. In DDD terms, this is failing to perform strategic design — accepting whatever bounded contexts the legacy implicitly created, whatever subdomain types the legacy implicitly chose, whatever was once core that has since become commodity. Pluggable emitters (Pattern 9) and tier-aware scaffolding (Pattern 8, grounded in Evans' core/supporting/generic typology) make architectural decisions explicit and replaceable.

Vendor oracle. Trusting a single vendor's tooling as the authoritative ground truth for the modernization. When the vendor's tool produces output, the modernization treats that output as correct. The vendor becomes the oracle, displacing the legacy itself. The legacy as oracle (Pattern 2) inverts this: the legacy is the truth source, and any tool — vendor or otherwise — is verified against it.

Behavioural equivalence without ontology. A modernization that achieves perfect behavioural fidelity to the legacy — Twin Verification passes, Hypothesis-Driven Verification confirms, every gate in the harness is green — and yet produces a modernized system that inherits the legacy's ontological confusion. Three definitions of "active customer" survive the cut. Two interpretations of "balance" remain incompatible. Vocabulary that meant one thing in 1992 still means another in 2008, now

expressed in clean modern code that makes the confusion harder to detect. The modernization is technically successful and fundamentally broken. In DDD terms, the modernization preserved tactical implementations but never established a ubiquitous language; the result is a modernized polyglot, fluent in three dialects of the same domain, mastering none. Anthony Alcaraz's argument applies with force: the orchestration is fine, the grounding is missing, the output is confidently wrong. The legacy as oracle (Pattern 2) is necessary but not sufficient; *Domain Ontology as Independent Substrate* (Pattern 4) is the corrective. Behavioural equivalence preserves what the legacy does. Ontology decides what the legacy *should have been about* — and the modernization team has to answer that question independently.

Synchronisation antipatterns during incremental migration. Three failure modes recur when incremental migration is overlaid on a real legacy landscape, all documented by Nick Tune. *Bi-directional Model Sync* keeps a legacy entity and a modernized entity in continuous round-trip synchronisation, with each side's invariants having to survive translation through the other's model; the arrangement is fragile and resists retirement. *Asymmetrical Validation* compounds the problem: the modernized side enforces stricter rules than the legacy, while the legacy holds years of data that does not satisfy those rules, producing chronic drift the team cannot resolve without amnesty or backfill. *Tri-directional Sync* is the combinatorial form: when the migration is overlaid on a landscape that already contains multiple legacy systems with overlapping data, the team must build and operate not two synchronisation flows but six, twelve, or more. The corrective is structural: define data authority per domain on an explicit schedule (Pattern 22), use anti-corruption layers (Pattern 10) to translate vocabularies rather than synchronise models, and resist the instinct to keep all sides equally authoritative throughout the transition. Definitions and pointers are in the glossary.

Agent army. An agentic system that scales by multiplying agents — more specialised agents, more coordination layers, hierarchies, swarms, agentic mesh. The instinct is *if one agent is good, many agents are better*. The operational failure is twofold. First, coordination overhead grows faster than capability — the system spends more compute on agents talking to each other than on doing the work. Second, the army generates vast volumes of output: documentation, code, alternatives, justifications, reasoning records. The volume saturates the reviewers downstream; humans cannot consume at the rate the agents produce. The modernization stalls at the bottleneck of cognitive load, not at any technical limit. The corrective is structural: build a harness (Pattern 18), not an army. The discipline of deterministic constraints, verifiable invariants, and explicit gates produces tractable modernization; more agents do not. Spec deltas (Pattern 19) operationalise the volume discipline at the review surface — fewer artifacts, denser articulation. *More is not better. Less is better.*

Naive self-observation. A harness that observes itself but does so naively — measuring gate passage rates, heuristic application frequency, agent confidence scores — and treating those measurements as ground truth. Goodhart's law applies immediately: when a measure becomes a target, it ceases to be a good measure. Agents that learn what the harness watches learn to produce signals that look good without doing the underlying work. Heuristic gaming, confidence inflation, gate-shape pattern-matching, reasoning-trace performance — each is a predictable consequence of self-observation that doesn't anticipate being gamed. The corrective is the layered observation that Pattern 18 (*The Harness as Self-Observing State Machine*) describes: action-and-outcome as primary signal, reasoning telemetry as supplementary, and continuous detection of divergence between the two. The harness must expect to be gamed, and must watch for the signature of gaming, not just for the metrics that an honest agent would produce honestly. A harness that watches outcomes can catch a heuristic that no longer measures what it was meant to measure;

a harness that only watches the heuristic itself cannot.

Antipatterns and corrective patterns reference table:

Antipattern	Corrective patterns
Jobol	Pattern 6 (Compiler Principle); Pattern 8 (Tier-Aware Scaffolding)
Silent semantics loss	Pattern 2 (Legacy as Oracle); Pattern 13 (Twin Verification)
False clean code	Pattern 14 (Hypothesis-Driven Verification)
Frozen architecture	Pattern 8 (Tier-Aware Scaffolding); Pattern 9 (Pluggable Emitters)
Vendor oracle	Pattern 2 (Legacy as Oracle)
Behavioural equivalence without ontology	Pattern 2 (Legacy as Oracle); Pattern 4 (Domain Ontology as Independent Substrate)
Synchronisation antipatterns during incremental migration	Pattern 10 (Commands and Events); Pattern 22 (Dual-Run Coexistence)
Agent army	Pattern 18 (The Harness); Pattern 19 (The Control Plane)
Naive self-observation	Pattern 18 (The Harness as Self-Observing State Machine)

Glossary

Agent execution failure modes. Three characteristic patterns of agent misbehaviour during execution, named by Uberto Barbini at the individual-developer scale and recurring at platform scale. *Loop of death*: the agent fixes one issue and breaks another, oscillating indefinitely. *Misunderstanding the requirement*: the agent works on the wrong part of the code, often because of poor naming or duplicated logic — a problem that gets worse, not better, with AI, which makes code quality and DDD discipline more important rather than less. *Desperate changes*: when the agent cannot find a solution within the current constraints, it escalates — rewriting large parts of the system, ignoring design rules, even modifying external libraries. The corrective in all three cases is structural: detect the failure mode (cycle detection, scaffold-boundary violation, invariant violation) and intervene, rather than letting the agent continue. *See Uberto Barbini,* Process Over Magic: Beyond Vibe Coding* (2026); related to Pattern 18 (The Harness as Self-Observing State Machine) and Pattern 17 (Heuristics as Explicit Artifacts).*

Aggregate. A cluster of domain objects treated as a single unit for state changes, with invariants that hold across the cluster. Vaughn Vernon's *Implementing Domain-Driven Design* (2013) elaborates the consistency boundary discipline. *Related to Pattern 7 (The Intermediate Representation) and Pattern 11 (Transactional Boundaries).*

Alignment Record. A first-class artifact created at each architectural decision point in the modernization, capturing what was decided, by whom, against what evidence, under what constraints. Fields include reference to AsIs evidence, ToBe outcome (architectural pattern, tier, seam, module), ontology version, agents and alternatives considered, the human who approved with reasoning where overrides applied, and the attestation contract the oracle must verify. The Alignment Record is the working contract between recovery and generation, the input that feeds promotion gates in the heuristic catalog, and the evidence trail that lets regulators, auditors, or future modernization teams retrace why a decision was reached. *Related to Pattern 17 (Heuristics as Explicit Artifacts) and Pattern 19 (The Control Plane).*

Anti-corruption layer. Eric Evans' term for a translation layer that prevents legacy concepts from polluting the modernized domain model. The layer is structurally a bounded context with a single responsibility — protect the modernized side from inheriting whatever the legacy still calls things. *See Evans,* Domain-Driven Design* (2003); related to Pattern 10 (Commands and Events as Logical Boundary), Pattern 12 (Transitional Architecture), and Pattern 22 (Dual-Run Coexistence).*

Asymmetrical Validation. A synchronisation antipattern in incremental migrations where the modernized side enforces stricter validation rules than the legacy holds historically. Years of legacy data fail the modernized rules; the team faces chronic drift it cannot resolve without amnesty or backfill. *See Nick Tune, legacy-modernization.io; related to Pattern 22 (Dual-Run Coexistence).*

AsIs / ToBe ownership discipline. The third foundation across the catalog (alongside the three-

layer recovery architecture and source provenance discipline). AsIs is the recovered specification — evidence about what the legacy actually does, owned by deterministic infrastructure (parsers, pattern detectors, heuristic catalog). ToBe is the target design — judgment about how the legacy should be expressed in the modernized architecture, owned jointly by agents, the ontology, and humans. The Intermediate Representation (Pattern 7) is the bridge contract; decisions are made in ToBe, not in IR. Where the dichotomy collapses, the catalog's structural commitments collapse with it. *Introduced in the Architectural Interlude; referenced from Pattern 6 (The Compiler Principle), Pattern 13 (Twin Verification), Pattern 18 (The Harness as Self-Observing State Machine), and Pattern 19 (The Control Plane).*

Autonomous Bubble. A variant of the Bubble pattern in which the new subsystem holds its own local data store, populated asynchronously from legacy events through an anti-corruption layer. Unlike a pure Bubble, the autonomous bubble can fulfil reads without calling back into the legacy, which decouples its runtime from legacy availability and performance. *See Nick Tune, legacy-modernization.io; related to Pattern 12 (Transitional Architecture) and Pattern 10 (Commands and Events as Logical Boundary).*

Bi-directional Model Sync. A synchronisation antipattern in incremental migrations where legacy and modernized entities are kept in continuous round-trip synchronisation, with each side's invariants having to survive translation through the other's model. The arrangement is fragile and resists retirement. *See Nick Tune, legacy-modernization.io; related to Pattern 22 (Dual-Run Coexistence).*

Bounded context. DDD's term for explicit boundaries within which a particular domain model applies. Inside the boundary, vocabulary is precise and the model is consistent. Across the boundary, different bounded contexts may use the same word for different concepts. *See Eric Evans,* Domain-Driven Design* (2003); related to most patterns in the catalog.*

Bubble. A migration pattern in which a new subsystem is built with a clean target domain model and accesses legacy data exclusively through an anti-corruption layer, without local persistence and without synchronisation. The bubble eventually "pops" when the legacy data store is retired. Useful when the target model can be designed independently but the underlying data must remain in the legacy during the transition. *See Nick Tune, legacy-modernization.io; related to Pattern 12 (Transitional Architecture).*

CDC vs Application-level Events. Two distinct techniques for propagating state changes from legacy to modernized systems. *Change Data Capture (CDC)* reads transaction logs or journals at the data-store level — DB2 log readers, VSAM journaling, IMS DPROP — and emits change events without requiring legacy application code to be modified. *Application-level events* are emitted by instrumented legacy application code itself, typically as transactional outbox writes. CDC has lower legacy disruption but emits change-at-the-data-level which lacks domain context; application-level events carry richer intent but require legacy code changes. The choice is per data domain. *Related to Pattern 22 (Dual-Run Coexistence).*

Context map. A DDD artifact articulating how bounded contexts relate — which integrate, which translate, which conflict, which share kernel concepts. Eric Evans introduced the concept; Vaughn Vernon and Nick Tune have elaborated it for modern practice. *See Evans (2003), Vernon (2013), Tune & Hirth's Bounded Context Canvas; related to Pattern 3 (The Graph as Projection) and Pattern 9 (Pluggable Emitters).*

Conway's Law. Melvin Conway's 1968 observation that organizations design systems whose structure mirrors their communication structure. The original formulation: "Any organization that designs a system

(defined broadly) will produce a design whose structure is a copy of the organization's communication structure." The implication for modernization: bounded contexts that are designed without regard to team structure will be reshaped informally by team dynamics, often in ways that undermine the intended architecture. *See Conway, "How Do Committees Invent?", 1968; related to Pattern 20 (Team Topology and Bounded Context Alignment).*

Core domain. Eric Evans' classification for the subdomain that contains the strategic differentiators of the business — the parts of the system where investment compounds and where the business outperforms its competitors. *See Evans (2003); related to Pattern 1 (Business-Aligned Capability Strategy) and Pattern 8 (Tier-Aware Scaffolding).*

Distillation. Eric Evans' term for separating what is essential about the business from what is incidental about how the legacy happened to express it. Distillation is the work of recovering the canonical domain from accumulated implementation. *See Evans (2003); related to Pattern 4 (Domain Ontology as Independent Substrate).*

Domain event. A first-class artifact representing something observable that has happened in the domain — a customer registered, an order shipped, a payment settled. Events are immutable, named in past tense, and carry the data necessary to understand what happened. *See Evans (2003), Vernon (2013); related to Pattern 7 (The Intermediate Representation) and Pattern 10 (Commands and Events as Logical Boundary).*

Drifting Domain Model. Nick Tune's term for the phenomenon where the target domain model itself drifts during migration. Concepts that began as straightforward renames end up restructured as the team's understanding sharpens through contact with the legacy and with domain experts. The drift is not a failure mode but an expected consequence of learning; the modernization must accommodate it. *See Nick Tune, legacy-modernization.io; related to Pattern 4 (Domain Ontology as Independent Substrate).*

Enabling team. In the Skelton-Pais team-topology framework, a team that transfers expertise into stream-aligned teams without taking ownership. Enabling teams are time-bounded; their goal is to make themselves unnecessary. In mainframe modernization, enabling teams typically operate at the boundary between legacy operations and modernized development, transferring institutional knowledge in both directions until the modernized team can operate the system independently. *See Skelton & Pais,* Team Topologies*, 2019; related to Pattern 20 (Team Topology and Bounded Context Alignment).*

Event Storming. Alberto Brandolini's collaborative workshop technique for recovering domain understanding from systems and people. A room full of domain experts, developers, and operators map events, commands, and aggregates against shared vocabulary on a wall of sticky notes. Event Storming surfaces what no single participant knew alone. *See Brandolini,* Introducing EventStorming* (2013); related to Pattern 4 (Domain Ontology as Independent Substrate) and Pattern 5 (Vertical Slice Discovery).*

Expose Legacy Asset. Nick Tune's term for the integration pattern in which the legacy publishes events that modernized subsystems consume as their integration surface. The legacy becomes a queryable asset rather than a direct integration target. *See Nick Tune, legacy-modernization.io; related to Pattern 10 (Commands and Events as Logical Boundary) and Pattern 15 (Bounded MCP Servers).*

Faithfulness (of reasoning traces). The degree to which an LLM-emitted explanation of its reasoning reflects the actual computation that produced the decision. Research in this area (Lanham et al. on faithful

chain-of-thought; broader work from Anthropic and DeepMind on model self-explanation) suggests that chain-of-thought traces frequently rationalize post-hoc rather than faithfully report internal state. The implication for modernization: reasoning telemetry from agents is valuable as supplementary signal and as input to retrospective analysis, but should not be treated as authoritative about why a decision was reached. The authoritative signal is action-and-outcome. *Related to Pattern 18 (The Harness as Self-Observing State Machine), specifically the reasoning telemetry section.*

Generic subdomain. Eric Evans' classification for capabilities the business needs but does not differentiate on — accounting, authentication, audit logging. Generic subdomains should be solved with off-the-shelf solutions, SaaS, or minimal custom code. *See Evans (2003); related to Pattern 1 (Business-Aligned Capability Strategy) and Pattern 8 (Tier-Aware Scaffolding).*

Goodhart's Law. Charles Goodhart's observation, popularised in economics and later extended to general measurement theory: "When a measure becomes a target, it ceases to be a good measure." In agentic systems, Goodhart pressure is the default: agents evaluated against any specific measure (heuristic catalog satisfaction, gate passage rate, confidence score) will learn to optimize for the measure rather than for the underlying property the measure was meant to capture. The implication for modernization: self-observation in the harness must anticipate Goodhart pressure, distinguishing between honest signals from agents producing the right work and gamed signals from agents producing convincing artifacts of the right work without the underlying substance. *Related to Pattern 18 (The Harness as Self-Observing State Machine) and the* Naive self-observation* antipattern.*

Hexagonal architecture. Alistair Cockburn's articulation of the ports-and-adapters pattern: the domain at the centre, with explicit ports through which external actors interact and adapters that translate between external protocols and internal domain operations. *See Cockburn (2005); related to Pattern 8 (Tier-Aware Scaffolding) and Pattern 9 (Pluggable Emitters).*

IR-Domain and IR-Scaffold. The two substrates that compose the Intermediate Representation (Pattern 7), separated by the same compiler discipline that governs the boundary between agentic and deterministic work. *IR-Domain* holds architectural intent — commands, events, handlers, aggregates, sagas, side-effect surfaces — as a typed model that agents reason about. *IR-Scaffold* holds the structural blueprint — class layouts, file paths, project structure, scaffold-meta constitutional contracts — as a deterministic projection from IR-Domain that agents do not modify. Architectural decisions live in IR-Domain; rendering errors live in IR-Scaffold; conflating the two is the most common way the compiler principle silently fails. *Related to Pattern 6 (The Compiler Principle), Pattern 7 (The Intermediate Representation), and Pattern 18 (The Harness as Self-Observing State Machine).*

Migrate Reads First / Migrate Writes First. Two complementary strategies for sequencing data migration during dual-run. *Migrate Reads First* directs read traffic to the modernized side while writes continue on the legacy side; the modernized side reads through CDC-populated stores. *Migrate Writes First* directs write traffic to the modernized side; the legacy reads through reverse CDC. The choice depends on consistency requirements and which side has more sophisticated read patterns. *See microservices migration literature; related to Pattern 22 (Dual-Run Coexistence).*

Platform team. In the Skelton-Pais team-topology framework, a team that provides internal services and capabilities that stream-aligned teams consume. Platform teams reduce cognitive load on stream-aligned teams by offering well-defined services with clear interfaces. In mainframe modernization, platform teams typically own the modernization toolchain, the cloud infrastructure, the observability stack, and

the security tooling. The vendor partner providing the modernization platform (Microsoft via the AIM offering, in Rosetta's case) is structurally an external platform team. *See Skelton & Pais,* Team Topologies*, 2019; related to Pattern 20 (Team Topology and Bounded Context Alignment).*

Republishing Legacy Events. Nick Tune's term for an autonomous bubble pattern in which a modernized subsystem consumes legacy events and re-emits them in the canonical target vocabulary, allowing downstream consumers to decouple from the legacy model before the migration is complete. *See Nick Tune, legacy-modernization.io; related to Pattern 10 (Commands and Events as Logical Boundary).*

Strategic design. Eric Evans' term for the upstream DDD work of identifying bounded contexts, establishing ubiquitous language, and articulating subdomain types. Strategic design determines what the system is *about*; tactical design determines *how* the system is built. *See Evans (2003); related to Part I of this catalog as a whole.*

Stream-aligned team. In the Skelton-Pais team-topology framework, a team aligned to a flow of work — typically a business domain, a product, or a customer journey. Stream-aligned teams own one or more bounded contexts and have end-to-end responsibility for delivering business value through them. In mainframe modernization, stream-aligned teams are typically organised by business domain (claims, underwriting, billing, reporting) rather than by technical concern. *See Skelton & Pais,* Team Topologies*, 2019; related to Pattern 20 (Team Topology and Bounded Context Alignment).*

Subdomain types (core / supporting / generic). Eric Evans' three-way classification of subdomains. Core subdomains contain strategic differentiators and deserve deep investment. Supporting subdomains are necessary for the core to function and deserve appropriate investment. Generic subdomains are commodity capabilities the business needs but does not differentiate on, and should be solved with minimal custom investment. *See Evans (2003); related to Pattern 1 (Business-Aligned Capability Strategy) and Pattern 8 (Tier-Aware Scaffolding).*

Supporting subdomain. Eric Evans' classification for subdomains that are not strategic differentiators but are necessary for the core domain to function — order processing, inventory management, customer service. Supporting subdomains deserve more investment than generic ones but less than core. *See Evans (2003); related to Pattern 1 (Business-Aligned Capability Strategy) and Pattern 8 (Tier-Aware Scaffolding).*

Tactical design. Eric Evans' term for the downstream DDD work of modelling aggregates, entities, value objects, domain events, repositories, and the implementation patterns that realise strategic design. Tactical design lives within bounded contexts that strategic design has established. *See Evans (2003); related to Part II of this catalog as a whole.*

Tri-directional Sync. A synchronisation antipattern in incremental migrations where the team must build and operate synchronisation flows between three or more systems with overlapping data — typically when the modernization is overlaid on a landscape that already contains multiple legacy systems. The combinatorial explosion of synchronisation flows quickly becomes unmanageable. *See Nick Tune, legacy-modernization.io; related to Pattern 22 (Dual-Run Coexistence).*

Ubiquitous language. DDD's term for the shared vocabulary of the domain that the team and the system both speak. The ubiquitous language is established for each bounded context; the same word may mean different things in different contexts. Establishing and maintaining ubiquitous language is the foundational

work of strategic design. *See Evans (2003); related to Pattern 4 (Domain Ontology as Independent Substrate).*

Vertical slice. A unit of work that crosses multiple architectural layers to deliver a single user-visible capability. In modernization, the vertical slice typically corresponds to a use case within a bounded context, framed by the aggregates it touches. The slice is the unit of agentic translation in this catalog. *See Jimmy Bogard (2018), Steven Smith (2018); related to Pattern 5 (Vertical Slice Discovery) and Pattern 8 (Tier-Aware Scaffolding).*

For broader DDD reference, accessible entry points include Vladik Khononov's *Learning Domain-Driven Design* (2021), the DDD Crew GitHub repository, and the Bounded Context Canvas by Nick Tune and Krisztina Hirth. For deeper engagement, Eric Evans' original *Domain-Driven Design* (2003) and Vaughn Vernon's *Implementing Domain-Driven Design* (2013) remain the canonical sources.

For team-topology reference, Matthew Skelton and Manuel Pais's *Team Topologies* (2019) is the canonical source. Melvin Conway's original 1968 paper "How Do Committees Invent?" remains the foundational text for the underlying observation.

Reference implementations in Rosetta

The patterns above are written abstractly because principles outlive implementations. This section names the concrete technologies that realise each pattern in Rosetta today. It's a snapshot — as of early 2026 — and will date faster than the patterns themselves. When a technology changes, this section updates; the pattern bodies stay stable.

Only patterns with concrete implementations today appear here. Patterns marked *designed* are not yet built, so they have no reference implementation to record.

The source provenance discipline (architectural interlude) is realised across all patterns through `source_file`, `start_line`, `end_line` fields on every graph node, carried forward into IR elements, generated C# (as comments and metadata), semantic index entries, and agent reasoning records. The discipline is implementation-wide, not pattern-specific.

Pattern 2 — The Legacy as Oracle

- Legacy compiler: Raincode (compiles COBOL to .NET IL)
- Container runtime: Docker
- Local execution: developer workstation, in-process invocation

Pattern 3 — The Graph as Projection

- Graph database: Neo4j
- Schema: COBOL/CICS-specific property graph (programs, paragraphs, data structures, control flow, side effects, predicates, entry points)
- Query language: Cypher
- Ingestion: custom COBOL/CICS parser feeding the graph schema
- Semantic index: Azure AI Search
- Embedding granularity: paragraph-level and program-level
- Vocabulary inference from comments, display literals, naming conventions, IR
- Synchronization: shared ingestion pipeline updates both graph and index from the same source events

Pattern 5 — Vertical Slice Discovery from Structural and Behavioural Signals

- Structural analysis: Cypher queries over the Neo4j graph
- Behavioural signal (when available): observation telemetry from the Witness production layer
- CICS-specific anchors: `RETURN TRANSID/COMMAREA` cycles as primary slice boundary signal

Pattern 6 — The Compiler Principle

- Deterministic emitter: Roslyn SyntaxFactory

- Validation layer: architect review through Rosetta Studio (Pattern 19)
- Probabilistic layer: GitHub Copilot SDK-driven agents inside the rendered scaffold

Pattern 7 — The Intermediate Representation

- IR name: WolverineIntentModel
- Schema: typed C# classes representing commands, events, handlers, sagas, aggregates, side-effect surfaces
- Grounding: each IR element references the graph nodes that derived it

Pattern 8 — Tier-Aware Scaffolding

- Tier annotation: stored on bounded context nodes in the graph
- Scaffold variants: VSA emitter (tier 0–1), hybrid emitter (tier 2), hexagonal emitter (tier 3)
- Heuristic derivation: cyclomatic complexity, coupling, change frequency

Pattern 9 — Pluggable Emitters

- Current code emitters: vertical slice (Wolverine, C#), hexagonal (Wolverine with ports/adapters, C#), hybrid (Wolverine with lightweight domain models, C#)
- Future code emitter targets under consideration: Jade (event-sourced workloads), Java (Spring Boot or Quarkus emitter)
- Documentation emitters: *designed*, not yet built; planned to use the same Roslyn-based emitter infrastructure for diagram and markdown rendering

Pattern 10 — Commands and Events as Logical Boundary

- Framework: Wolverine for current implementations
- Transactional guarantees: Wolverine transactional outbox preserves CICS-equivalent consistency semantics
- In-process default: Wolverine handles dispatching synchronously when contexts share a process
- Distributed extension: same command/event surface flows over messaging when contexts are extracted to services

Pattern 12 — Transitional Architecture: The Modular Monolith as Migration Vehicle

- Module boundary enforcement: `internal` access modifier + assembly boundaries (first layer); NetArchTest for architecture-rule enforcement (second layer); custom Roslyn analyzers for source-level diagnostics (third layer)

Pattern 13 — Twin Verification

- Local oracle: Raincode-compiled COBOL packaged as Docker container (Legacy Twin)
- Comparison: in-process semantic comparison of outputs against the Legacy Twin
- Test framework: xUnit
- Inner loop: candidate generation → dual execution → semantic comparison → verdict in milliseconds

Pattern 14 — Hypothesis-Driven Verification

- Framework: Witness (production-mode counterpart to Twin Verification)
- Capture-replay lineage: pmilet/playback (open-source HTTP capture-replay middleware, 2016)
- MCP integration: Witness MCP Server owns the evidence lifecycle

- Behavioural specifications from production captures: *designed*, not yet built; planned as extension of Witness pattern-matching layer

Pattern 15 — Bounded MCP Servers

- Protocol: Model Context Protocol (MCP)
- SDK: MCP SDK for .NET
- Server count in current architecture: four (Discovery, Legacy, Twin, Witness)

Pattern 16 — Durable Orchestration Above Bounded Capabilities

- Orchestrating agent: Cortex (Rosetta-internal name)
- Framework: Microsoft Agent Framework (MAF)
- Retrospective layer: dedicated retrospective agent that learns across sessions
- Durable workflow infrastructure: Azure Durable Functions
- Hosted agentic platform: Azure AI Foundry
- GitHub-native gates: branch protection, required status checks, GitHub Actions, issue templates

Pattern 17 — Heuristics as Explicit Artifacts

- Heuristic catalog: queryable schema with conditions of application, evidence weights, version status, observability hooks
- Catalog hosting: served through a Discovery server tool (Pattern 15)
- Refinement loop: telemetry from Pattern 18's self-observation layer feeds catalog evolution

Pattern 18 — The Harness as Self-Observing State Machine

- State machine implementation: typed C# (Microsoft Agent Framework)
- Hooks: PreToolUse and PostToolUse hooks defined in code
- Per-project contract: scaffold-meta.json
- GitHub-native enforcement: branch protection rules, required status checks, GitHub Actions
- Self-observation telemetry: structured records of every gate evaluation, transition, escalation, cycle
- Retrospective agent: *in construction* — pattern detection over long-running workflows
- Reasoning telemetry: *in construction* — structured records alongside each agentic decision, queryable through standard observability infrastructure

Pattern 19 — The Control Plane

- Implementation: Rosetta Studio
- Surfaces: graph decisions, translation candidates, gate transitions, spec deltas, audit trail, diff views, documentation re-renders
- Spec delta format: OpenSpec (or equivalent structured format) — *designed*, integration with the harness gate model planned

Pattern 20 — Team Topology and Bounded Context Alignment

- Mapping documentation: team-context mapping stored alongside bounded context definitions in the graph
- Authority routing: harness gate metadata declares team or role required for each transition
- Control Plane routing: Studio surfaces decisions to the relevant team based on the team-context mapping

Pattern 21 — Rollout and Cutover at Bounded Context Granularity

- Routing layer: API gateway / message bus / transaction router depending on engagement
- Cutover gate orchestration: Pattern 18 harness states encode parallel run, shadow validation, incremental routing, canary monitoring, cutover, retention, decommission
- Rollback rehearsal: pre-production test of rollback procedure required before cutover gate clears

Pattern 22 — Dual-Run Coexistence: CDC, Reconciliation, and the Bridge Period

- CDC mechanisms: vary by legacy store — IBM InfoSphere CDC, Oracle GoldenGate, or Debezium-style readers for DB2; journaling for VSAM; DPROP or equivalent for IMS
- Outbox-pattern dual-write: Wolverine transactional outbox (recommended default)
- Reconciliation: continuous sampling and scheduled comparison, with remediation playbooks per data domain
- Bridge APIs: anti-corruption translation layer between legacy and modernized vocabularies

---

Closing: What the Catalog Claims

Twenty-three patterns and nine antipatterns. The catalog isn't comprehensive — it's calibrated to what Project Rosetta has surfaced over a year of building. Other patterns exist in the field; some I haven't encountered yet, some I've encountered but not yet articulated, some are being articulated by others doing parallel work.

Three claims hold the catalog together. Each is testable; each could be wrong.

The first claim is that mainframe modernization is, at its core, a Domain-Driven Design activity at scale. Strategic design — recovering the domain, identifying bounded contexts, establishing ubiquitous language — is the work that determines whether the modernization delivers business value. Tactical design — aggregates, domain events, handlers — is the work that determines whether the modernized code is maintainable. The patterns in Parts I and II operationalise both. If this claim is right, modernizations that skip the DDD work will produce technically successful systems that fail commercially. If it's wrong, modernizations can succeed through behavioural fidelity alone.

The second claim is that AI assistance changes the economics of modernization but not its discipline. What was previously infeasible at scale — recovering specifications from decade-old COBOL, generating idiomatic C# at paragraph granularity, verifying behavioural equivalence in tight inner loops — becomes tractable when agents handle the mechanical work. But the discipline that determines whether the modernization succeeds is the same as it was before agents existed: strategic design done well, tactical design grounded in canonical ontology, verification that grades against ground truth rather than against the team's own assumptions. The patterns in Parts III and IV are the discipline; the agents are the labour. If this claim is right, teams investing in agents without investing in discipline will produce faster modernizations that fail at the same rate as manual ones. If it's wrong, the agents themselves are enough.

The third claim is that harness engineering, not agent multiplication, is what makes agentic modernization work. The default instinct in the field is to multiply agents — more specialised agents, more coordination layers, swarms, meshes. The patterns above argue the opposite direction. Better harness, more deterministic constraints, more verifiable invariants, fewer agents doing more focused work inside structured cages. If this claim is right, agent armies will plateau and harness-first systems will scale. If it's wrong, the field's current direction is correct and Rosetta's approach is a local maximum.

None of these claims has been validated against a real customer engagement. That's the next phase.

What I'd want to know if I were reading this catalog rather than writing it:

- Which patterns survive contact with engagements at scale, and which collapse under the operational reality I haven't yet encountered?

- Which patterns are specific to CICS COBOL and don't generalise to PL/I, RPG, Adabas Natural, or Java legacy?
- Which patterns I've named as original are actually well-known in adjacent fields under different names, and which adjacent fields would teach me something the catalog is missing?
- Which patterns interact in ways I haven't documented? The Related patterns sections trace pairwise interactions; the network structure is something else.

The catalog has visible gaps I haven't filled. *One-shot data migration at cutover* is a pattern I've designed but deferred from this catalog — real engagements live or die on the discrete data migration at the cut moment, and what I've written so far doesn't reflect enough engagement experience to articulate it well. *Observability of the modernized system in production* — beyond Witness, beyond divergence detection, the steady-state observability story for the modernized C# itself — is a pattern this catalog gestures at through Pattern 14 but doesn't develop. *Test strategy for the modernized code* — how agent-generated tests fit, how Twin-anchored tests evolve when the legacy is decommissioned, how the test suite participates in the harness gates — is important enough to deserve its own pattern, but I don't yet have the validated experience to write it well. Future revisions of the catalog will address these.

The catalog improves through use. If you've found a pattern that works where mine doesn't, I'd want to hear about it. If you've found a pattern of mine that fails under conditions I haven't anticipated, I'd want to hear about that too. The substrate this catalog stands on — the work of Evans, Vernon, Fowler, Newman, Brandolini, Brown, Majors, Tune, Miller, Böckeler, Alcaraz, Barbini, Skelton, Pais, and many others — has accumulated across decades through exactly this kind of exchange. Adding to it is what justifies the work.

A note on what comes next. The Rosetta prototype enters customer engagements through Microsoft's SMM AIM offering during 2026. Whatever those engagements teach will revise this catalog. Some patterns will sharpen with use. Some will turn out to be specific to environments I haven't yet encountered. Some will be replaced by patterns I haven't yet found. The next version of this catalog will be honest about what changed and why.

The methodology is stack-agnostic. The conviction is not.

— Pierre